EBOOK

Michael Andre Franiatte

Wiimote/Joycon Configuration with Codes in Gamepad Libraries to Play PC Games

WiiJoy4FPS Configuration Files

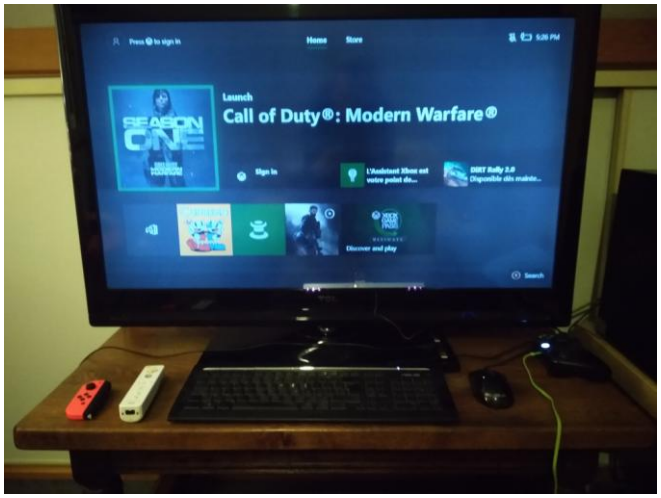
Copyright 2007-2017

EBOOK

Wiimote/Joycon Configuration with Codes in Gamepad Libraries to Play PC Games

WiiJoy4FPS Configuration Files

Michael Franiatte
22/02/2020



This book presents some configuration to play PC games using mouse and keyboard events. Information about license, EULA and contract for using these following works can be found at https://michaelfraniatte.wordpress.com.

Wiimote/Joycon Configuration with Codes in Gamepad Libraries to Play PC Games

Michael Franiatte*

Abstract

Some example is given in this book for configuration in PC games using WiiJoy4FPS interfacing with keyboard and mouse events. With this book and the book untitled "C# and C++ in Gamepad Libraries to play PC Games", by the same author, it's possible to understand and have fast practice knowledge, in order to configure easily custom programs to play your favorite PC games.

Keywords: *configuration, files, WiiJoy4FPS, mouse, keyboard, events, games*

* Author correspondence: michael.franiatte@gmail.com

1. Introduction

• Knowing how to program isn't enough when you want to make a competitive gamepad using gamepad libraries on PC. Obviously it needs you play different games and a lot, also a lot of time. The works have been achieved previously by the same author of this book, in a book giving codes to play very well with gamepads, corresponding to 6 or 7 years of works. When well designed electronically and technically, the only limit to make a competitive gamepad is how it's coded.

2. WiiJoy4FPS Configuration Files (xbox)

JoyLeft4F12019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
        public bool StickView = false, SLSRView = true;
        public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
        public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
    if (!StickView)
    {
        if (valListX.Count >= 60)
        {
            valListX.RemoveAt(0);
            valListX.Add(-DirectAnglesLeftX * 1500f);
            irx = valListXAverage(valListX);
        }
        else
        {
            valListX.Add(0);
        }
    }
    if (StickView)
       irx = (GetStickLeftY * 1500f);
    if (SLSRView)
    {
       if (LeftButtonSR & Acceleration >= -1500f)
          Acceleration -= 3f * watchM;
       if (!LeftButtonSR & Acceleration <= 0)
          Acceleration += 3f * watchM;
       if (LeftButtonSL & Breaking <= 1500f)
          Breaking += 3f * watchM;
       if (!LeftButtonSL & Breaking >= 0)
          Breaking -= 3f * watchM;
       if (Acceleration >= 0)
          Acceleration = 0;
       if (Breaking <= 0)
          Breaking = 0;
```

```
                if (LeftButtonSR & !LeftButtonSL)
                    Breaking = 0;
                if (!LeftButtonSR & LeftButtonSL)
                    Acceleration = 0;
                iry = (Acceleration + Breaking);
            }
            if (!StickView & !SLSRView)
            {
                    if (valListY.Count >= 60)
                    {
                            valListY.RemoveAt(0);
                            valListY.Add((DirectAnglesLeftY * 1500f) / 0.5f);
                            iry = valListYAverage(valListY);
                    }
                    else
                    {
                            valListY.Add(0);
                    }
            }
            if (StickView & !SLSRView)
                iry = -(GetStickLeftX * 1500f);
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
            sumX = 0f;
            foreach (double val in valList)
                sumX += val;
            return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
```

```
            sumY = 0f;
            foreach (double val in valList)
               sumY += val;
            return sumY / 60f;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
           this[3] = LeftButtonSL;
           if (_Valuechanged[3] & this[3])
              LeftClick();
```

```
if (_Valuechanged[3] & !this[3])
   LeftClickF();
this[2] = LeftButtonSR;
if (_Valuechanged[2] & this[2])
   RightClick();
if (_Valuechanged[2] & !this[2])
   RightClickF();
this[6] = LeftButtonSHOULDER_2;
if (_Valuechanged[6] & this[6])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUp(VK_G, S_G);
this[11] = LeftButtonDPAD_UP;
if (_Valuechanged[11] & this[11])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[11] & !this[11])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[22] = LeftButtonDPAD_DOWN;
if (_Valuechanged[22] & this[22])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[12] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[12] & this[12])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[12] & !this[12])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[13] = LeftButtonDPAD_LEFT;
if (_Valuechanged[13] & this[13])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = LeftButtonSTICK;
if (_Valuechanged[7] & this[7])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUp(VK_Tab, S_Tab);
this[23] = LeftButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
```

```
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[23] & !this[23])
            SimulateKeyUp(VK_T, S_T);
        this[8] = LeftButtonMINUS;
        if (_Valuechanged[8] & this[8])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[8] & !this[8])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[16] = GetStickLeftX < -0.33f;
        this[17] = GetStickLeftX > 0.33f;
        this[18] = GetStickLeftY < -0.33f;
        this[19] = GetStickLeftY > 0.33f;
        if (_Valuechanged[16] & this[16])
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[16] & !this[16])
            SimulateKeyUp(VK_S, S_S);
        if (_Valuechanged[17] & this[17])
            SimulateKeyDown(VK_Z, S_Z);
        if (_Valuechanged[17] & !this[17])
            SimulateKeyUp(VK_Z, S_Z);
        if (_Valuechanged[18] & this[18])
            SimulateKeyDown(VK_D, S_D);
        if (_Valuechanged[18] & !this[18])
            SimulateKeyUp(VK_D, S_D);
        if (_Valuechanged[19] & this[19])
            SimulateKeyDown(VK_Q, S_Q);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_Q, S_Q);
        this[24] = LeftButtonCAPTURE;
        if (_Valuechanged[24] & this[24])
            SimulateKeyDown(VK_P, S_P);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_P, S_P);
    }
```

JoyLeft4FH22019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
public bool StickView = false, SLSRView = true;
public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
public static double sumX = 0f, sumY = 0f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
    if (!StickView)
    {
        if (valListX.Count >= 60)
        {
```

```
                valListX.RemoveAt(0);
                valListX.Add(-DirectAnglesLeftX * 1500f);
                irx = valListXAverage(valListX);
            }
            else
            {
                valListX.Add(0);
            }
    }
    if (StickView)
        irx = (GetStickLeftY * 1500f);
    if (SLSRView)
    {
        if (LeftButtonSR & Acceleration >= -1500f)
            Acceleration -= 3f * watchM;
        if (!LeftButtonSR & Acceleration <= 0)
            Acceleration += 3f * watchM;
        if (LeftButtonSL & Breaking <= 1500f)
            Breaking += 3f * watchM;
        if (!LeftButtonSL & Breaking >= 0)
            Breaking -= 3f * watchM;
        if (Acceleration >= 0)
            Acceleration = 0;
        if (Breaking <= 0)
            Breaking = 0;
        if (LeftButtonSR & !LeftButtonSL)
            Breaking = 0;
        if (!LeftButtonSR & LeftButtonSL)
            Acceleration = 0;
        iry = (Acceleration + Breaking);
    }
    if (!StickView & !SLSRView)
    {
            if (valListY.Count >= 60)
            {
                valListY.RemoveAt(0);
                valListY.Add((DirectAnglesLeftY * 1500f) / 0.5f);
                iry = valListYAverage(valListY);
            }
```

```
                else
                {
                        valListY.Add(0);
                }
            }
            if (StickView & !SLSRView)
               iry = -(GetStickLeftX * 1500f);
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
            sumX = 0f;
            foreach (double val in valList)
               sumX += val;
            return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
            sumY = 0f;
            foreach (double val in valList)
               sumY += val;
            return sumY / 60f;
    }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[3] = LeftButtonSL;
   if (_Valuechanged[3] & this[3])
      LeftClick();
   if (_Valuechanged[3] & !this[3])
      LeftClickF();
   this[2] = LeftButtonSR;
   if (_Valuechanged[2] & this[2])
      RightClick();
   if (_Valuechanged[2] & !this[2])
      RightClickF();
   this[6] = LeftButtonSHOULDER_2;
   if (_Valuechanged[6] & this[6])
      SimulateKeyDown(VK_G, S_G);
   if (_Valuechanged[6] & !this[6])
      SimulateKeyUp(VK_G, S_G);
   this[11] = LeftButtonDPAD_UP;
   if (_Valuechanged[11] & this[11])
```

```
    SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[11] & !this[11])
    SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[22] = LeftButtonDPAD_DOWN;
if (_Valuechanged[22] & this[22])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[12] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[12] & this[12])
    SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[12] & !this[12])
    SimulateKeyUpArrows(VK_UP, S_UP);
this[13] = LeftButtonDPAD_LEFT;
if (_Valuechanged[13] & this[13])
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = LeftButtonSTICK;
if (_Valuechanged[7] & this[7])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUp(VK_Tab, S_Tab);
this[23] = LeftButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_T, S_T);
this[8] = LeftButtonMINUS;
if (_Valuechanged[8] & this[8])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUp(VK_Escape, S_Escape);
this[16] = GetStickLeftX < -0.33f;
this[17] = GetStickLeftX > 0.33f;
this[18] = GetStickLeftY < -0.33f;
this[19] = GetStickLeftY > 0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_S, S_S);
```

```
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUp(VK_S, S_S);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_D, S_D);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_D, S_D);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_Q, S_Q);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_Q, S_Q);
            this[24] = LeftButtonCAPTURE;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_P, S_P);
         }
```

JoyRight4F12019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning
= 100f, hardness = 300f, Acceleration, Breaking;
         public bool StickView = false, SLSRView = true;
         public static System.Collections.Generic.List<double> valListX = new
System.Collections.Generic.List<double>(), valListY = new
System.Collections.Generic.List<double>();
         public static double sumX = 0f, sumY = 0f;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
         double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
    if (!StickView)
    {
        if (valListX.Count >= 60)
        {
            valListX.RemoveAt(0);
            valListX.Add(DirectAnglesRightX * 1500f);
            irx = valListXAverage(valListX);
        }
        else
        {
            valListX.Add(0);
        }
    }
    if (StickView)
       irx = -(GetStickRightY * 1500f);
    if (SLSRView)
    {
       if (RightButtonSR & Acceleration >= -1500f)
          Acceleration -= 3f * watchM;
```

```
            if (!RightButtonSR & Acceleration <= 0)
                Acceleration += 3f * watchM;
            if (RightButtonSL & Breaking <= 1500f)
                Breaking += 3f * watchM;
            if (!RightButtonSL & Breaking >= 0)
                Breaking -= 3f * watchM;
            if (Acceleration >= 0)
                Acceleration = 0;
            if (Breaking <= 0)
                Breaking = 0;
            if (RightButtonSR & !RightButtonSL)
                Breaking = 0;
            if (!RightButtonSR & RightButtonSL)
                Acceleration = 0;
            iry = (Acceleration + Breaking);
        }
        if (!StickView & !SLSRView)
        {
                if (valListY.Count >= 60)
                {
                        valListY.RemoveAt(0);
                        valListY.Add(-((DirectAnglesRightY * 1500f) / 0.5f));
                        iry = valListYAverage(valListY);
                }
                else
                {
                        valListY.Add(0);
                }
        }
        if (StickView & !SLSRView)
            iry = (GetStickRightX * 1500f);
        mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousexp += mousex * watchM / 40f;
        mouseyp += mousey * watchM / 40f;
        input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
```

```
    }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
            sumX = 0f;
            foreach (double val in valList)
        sumX += val;
            return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
            sumY = 0f;
            foreach (double val in valList)
        sumY += val;
            return sumY / 60f;
    }
```

```
//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[3] = RightButtonSR;
            if (_Valuechanged[3] & this[3])
                LeftClick();
            if (_Valuechanged[3] & !this[3])
                LeftClickF();
            this[2] = RightButtonSL;
            if (_Valuechanged[2] & this[2])
                RightClick();
            if (_Valuechanged[2] & !this[2])
                RightClickF();
            this[6] = RightButtonSHOULDER_2;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUp(VK_G, S_G);
            this[11] = RightButtonDPAD_DOWN;
            if (_Valuechanged[11] & this[11])
                SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[11] & !this[11])
                SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[22] = RightButtonDPAD_UP;
            if (_Valuechanged[22] & this[22])
                SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[22] & !this[22])
                SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[12] = RightButtonDPAD_LEFT;
            if (_Valuechanged[12] & this[12])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[12] & !this[12])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[13] = RightButtonDPAD_RIGHT;
```

```
if (_Valuechanged[13] & this[13])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = RightButtonSTICK;
if (_Valuechanged[7] & this[7])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUp(VK_Tab, S_Tab);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_T, S_T);
this[8] = RightButtonPLUS;
if (_Valuechanged[8] & this[8])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUp(VK_Escape, S_Escape);
this[16] = GetStickRightX > 0.33f;
this[17] = GetStickRightX < -0.33f;
this[18] = GetStickRightY > 0.33f;
this[19] = GetStickRightY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_S, S_S);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_Q, S_Q);
```

```
            this[24] = RightButtonHOME;
            if (_Valuechanged[24] & this[24])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[24] & !this[24])
                SimulateKeyUp(VK_P, S_P);
        }
```

JoyRight4FH22019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
        public bool StickView = false, SLSRView = true;
        public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
        public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            if (!StickView)
            {
                if (valListX.Count >= 60)
                {
                    valListX.RemoveAt(0);
                    valListX.Add(DirectAnglesRightX * 1500f);
                    irx = valListXAverage(valListX);
                }
                else
                {
                    valListX.Add(0);
                }
            }
            if (StickView)
               irx = -(GetStickRightY * 1500f);
            if (SLSRView)
            {
               if (RightButtonSR & Acceleration >= -1500f)
                  Acceleration -= 3f * watchM;
               if (!RightButtonSR & Acceleration <= 0)
                  Acceleration += 3f * watchM;
               if (RightButtonSL & Breaking <= 1500f)
                  Breaking += 3f * watchM;
               if (!RightButtonSL & Breaking >= 0)
                  Breaking -= 3f * watchM;
               if (Acceleration >= 0)
                  Acceleration = 0;
               if (Breaking <= 0)
                  Breaking = 0;
               if (RightButtonSR & !RightButtonSL)
                  Breaking = 0;
               if (!RightButtonSR & RightButtonSL)
                  Acceleration = 0;
```

```
                iry = (Acceleration + Breaking);
            }
            if (!StickView & !SLSRView)
            {
                if (valListY.Count >= 60)
                {
                    valListY.RemoveAt(0);
                    valListY.Add(-((DirectAnglesRightY * 1500f) / 0.5f));
                    iry = valListYAverage(valListY);
                }
                else
                {
                    valListY.Add(0);
                }
            }
            if (StickView & !SLSRView)
                iry = (GetStickRightX * 1500f);
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
            sumX = 0f;
            foreach (double val in valList)
            sumX += val;
            return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
            sumY = 0f;
            foreach (double val in valList)
            sumY += val;
            return sumY / 60f;
```

```
    }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[3] = RightButtonSR;
            if (_Valuechanged[3] & this[3])
                LeftClick();
            if (_Valuechanged[3] & !this[3])
                LeftClickF();
            this[2] = RightButtonSL;
            if (_Valuechanged[2] & this[2])
```

```
    RightClick();
if (_Valuechanged[2] & !this[2])
    RightClickF();
this[6] = RightButtonSHOULDER_2;
if (_Valuechanged[6] & this[6])
    SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[6] & !this[6])
    SimulateKeyUp(VK_G, S_G);
this[11] = RightButtonDPAD_DOWN;
if (_Valuechanged[11] & this[11])
    SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[11] & !this[11])
    SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[22] = RightButtonDPAD_UP;
if (_Valuechanged[22] & this[22])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[12] = RightButtonDPAD_LEFT;
if (_Valuechanged[12] & this[12])
    SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[12] & !this[12])
    SimulateKeyUpArrows(VK_UP, S_UP);
this[13] = RightButtonDPAD_RIGHT;
if (_Valuechanged[13] & this[13])
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = RightButtonSTICK;
if (_Valuechanged[7] & this[7])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUp(VK_Tab, S_Tab);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_T, S_T);
this[8] = RightButtonPLUS;
```

```
            if (_Valuechanged[8] & this[8])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[16] = GetStickRightX > 0.33f;
            this[17] = GetStickRightX < -0.33f;
            this[18] = GetStickRightY > 0.33f;
            this[19] = GetStickRightY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUp(VK_S, S_S);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_D, S_D);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_D, S_D);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_Q, S_Q);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_Q, S_Q);
            this[24] = RightButtonHOME;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_P, S_P);
         }
```

Joys4F12019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning
= 100f, hardness = 300f, Acceleration, Breaking;
         public bool StickView = false, SLSRView = true;
```

```
public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
public static double sumX = 0f, sumY = 0f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
    if (RightButtonSHOULDER_2 & Acceleration >= -1500f)
        Acceleration -= 3f * watchM;
    if (!RightButtonSHOULDER_2 & Acceleration <= 0)
        Acceleration += 3f * watchM;
    if (LeftButtonSHOULDER_2 & Breaking <= 1500f)
        Breaking += 3f * watchM;
    if (!LeftButtonSHOULDER_2 & Breaking >= 0)
```

```
                Breaking -= 3f * watchM;
             if (Acceleration >= 0)
                Acceleration = 0;
             if (Breaking <= 0)
                Breaking = 0;
             if (RightButtonSHOULDER_2 & !LeftButtonSHOULDER_2)
                Breaking = 0;
             if (!RightButtonSHOULDER_2 & LeftButtonSHOULDER_2)
                Acceleration = 0;
            if (valListX.Count >= 60)
            {
                valListX.RemoveAt(0);
                valListX.Add(-(DirectAnglesY * 1500f));
                irx = valListXAverage(valListX) - GetStickLeftX * 1500f;
             }
             else
             {
                valListX.Add(0);
             }
             if (valListY.Count >= 60)
                {
                        valListY.RemoveAt(0);
                        valListY.Add(-(DirectAnglesX * 1500f) / 0.5f);
                        iry = valListYAverage(valListY) - GetStickLeftY * 1500f +
(Acceleration + Breaking);
                }
                else
                {
                        valListY.Add(0);
                }
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
```

```
public double valListXAverage(System.Collections.Generic.List<double> valList)
{
        sumX = 0f;
        foreach (double val in valList)
    sumX += val;
        return sumX / 60f;
}
public double valListYAverage(System.Collections.Generic.List<double> valList)
{
        sumY = 0f;
        foreach (double val in valList)
    sumY += val;
        return sumY / 60f;
}
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
    public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false, HomeFTG = false;
    public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
    double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
    float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
    float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_1;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
                SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
```

```
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
```

```
    SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
    SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
```

```
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
            if (LeftButtonSHOULDER_2io & foraorcison)
            {
                LeftButtonSHOULDER_2io = false;
                randA = false;
            }
```

```
            this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
            if (_Valuechanged[33] & this[33])
               RightClick();
            if (_Valuechanged[33] & !this[33])
               RightClickF();
         }
         else
         {
            this[34] = LeftButtonSHOULDER_2;
            if (_Valuechanged[34] & this[34])
               RightClick();
            if (_Valuechanged[34] & !this[34])
               RightClickF();
         }
         this[35] = GetStickRightX > 0.33f;
         this[36] = GetStickRightX < -0.33f;
         this[37] = GetStickRightY > 0.33f;
         this[38] = GetStickRightY < -0.33f;
         if (_Valuechanged[35] & this[35])
            SimulateKeyDown(VK_1, S_1);
         if (_Valuechanged[35] & !this[35])
            SimulateKeyUp(VK_1, S_1);
         if (_Valuechanged[36] & this[36])
            SimulateKeyDown(VK_2, S_2);
         if (_Valuechanged[36] & !this[36])
            SimulateKeyUp(VK_2, S_2);
         if (_Valuechanged[37] & this[37])
            SimulateKeyDown(VK_3, S_3);
         if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
         if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
         if (_Valuechanged[38] & !this[38])
            SimulateKeyUp(VK_4, S_4);
      }
```

Joys4FH22019.txt

```
//mouse control
```

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
        public bool StickView = false, SLSRView = true;
        public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
        public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
                if (RightButtonSHOULDER_2 & Acceleration >= -1500f)
                   Acceleration -= 3f * watchM;
                if (!RightButtonSHOULDER_2 & Acceleration <= 0)
```

```
                Acceleration += 3f * watchM;
            if (LeftButtonSHOULDER_2 & Breaking <= 1500f)
                Breaking += 3f * watchM;
            if (!LeftButtonSHOULDER_2 & Breaking >= 0)
                Breaking -= 3f * watchM;
            if (Acceleration >= 0)
                Acceleration = 0;
            if (Breaking <= 0)
                Breaking = 0;
            if (RightButtonSHOULDER_2 & !LeftButtonSHOULDER_2)
                Breaking = 0;
            if (!RightButtonSHOULDER_2 & LeftButtonSHOULDER_2)
                Acceleration = 0;
           if (valListX.Count >= 60)
           {
                valListX.RemoveAt(0);
                valListX.Add(-(DirectAnglesY * 1500f));
                irx = valListXAverage(valListX) - GetStickLeftX * 1500f;
            }
            else
            {
                valListX.Add(0);
            }
            if (valListY.Count >= 60)
                {
                        valListY.RemoveAt(0);
                        valListY.Add(-(DirectAnglesX * 1500f) / 0.5f);
                        iry = valListYAverage(valListY) - GetStickLeftY * 1500f +
(Acceleration + Breaking);
                }
                else
                {
                        valListY.Add(0);
                }
        mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousexp += mousex * watchM / 40f;
```

```
                mouseyp += mousey * watchM / 40f;
                input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
            }
        public double valListXAverage(System.Collections.Generic.List<double> valList)
        {
                sumX = 0f;
                foreach (double val in valList)
            sumX += val;
                return sumX / 60f;
        }
        public double valListYAverage(System.Collections.Generic.List<double> valList)
        {
                sumY = 0f;
                foreach (double val in valList)
            sumY += val;
                return sumY / 60f;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
            public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
```

```
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
           this[2] = LeftButtonSHOULDER_1;
           if (_Valuechanged[2] & this[2])
              SimulateKeyDown(VK_LeftControl, S_LeftControl);
           if (_Valuechanged[2] & !this[2])
              SimulateKeyUp(VK_LeftControl, S_LeftControl);
           this[3] = LeftButtonMINUS;
           if (_Valuechanged[3] & this[3])
              SimulateKeyDown(VK_Return, S_Return);
           if (_Valuechanged[3] & !this[3])
              SimulateKeyUp(VK_Return, S_Return);
           this[27] = LeftButtonCAPTURE;
           if (_Valuechanged[27] & this[27])
              SimulateKeyDown(VK_P, S_P);
           if (_Valuechanged[27] & !this[27])
              SimulateKeyUp(VK_P, S_P);
           this[5] = LeftButtonDPAD_UP;
           if (_Valuechanged[5] & this[5])
              SimulateKeyDownArrows(VK_UP, S_UP);
           if (_Valuechanged[5] & !this[5])
              SimulateKeyUpArrows(VK_UP, S_UP);
           this[6] = LeftButtonDPAD_LEFT;
           if (_Valuechanged[6] & this[6])
              SimulateKeyDownArrows(VK_LEFT, S_LEFT);
           if (_Valuechanged[6] & !this[6])
              SimulateKeyUpArrows(VK_LEFT, S_LEFT);
           this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
```

```
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
```

```
            SimulateKeyUp(VK_X, S_X);
        this[26] = RightButtonSTICK;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
```

```
            if (LeftButtonSHOULDER_2io & foraorcison)
            {
                LeftButtonSHOULDER_2io = false;
                randA = false;
            }
            this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = LeftButtonSHOULDER_2;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
        this[35] = GetStickRightX > 0.33f;
        this[36] = GetStickRightX < -0.33f;
        this[37] = GetStickRightY > 0.33f;
        this[38] = GetStickRightY < -0.33f;
        if (_Valuechanged[35] & this[35])
            SimulateKeyDown(VK_1, S_1);
        if (_Valuechanged[35] & !this[35])
            SimulateKeyUp(VK_1, S_1);
        if (_Valuechanged[36] & this[36])
            SimulateKeyDown(VK_2, S_2);
        if (_Valuechanged[36] & !this[36])
            SimulateKeyUp(VK_2, S_2);
        if (_Valuechanged[37] & this[37])
            SimulateKeyDown(VK_3, S_3);
        if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
        if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
        if (_Valuechanged[38] & !this[38])
```

```
            SimulateKeyUp(VK_4, S_4);
        }
```

Joys4HotlineMiamiCollection.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 250f, hardness = 330f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            if (RightButtonSHOULDER_1 & RightButtonSHOULDER_2)
            {
                mousexp = 0;
```

```
                mouseyp = 0;
            }
            irx = -(EulerAnglesRightX * 1500f) / 0.5f;
            iry = -(EulerAnglesRightZ * 1500f) / 0.5f;
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[1] = RightButtonSHOULDER_1 & RightButtonSHOULDER_2;
          if (_Valuechanged[1] & this[1])
            SimulateKeyDown(VK_NUMPAD0, S_NUMPAD0);
          if (_Valuechanged[1] & !this[1])
            SimulateKeyUp(VK_NUMPAD0, S_NUMPAD0);
          this[2] = LeftButtonSHOULDER_1;
          if (_Valuechanged[2] & this[2])
            SimulateKeyDown(VK_LeftControl, S_LeftControl);
          if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_LeftControl, S_LeftControl);
          this[3] = LeftButtonMINUS;
          if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_Return, S_Return);
          if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_Return, S_Return);
          this[27] = LeftButtonCAPTURE;
          if (_Valuechanged[27] & this[27])
            SimulateKeyDown(VK_P, S_P);
          if (_Valuechanged[27] & !this[27])
            SimulateKeyUp(VK_P, S_P);
          this[5] = LeftButtonDPAD_UP;
          if (_Valuechanged[5] & this[5])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[5] & !this[5])
            SimulateKeyUpArrows(VK_UP, S_UP);
          this[6] = LeftButtonDPAD_LEFT;
          if (_Valuechanged[6] & this[6])
            SimulateKeyDownArrows(VK_LEFT, S_LEFT);
          if (_Valuechanged[6] & !this[6])
            SimulateKeyUpArrows(VK_LEFT, S_LEFT);
          this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
```

```
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
```

```
            SimulateKeyUp(VK_X, S_X);
        this[26] = RightButtonSTICK;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
```

```
            if (LeftButtonSHOULDER_2io & foraorcison)
            {
                LeftButtonSHOULDER_2io = false;
                randA = false;
            }
            this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = LeftButtonSHOULDER_2;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
        this[35] = GetStickRightX > 0.33f;
        this[36] = GetStickRightX < -0.33f;
        this[37] = GetStickRightY > 0.33f;
        this[38] = GetStickRightY < -0.33f;
        if (_Valuechanged[35] & this[35])
            SimulateKeyDown(VK_1, S_1);
        if (_Valuechanged[35] & !this[35])
            SimulateKeyUp(VK_1, S_1);
        if (_Valuechanged[36] & this[36])
            SimulateKeyDown(VK_2, S_2);
        if (_Valuechanged[36] & !this[36])
            SimulateKeyUp(VK_2, S_2);
        if (_Valuechanged[37] & this[37])
            SimulateKeyDown(VK_3, S_3);
        if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
        if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
        if (_Valuechanged[38] & !this[38])
```

```
            SimulateKeyUp(VK_4, S_4);
        }
```

Joys4MrShifty.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f, hardness = 0f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = -GetStickLeftX * 1500f;
            iry = -GetStickLeftY * 1500f;
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[2] = LeftButtonSHOULDER_1;
          if (_Valuechanged[2] & this[2])
            SimulateKeyDown(VK_LeftControl, S_LeftControl);
          if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_LeftControl, S_LeftControl);
          this[3] = LeftButtonMINUS;
          if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_Return, S_Return);
          if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_Return, S_Return);
          this[27] = LeftButtonCAPTURE;
          if (_Valuechanged[27] & this[27])
            SimulateKeyDown(VK_P, S_P);
          if (_Valuechanged[27] & !this[27])
            SimulateKeyUp(VK_P, S_P);
          this[5] = LeftButtonDPAD_UP;
          if (_Valuechanged[5] & this[5])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[5] & !this[5])
            SimulateKeyUpArrows(VK_UP, S_UP);
          this[6] = LeftButtonDPAD_LEFT;
          if (_Valuechanged[6] & this[6])
            SimulateKeyDownArrows(VK_LEFT, S_LEFT);
          if (_Valuechanged[6] & !this[6])
            SimulateKeyUpArrows(VK_LEFT, S_LEFT);
          this[7] = LeftButtonDPAD_DOWN;
          if (_Valuechanged[7] & this[7])
            SimulateKeyDownArrows(VK_DOWN, S_DOWN);
          if (_Valuechanged[7] & !this[7])
            SimulateKeyUpArrows(VK_DOWN, S_DOWN);
          this[8] = LeftButtonDPAD_RIGHT;
          if (_Valuechanged[8] & this[8])
            SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
          if (_Valuechanged[8] & !this[8])
            SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
          this[9] = LeftButtonSTICK;
```

```
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
```

```
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
```

```
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
            if (LeftButtonSHOULDER_2io & foraorcison)
            {
                LeftButtonSHOULDER_2io = false;
                randA = false;
            }
            this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
```

```
            RightClickF();
        }
        else
        {
            this[34] = LeftButtonSHOULDER_2;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
        this[35] = GetStickRightX > 0.33f;
        this[36] = GetStickRightX < -0.33f;
        this[37] = GetStickRightY > 0.33f;
        this[38] = GetStickRightY < -0.33f;
        if (_Valuechanged[35] & this[35])
            SimulateKeyDown(VK_1, S_1);
        if (_Valuechanged[35] & !this[35])
            SimulateKeyUp(VK_1, S_1);
        if (_Valuechanged[36] & this[36])
            SimulateKeyDown(VK_2, S_2);
        if (_Valuechanged[36] & !this[36])
            SimulateKeyUp(VK_2, S_2);
        if (_Valuechanged[37] & this[37])
            SimulateKeyDown(VK_3, S_3);
        if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
        if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
        if (_Valuechanged[38] & !this[38])
            SimulateKeyUp(VK_4, S_4);
    }
```

Joys4NotAHero.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f,
hardness = 0f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            irx = -GetStickLeftX * 1500f;
            iry = -GetStickLeftY * 1500f;
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
```

```
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_1;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
```

```
if (_Valuechanged[3] & !this[3])
   SimulateKeyUp(VK_Return, S_Return);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
   SimulateKeyDown(VK_P, S_P);
if (_Valuechanged[27] & !this[27])
   SimulateKeyUp(VK_P, S_P);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
```

```
    SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
    SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
    SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
```

```
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
if (_Valuechanged[15] & this[15])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
this[11] = RightButtonSHOULDER_2;
if (_Valuechanged[11] & this[11])
   LeftClick();
if (_Valuechanged[11] & !this[11])
   LeftClickF();
```

```
            if (ApressIO)
            {
               foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
               this[32] = LeftButtonSHOULDER_2;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     LeftButtonSHOULDER_2io = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                     }
               if (LeftButtonSHOULDER_2io & foraorcison)
               {
                  LeftButtonSHOULDER_2io = false;
                  randA = false;
               }
               this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
               if (_Valuechanged[33] & this[33])
                  RightClick();
               if (_Valuechanged[33] & !this[33])
                  RightClickF();
            }
            else
            {
               this[34] = LeftButtonSHOULDER_2;
               if (_Valuechanged[34] & this[34])
                  RightClick();
               if (_Valuechanged[34] & !this[34])
                  RightClickF();
            }
            this[35] = GetStickRightX > 0.33f;
```

```
            this[36] = GetStickRightX < -0.33f;
            this[37] = GetStickRightY > 0.33f;
            this[38] = GetStickRightY < -0.33f;
            if (_Valuechanged[35] & this[35])
               SimulateKeyDown(VK_1, S_1);
            if (_Valuechanged[35] & !this[35])
               SimulateKeyUp(VK_1, S_1);
            if (_Valuechanged[36] & this[36])
               SimulateKeyDown(VK_2, S_2);
            if (_Valuechanged[36] & !this[36])
               SimulateKeyUp(VK_2, S_2);
            if (_Valuechanged[37] & this[37])
               SimulateKeyDown(VK_3, S_3);
            if (_Valuechanged[37] & !this[37])
               SimulateKeyUp(VK_3, S_3);
            if (_Valuechanged[38] & this[38])
               SimulateKeyDown(VK_4, S_4);
            if (_Valuechanged[38] & !this[38])
               SimulateKeyUp(VK_4, S_4);
         }
```

Joys4Ori2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f,
hardness = 0f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   irx = -GetStickLeftX * 1500f;
   iry = -GetStickLeftY * 1500f;
   mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
   mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
   mousexp += mousex * watchM / 40f;
   mouseyp += mousey * watchM / 40f;
   input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
}
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
```

public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false, HomeFTG = false;

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[2] = LeftButtonSHOULDER_1;
   if (_Valuechanged[2] & this[2])
      SimulateKeyDown(VK_LeftControl, S_LeftControl);
   if (_Valuechanged[2] & !this[2])
      SimulateKeyUp(VK_LeftControl, S_LeftControl);
   this[3] = LeftButtonMINUS;
   if (_Valuechanged[3] & this[3])
      SimulateKeyDown(VK_Return, S_Return);
   if (_Valuechanged[3] & !this[3])
      SimulateKeyUp(VK_Return, S_Return);
   this[27] = LeftButtonCAPTURE;
   if (_Valuechanged[27] & this[27])
      SimulateKeyDown(VK_P, S_P);
   if (_Valuechanged[27] & !this[27])
      SimulateKeyUp(VK_P, S_P);
   this[5] = LeftButtonDPAD_UP;
   if (_Valuechanged[5] & this[5])
      SimulateKeyDownArrows(VK_UP, S_UP);
   if (_Valuechanged[5] & !this[5])
```

```
    SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
    SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
```

```
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
```

```
            SimulateKeyDown(VK_Y, S_Y);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_Y, S_Y);
        this[25] = RightButtonDPAD_UP;
        if (_Valuechanged[25] & this[25])
            SimulateKeyDown(VK_X, S_X);
        if (_Valuechanged[25] & !this[25])
            SimulateKeyUp(VK_X, S_X);
        this[26] = RightButtonSTICK;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
```

```
            }
            else
               if (randA)
               {
                  LeftButtonSHOULDER_2io = false;
                  randA = false;
               }
         if (LeftButtonSHOULDER_2io & foraorcison)
         {
            LeftButtonSHOULDER_2io = false;
            randA = false;
         }
         this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
         if (_Valuechanged[33] & this[33])
            RightClick();
         if (_Valuechanged[33] & !this[33])
            RightClickF();
      }
      else
      {
         this[34] = LeftButtonSHOULDER_2;
         if (_Valuechanged[34] & this[34])
            RightClick();
         if (_Valuechanged[34] & !this[34])
            RightClickF();
      }
      this[35] = GetStickRightX > 0.33f;
      this[36] = GetStickRightX < -0.33f;
      this[37] = GetStickRightY > 0.33f;
      this[38] = GetStickRightY < -0.33f;
      if (_Valuechanged[35] & this[35])
         SimulateKeyDown(VK_1, S_1);
      if (_Valuechanged[35] & !this[35])
         SimulateKeyUp(VK_1, S_1);
      if (_Valuechanged[36] & this[36])
         SimulateKeyDown(VK_2, S_2);
      if (_Valuechanged[36] & !this[36])
         SimulateKeyUp(VK_2, S_2);
```

```
            if (_Valuechanged[37] & this[37])
                SimulateKeyDown(VK_3, S_3);
            if (_Valuechanged[37] & !this[37])
                SimulateKeyUp(VK_3, S_3);
            if (_Valuechanged[38] & this[38])
                SimulateKeyDown(VK_4, S_4);
            if (_Valuechanged[38] & !this[38])
                SimulateKeyUp(VK_4, S_4);
        }
```

Joys4ShantaeHalfGenieHero.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f, hardness = 0f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = -GetStickLeftX * 1500f;
          iry = -GetStickLeftY * 1500f;
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) / Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 40f;
          mouseyp += mousey * watchM / 40f;
          input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
      this[2] = LeftButtonSHOULDER_1;
      if (_Valuechanged[2] & this[2])
        SimulateKeyDown(VK_LeftControl, S_LeftControl);
      if (_Valuechanged[2] & !this[2])
        SimulateKeyUp(VK_LeftControl, S_LeftControl);
      this[3] = LeftButtonMINUS;
      if (_Valuechanged[3] & this[3])
        SimulateKeyDown(VK_Return, S_Return);
      if (_Valuechanged[3] & !this[3])
        SimulateKeyUp(VK_Return, S_Return);
      this[27] = LeftButtonCAPTURE;
      if (_Valuechanged[27] & this[27])
        SimulateKeyDown(VK_P, S_P);
      if (_Valuechanged[27] & !this[27])
        SimulateKeyUp(VK_P, S_P);
      this[5] = LeftButtonDPAD_UP;
      if (_Valuechanged[5] & this[5])
        SimulateKeyDownArrows(VK_UP, S_UP);
      if (_Valuechanged[5] & !this[5])
        SimulateKeyUpArrows(VK_UP, S_UP);
      this[6] = LeftButtonDPAD_LEFT;
      if (_Valuechanged[6] & this[6])
        SimulateKeyDownArrows(VK_LEFT, S_LEFT);
      if (_Valuechanged[6] & !this[6])
        SimulateKeyUpArrows(VK_LEFT, S_LEFT);
      this[7] = LeftButtonDPAD_DOWN;
      if (_Valuechanged[7] & this[7])
        SimulateKeyDownArrows(VK_DOWN, S_DOWN);
      if (_Valuechanged[7] & !this[7])
```

```
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
```

```
    SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
    SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
```

```
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = RightButtonSHOULDER_2;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
            this[32] = LeftButtonSHOULDER_2;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    LeftButtonSHOULDER_2io = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
            if (LeftButtonSHOULDER_2io & foraorcison)
            {
                LeftButtonSHOULDER_2io = false;
```

```
                randA = false;
            }
            this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = LeftButtonSHOULDER_2;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
        this[35] = GetStickRightX > 0.33f;
        this[36] = GetStickRightX < -0.33f;
        this[37] = GetStickRightY > 0.33f;
        this[38] = GetStickRightY < -0.33f;
        if (_Valuechanged[35] & this[35])
            SimulateKeyDown(VK_1, S_1);
        if (_Valuechanged[35] & !this[35])
            SimulateKeyUp(VK_1, S_1);
        if (_Valuechanged[36] & this[36])
            SimulateKeyDown(VK_2, S_2);
        if (_Valuechanged[36] & !this[36])
            SimulateKeyUp(VK_2, S_2);
        if (_Valuechanged[37] & this[37])
            SimulateKeyDown(VK_3, S_3);
        if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
        if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
        if (_Valuechanged[38] & !this[38])
            SimulateKeyUp(VK_4, S_4);
    }
```

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 27f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max
/ Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
```

```
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[2] = LeftButtonSHOULDER_2;
          if (_Valuechanged[2] & this[2])
            SimulateKeyDown(VK_LeftControl, S_LeftControl);
          if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_LeftControl, S_LeftControl);
          this[3] = LeftButtonMINUS;
          if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_Return, S_Return);
          if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_Return, S_Return);
          this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
          if (_Valuechanged[4] & this[4])
            SimulateKeyDown(VK_V, S_V);
          if (_Valuechanged[4] & !this[4])
            SimulateKeyUp(VK_V, S_V);
          this[27] = LeftButtonCAPTURE;
          if (_Valuechanged[27] & this[27])
            SimulateKeyDown(VK_P, S_P);
          if (_Valuechanged[27] & !this[27])
            SimulateKeyUp(VK_P, S_P);
          this[5] = LeftButtonDPAD_UP;
          if (_Valuechanged[5] & this[5])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[5] & !this[5])
            SimulateKeyUpArrows(VK_UP, S_UP);
```

```
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               SimulateKeyDown(VK_D, S_D);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUp(VK_D, S_D);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDown(VK_Q, S_Q);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_S, S_S);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               SimulateKeyDown(VK_A, S_A);
            if (_Valuechanged[30] & !this[30])
               SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
```

```
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  RightClick();
               if (_Valuechanged[33] & !this[33])
                  RightClickF();
            }
            else
            {
```

```
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               RightClick();
            if (_Valuechanged[34] & !this[34])
               RightClickF();
         }
      }
```

WiiJoy4BO3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning = 250f, hardness = 330f, center = 200f, max = 1900f;

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = true;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
    this[2] = LeftButtonSHOULDER_2;
    if (_Valuechanged[2] & this[2])
        SimulateKeyDown(VK_LeftControl, S_LeftControl);
    if (_Valuechanged[2] & !this[2])
        SimulateKeyUp(VK_LeftControl, S_LeftControl);
    this[3] = LeftButtonMINUS;
    if (_Valuechanged[3] & this[3])
        SimulateKeyDown(VK_Return, S_Return);
    if (_Valuechanged[3] & !this[3])
        SimulateKeyUp(VK_Return, S_Return);
    this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
    if (_Valuechanged[4] & this[4])
        SimulateKeyDown(VK_V, S_V);
    if (_Valuechanged[4] & !this[4])
        SimulateKeyUp(VK_V, S_V);
    this[27] = LeftButtonCAPTURE;
```

```
if (_Valuechanged[27] & this[27])
   SimulateKeyDown(VK_P, S_P);
if (_Valuechanged[27] & !this[27])
   SimulateKeyUp(VK_P, S_P);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
```

```
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               SimulateKeyDown(VK_N, S_N);
            if (_Valuechanged[28] & !this[28])
               SimulateKeyUp(VK_N, S_N);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SimulateKeyDown(VK_D, S_D);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUp(VK_D, S_D);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDown(VK_Q, S_Q);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_S, S_S);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               SimulateKeyDown(VK_A, S_A);
            if (_Valuechanged[30] & !this[30])
               SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
```

```
    SimulateKeyDown(VK_R, S_R);
if (_Valuechanged[13] & !this[13])
    SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
    SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
               SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[14] & !this[14])
               SimulateKeyUp(VK_G, S_G);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
```

```
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4Borderlands32019.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 18f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
```

```
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
           return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
```

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
  this[2] = LeftButtonSHOULDER_2;
  if (_Valuechanged[2] & this[2])
    SimulateKeyDown(VK_LeftControl, S_LeftControl);
  if (_Valuechanged[2] & !this[2])
    SimulateKeyUp(VK_LeftControl, S_LeftControl);
  this[3] = LeftButtonMINUS;
  if (_Valuechanged[3] & this[3])
    SimulateKeyDown(VK_Return, S_Return);
  if (_Valuechanged[3] & !this[3])
    SimulateKeyUp(VK_Return, S_Return);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[9] & !this[9])
               SimulateKeyUp(VK_LeftShift, S_LeftShift);
            this[10] = LeftButtonSHOULDER_1;
```

```
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
```

```
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[31] & !this[31])
   SimulateKeyUp(VK_E, S_E);
this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
if (_Valuechanged[13] & this[13])
   SimulateKeyDown(VK_R, S_R);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
```

```
            SimulateKeyDown(VK_X, S_X);
        if (_Valuechanged[25] & !this[25])
            SimulateKeyUp(VK_X, S_X);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
```

```
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4DoomEternal.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 22f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 40f;

mouseyp += mousey * watchM / 40f;

input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

{

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
```

```
            SimulateKeyDown(VK_LeftControl, S_LeftControl);
        if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_LeftControl, S_LeftControl);
        this[3] = LeftButtonMINUS;
        if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_Return, S_Return);
        if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_Return, S_Return);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            SimulateKeyDown(VK_V, S_V);
        if (_Valuechanged[4] & !this[4])
            SimulateKeyUp(VK_V, S_V);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            SimulateKeyDown(VK_P, S_P);
        if (_Valuechanged[27] & !this[27])
            SimulateKeyUp(VK_P, S_P);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            SimulateKeyDownArrows(VK_UP, S_UP);
        if (_Valuechanged[5] & !this[5])
            SimulateKeyUpArrows(VK_UP, S_UP);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            SimulateKeyDownArrows(VK_LEFT, S_LEFT);
        if (_Valuechanged[6] & !this[6])
            SimulateKeyUpArrows(VK_LEFT, S_LEFT);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            SimulateKeyDownArrows(VK_DOWN, S_DOWN);
        if (_Valuechanged[7] & !this[7])
            SimulateKeyUpArrows(VK_DOWN, S_DOWN);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
```

```
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
```

```
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_S, S_S);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            SimulateKeyDown(VK_A, S_A);
        if (_Valuechanged[30] & !this[30])
            SimulateKeyUp(VK_A, S_A);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
            SimulateKeyUp(VK_R, S_R);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            SimulateKeyDown(VK_Tab, S_Tab);
        if (_Valuechanged[20] & !this[20])
            SimulateKeyUp(VK_Tab, S_Tab);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            SimulateKeyDown(VK_C, S_C);
        if (_Valuechanged[21] & !this[21])
            SimulateKeyUp(VK_C, S_C);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            SimulateKeyDown(VK_F, S_F);
        if (_Valuechanged[22] & !this[22])
            SimulateKeyUp(VK_F, S_F);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            SimulateKeyDown(VK_U, S_U);
        if (_Valuechanged[23] & !this[23])
```

```
            SimulateKeyUp(VK_U, S_U);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            SimulateKeyDown(VK_Y, S_Y);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_Y, S_Y);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            SimulateKeyDown(VK_X, S_X);
        if (_Valuechanged[25] & !this[25])
            SimulateKeyUp(VK_X, S_X);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
```

```
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4EnterTheGungeon.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning
= 100f, hardness = 300f, center = 200f, max = 1500f;
```

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 40f;

```
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
```

```
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_S, S_S);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               SimulateKeyDown(VK_A, S_A);
            if (_Valuechanged[30] & !this[30])
               SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               SimulateKeyDown(VK_C, S_C);
            if (_Valuechanged[21] & !this[21])
               SimulateKeyUp(VK_C, S_C);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
    SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
    SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
    SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
    SimulateKeyUp(VK_T, S_T);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
    LeftClick();
if (_Valuechanged[11] & !this[11])
    LeftClickF();
if (ApressIO)
```

```
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
```

```
        }
```

WiiJoy4FarCry5.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 17f, zoning = 290f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max
/ Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
```

```
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[31] & !this[31])
   SimulateKeyUp(VK_E, S_E);
this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
if (_Valuechanged[13] & this[13])
   SimulateKeyDown(VK_R, S_R);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
```

```
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
    SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
    SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[15] & this[15])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  RightClick();
               if (_Valuechanged[33] & !this[33])
```

```
            RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4FarCryNewDawn.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 17f, zoning
= 290f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
           return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
    this[2] = LeftButtonSHOULDER_2;
    if (_Valuechanged[2] & this[2])
        SimulateKeyDown(VK_LeftControl, S_LeftControl);
    if (_Valuechanged[2] & !this[2])
        SimulateKeyUp(VK_LeftControl, S_LeftControl);
    this[3] = LeftButtonMINUS;
    if (_Valuechanged[3] & this[3])
        SimulateKeyDown(VK_Return, S_Return);
    if (_Valuechanged[3] & !this[3])
        SimulateKeyUp(VK_Return, S_Return);
    this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
    if (_Valuechanged[4] & this[4])
        SimulateKeyDown(VK_V, S_V);
    if (_Valuechanged[4] & !this[4])
```

```
    SimulateKeyUp(VK_V, S_V);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    SimulateKeyDown(VK_P, S_P);
if (_Valuechanged[27] & !this[27])
    SimulateKeyUp(VK_P, S_P);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
    SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
    SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_B, S_B);
```

```
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[31] & !this[31])
   SimulateKeyUp(VK_E, S_E);
```

```
this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
if (_Valuechanged[13] & this[13])
   SimulateKeyDown(VK_R, S_R);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
```

```
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
```

```
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4Fortnite2019.txt

```
//mouse control
```

public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];

public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 22f, zoning = 250f, hardness = 300f, center = 200f, max = 1900f;

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
           return scaled;
        }

//keyboard control
```

```
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  this[2] = LeftButtonSHOULDER_2;
  if (_Valuechanged[2] & this[2])
    SimulateKeyDown(VK_LeftControl, S_LeftControl);
  if (_Valuechanged[2] & !this[2])
    SimulateKeyUp(VK_LeftControl, S_LeftControl);
  this[3] = LeftButtonMINUS;
  if (_Valuechanged[3] & this[3])
    SimulateKeyDown(VK_Return, S_Return);
```

```
            if (_Valuechanged[3] & !this[3])
               SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[9] & !this[9])
```

```
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
    SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
    SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    SimulateKeyDown(VK_A, S_A);
```

```
            if (_Valuechanged[30] & !this[30])
               SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               SimulateKeyDown(VK_C, S_C);
            if (_Valuechanged[21] & !this[21])
               SimulateKeyUp(VK_C, S_C);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               SimulateKeyDown(VK_F, S_F);
            if (_Valuechanged[22] & !this[22])
               SimulateKeyUp(VK_F, S_F);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               SimulateKeyDown(VK_U, S_U);
            if (_Valuechanged[23] & !this[23])
               SimulateKeyUp(VK_U, S_U);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_Y, S_Y);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_Y, S_Y);
```

```
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               SimulateKeyDown(VK_X, S_X);
            if (_Valuechanged[25] & !this[25])
               SimulateKeyUp(VK_X, S_X);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[26] & !this[26])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[14] & !this[14])
               SimulateKeyUp(VK_G, S_G);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
```

```
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
        }
```

WiiJoy4Hitman2.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 15f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 40f;

mouseyp += mousey * watchM / 40f;

input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
```

```
            if (_Valuechanged[2] & this[2])
               SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
               SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
               SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
    SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_Z, S_Z);
```

```
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_S, S_S);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               SimulateKeyDown(VK_A, S_A);
            if (_Valuechanged[30] & !this[30])
               SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               SimulateKeyDown(VK_C, S_C);
            if (_Valuechanged[21] & !this[21])
               SimulateKeyUp(VK_C, S_C);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               SimulateKeyDown(VK_F, S_F);
            if (_Valuechanged[22] & !this[22])
               SimulateKeyUp(VK_F, S_F);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               SimulateKeyDown(VK_U, S_U);
```

```
            if (_Valuechanged[23] & !this[23])
               SimulateKeyUp(VK_U, S_U);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_Y, S_Y);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_Y, S_Y);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               SimulateKeyDown(VK_X, S_X);
            if (_Valuechanged[25] & !this[25])
               SimulateKeyUp(VK_X, S_X);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[26] & !this[26])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[14] & !this[14])
               SimulateKeyUp(VK_G, S_G);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
        }
```

WiiJoy4HotlineMiamiCollection.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning = 100f, hardness = 300f, center = 200f, max = 1500f;

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

```
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
```

```
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
```

```
            SimulateKeyDown(VK_D, S_D);
        if (_Valuechanged[16] & !this[16])
            SimulateKeyUp(VK_D, S_D);
        if (_Valuechanged[17] & this[17])
            SimulateKeyDown(VK_Q, S_Q);
        if (_Valuechanged[17] & !this[17])
            SimulateKeyUp(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
            SimulateKeyDown(VK_Z, S_Z);
        if (_Valuechanged[18] & !this[18])
            SimulateKeyUp(VK_Z, S_Z);
        if (_Valuechanged[19] & this[19])
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_S, S_S);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            SimulateKeyDown(VK_A, S_A);
        if (_Valuechanged[30] & !this[30])
            SimulateKeyUp(VK_A, S_A);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
            SimulateKeyUp(VK_R, S_R);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            SimulateKeyDown(VK_Tab, S_Tab);
        if (_Valuechanged[20] & !this[20])
            SimulateKeyUp(VK_Tab, S_Tab);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
```

```
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
    SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
    SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
    SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
    SimulateKeyUp(VK_T, S_T);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  RightClick();
               if (_Valuechanged[33] & !this[33])
                  RightClickF();
            }
            else
            {
               this[34] = mWSButtonStateA;
```

```
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4MW2019.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 15f, zoning
= 300f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max
/ Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
           return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[2] = LeftButtonSHOULDER_2;
   if (_Valuechanged[2] & this[2])
      SimulateKeyDown(VK_LeftControl, S_LeftControl);
   if (_Valuechanged[2] & !this[2])
      SimulateKeyUp(VK_LeftControl, S_LeftControl);
   this[3] = LeftButtonMINUS;
   if (_Valuechanged[3] & this[3])
      SimulateKeyDown(VK_Return, S_Return);
   if (_Valuechanged[3] & !this[3])
      SimulateKeyUp(VK_Return, S_Return);
   this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
   if (_Valuechanged[4] & this[4])
      SimulateKeyDown(VK_V, S_V);
   if (_Valuechanged[4] & !this[4])
      SimulateKeyUp(VK_V, S_V);
   this[27] = LeftButtonCAPTURE;
   if (_Valuechanged[27] & this[27])
      SimulateKeyDown(VK_P, S_P);
```

```
if (_Valuechanged[27] & !this[27])
   SimulateKeyUp(VK_P, S_P);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
```

```
            SimulateKeyDown(VK_N, S_N);
        if (_Valuechanged[28] & !this[28])
            SimulateKeyUp(VK_N, S_N);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            SimulateKeyDown(VK_D, S_D);
        if (_Valuechanged[16] & !this[16])
            SimulateKeyUp(VK_D, S_D);
        if (_Valuechanged[17] & this[17])
            SimulateKeyDown(VK_Q, S_Q);
        if (_Valuechanged[17] & !this[17])
            SimulateKeyUp(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
            SimulateKeyDown(VK_Z, S_Z);
        if (_Valuechanged[18] & !this[18])
            SimulateKeyUp(VK_Z, S_Z);
        if (_Valuechanged[19] & this[19])
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_S, S_S);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            SimulateKeyDown(VK_A, S_A);
        if (_Valuechanged[30] & !this[30])
            SimulateKeyUp(VK_A, S_A);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
```

```
    SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
    SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
    SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
    SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    SimulateKeyDown(VK_G, S_G);
```

```
            if (_Valuechanged[14] & !this[14])
               SimulateKeyUp(VK_G, S_G);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4PUBG.txt

```
//mouse control
```

public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];

public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 24f, zoning = 250f, hardness = 300f, center = 200f, max = 1900f;

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
           return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[9] & !this[9])
               SimulateKeyUp(VK_LeftShift, S_LeftShift);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               SimulateKeyDown(VK_Space, S_Space);
```

```
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
            SimulateKeyUp(VK_R, S_R);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            SimulateKeyDown(VK_Tab, S_Tab);
        if (_Valuechanged[20] & !this[20])
            SimulateKeyUp(VK_Tab, S_Tab);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            SimulateKeyDown(VK_C, S_C);
        if (_Valuechanged[21] & !this[21])
            SimulateKeyUp(VK_C, S_C);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            SimulateKeyDown(VK_F, S_F);
        if (_Valuechanged[22] & !this[22])
            SimulateKeyUp(VK_F, S_F);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            SimulateKeyDown(VK_U, S_U);
        if (_Valuechanged[23] & !this[23])
            SimulateKeyUp(VK_U, S_U);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            SimulateKeyDown(VK_Y, S_Y);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_Y, S_Y);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            SimulateKeyDown(VK_X, S_X);
        if (_Valuechanged[25] & !this[25])
```

```
            SimulateKeyUp(VK_X, S_X);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
```

```
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4TheDivision2.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
    irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
    iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
    mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
    mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
    mousexp += mousex * watchM / 40f;
    mouseyp += mousey * watchM / 40f;
    input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
}
private double Scale(double value, double min, double max, double minScale, double maxScale)
{
   double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
   return scaled;
```

```
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
```

```
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
               SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[9] = LeftButtonSTICK;
```

```
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
```

```
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[31] & !this[31])
   SimulateKeyUp(VK_E, S_E);
this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
if (_Valuechanged[13] & this[13])
   SimulateKeyDown(VK_R, S_R);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUp(VK_R, S_R);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
            SimulateKeyDown(VK_Y, S_Y);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_Y, S_Y);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            SimulateKeyDown(VK_X, S_X);
        if (_Valuechanged[25] & !this[25])
            SimulateKeyUp(VK_X, S_X);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
        if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
        if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
        if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            LeftClick();
        if (_Valuechanged[11] & !this[11])
            LeftClickF();
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
```

```
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
        }
```

WiiJoy4ZombieArmy.txt

```
//mouse control
        public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 27f, zoning
= 250f, hardness = 300f, center = 200f, max = 1900f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 40f;

mouseyp += mousey * watchM / 40f;

```
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 12);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
        public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
```

```
            SimulateKeyUp(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
            SimulateKeyDown(VK_Z, S_Z);
        if (_Valuechanged[18] & !this[18])
            SimulateKeyUp(VK_Z, S_Z);
        if (_Valuechanged[19] & this[19])
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_S, S_S);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            SimulateKeyDown(VK_A, S_A);
        if (_Valuechanged[30] & !this[30])
            SimulateKeyUp(VK_A, S_A);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
            SimulateKeyUp(VK_R, S_R);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            SimulateKeyDown(VK_Tab, S_Tab);
        if (_Valuechanged[20] & !this[20])
            SimulateKeyUp(VK_Tab, S_Tab);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            SimulateKeyDown(VK_C, S_C);
        if (_Valuechanged[21] & !this[21])
            SimulateKeyUp(VK_C, S_C);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            SimulateKeyDown(VK_F, S_F);
```

```
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   LeftClick();
if (_Valuechanged[11] & !this[11])
   LeftClickF();
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  RightClick();
               if (_Valuechanged[33] & !this[33])
                  RightClickF();
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  RightClick();
               if (_Valuechanged[34] & !this[34])
                  RightClickF();
            }
         }
```

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
        public bool SLSRView = true;
        public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
        public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
if (valListX.Count >= 60)
{
        valListX.RemoveAt(0);
        valListX.Add(mWSRawValuesY * 45f);
        irx = valListXAverage(valListX);
}
else
{
        valListX.Add(0);
}
if (SLSRView)
{
   if (mWSButtonStateTwo & Acceleration >= -1500f)
      Acceleration -= 3f * watchM;
   if (!mWSButtonStateTwo & Acceleration <= 0)
      Acceleration += 3f * watchM;
   if (mWSButtonStateOne & Breaking <= 1500f)
      Breaking += 3f * watchM;
   if (!mWSButtonStateOne & Breaking >= 0)
      Breaking -= 3f * watchM;
   if (Acceleration >= 0)
      Acceleration = 0;
   if (Breaking <= 0)
      Breaking = 0;
   if (mWSButtonStateTwo & !mWSButtonStateOne)
      Breaking = 0;
   if (!mWSButtonStateTwo & mWSButtonStateOne)
      Acceleration = 0;
   iry = (Acceleration + Breaking);
}
if (!SLSRView)
{
        if (valListY.Count >= 60)
        {
                valListY.RemoveAt(0);
                valListY.Add((mWSRawValuesX * 45f) / 0.5f);
                iry = valListYAverage(valListY);
        }
        else
```

```
                {
                    valListY.Add(0);
                }
            }
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
        sumX = 0f;
        foreach (double val in valList)
            sumX += val;
        return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
        sumY = 0f;
        foreach (double val in valList)
            sumY += val;
        return sumY / 60f;
    }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            this[13] = (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >=
24f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_E, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_D, 1);
            this[22] = mWSButtonStateHome;
```

```
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_Z, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_S, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_Q, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus;
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus;
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_A, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_A, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
```

Wiimote4FH22019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 20f, zoning = 100f, hardness = 300f, Acceleration, Breaking;
        public bool SLSRView = true;
        public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
        public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            if (valListX.Count >= 60)
            {
                    valListX.RemoveAt(0);
                    valListX.Add(mWSRawValuesY * 45f);
                    irx = valListXAverage(valListX);
            }
            else
            {
                    valListX.Add(0);
            }
            if (SLSRView)
            {
               if (mWSButtonStateTwo & Acceleration >= -1500f)
                  Acceleration -= 3f * watchM;
               if (!mWSButtonStateTwo & Acceleration <= 0)
                  Acceleration += 3f * watchM;
               if (mWSButtonStateOne & Breaking <= 1500f)
                  Breaking += 3f * watchM;
               if (!mWSButtonStateOne & Breaking >= 0)
                  Breaking -= 3f * watchM;
               if (Acceleration >= 0)
                  Acceleration = 0;
               if (Breaking <= 0)
                  Breaking = 0;
               if (mWSButtonStateTwo & !mWSButtonStateOne)
                  Breaking = 0;
               if (!mWSButtonStateTwo & mWSButtonStateOne)
                  Acceleration = 0;
               iry = (Acceleration + Breaking);
            }
            if (!SLSRView)
            {
                    if (valListY.Count >= 60)
```

```
                {
                        valListY.RemoveAt(0);
                        valListY.Add((mWSRawValuesX * 45f) / 0.5f);
                        iry = valListYAverage(valListY);
                }
                else
                {
                        valListY.Add(0);
                }
            }
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 40f;
            mouseyp += mousey * watchM / 40f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }
    public double valListXAverage(System.Collections.Generic.List<double> valList)
    {
            sumX = 0f;
            foreach (double val in valList)
                sumX += val;
            return sumX / 60f;
    }
    public double valListYAverage(System.Collections.Generic.List<double> valList)
    {
            sumY = 0f;
            foreach (double val in valList)
                sumY += val;
            return sumY / 60f;
    }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public int keys123, keys456;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[13] = (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >=
24f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_E, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
```

```
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_D, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_Z, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_S, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_Q, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus;
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus;
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_A, 1);
if (_Valuechanged[15] & !this[15])
```

```
            input.SendKeyF((Keys)S_A, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
```

3. WiiJoy4FPS Configuration Files (pc)

JoyLeft4BackToBed.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        { }
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[3] = LeftButtonSR;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Space, S_Space);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Space, S_Space);
            this[2] = LeftButtonSL;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_Return, S_Return);
            this[6] = LeftButtonSHOULDER_2;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUp(VK_G, S_G);
            this[11] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[11] & this[11])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[11] & !this[11])
```

```
    SimulateKeyUp(VK_P, S_P);
this[22] = LeftButtonDPAD_UP;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
    SimulateKeyUp(VK_F, S_F);
this[12] = LeftButtonDPAD_LEFT;
if (_Valuechanged[12] & this[12])
    SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[12] & !this[12])
    SimulateKeyUpArrows(VK_UP, S_UP);
this[13] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[13] & this[13])
    SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
    SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = LeftButtonSTICK;
if (_Valuechanged[7] & this[7])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
    SimulateKeyUp(VK_Tab, S_Tab);
this[23] = LeftButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
    SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[23] & !this[23])
    SimulateKeyUp(VK_T, S_T);
this[8] = LeftButtonMINUS;
if (_Valuechanged[8] & this[8])
    SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[8] & !this[8])
    SimulateKeyUp(VK_Escape, S_Escape);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_W, S_W);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_W, S_W);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_S, S_S);
if (_Valuechanged[18] & this[18])
    SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_A, S_A);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[19] & !this[19])
```

```
            SimulateKeyUp(VK_D, S_D);
        this[24] = LeftButtonCAPTURE;
        if (_Valuechanged[24] & this[24])
            SimulateKeyDown(VK_P, S_P);
        if (_Valuechanged[24] & !this[24])
            SimulateKeyUp(VK_P, S_P);
    }
```

JoyLeft4Flashback.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        { }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[3] = LeftButtonSR;
   if (LeftButtonSR)
      SimulateKeyDown(VK_LeftShift, S_LeftShift);
   if (_Valuechanged[3] & !this[3])
      SimulateKeyUp(VK_LeftShift, S_LeftShift);
   this[2] = LeftButtonSL;
   if (_Valuechanged[2] & this[2])
      SimulateKeyDownArrows(VK_UP, S_UP);
   if (_Valuechanged[2] & !this[2])
      SimulateKeyUpArrows(VK_UP, S_UP);
   this[6] = LeftButtonSHOULDER_2;
   if (_Valuechanged[6] & this[6])
      SimulateKeyDown(VK_G, S_G);
   if (_Valuechanged[6] & !this[6])
      SimulateKeyUp(VK_G, S_G);
   this[11] = LeftButtonDPAD_DOWN;
   if (_Valuechanged[11] & this[11])
      SimulateKeyDownArrows(VK_Return, S_Return);
   if (_Valuechanged[11] & !this[11])
      SimulateKeyUpArrows(VK_Return, S_Return);
   this[22] = LeftButtonDPAD_UP;
   if (_Valuechanged[22] & this[22])
      SimulateKeyDownArrows(VK_Escape, S_Escape);
   if (_Valuechanged[22] & !this[22])
      SimulateKeyUpArrows(VK_Escape, S_Escape);
   this[12] = LeftButtonDPAD_LEFT;
   if (_Valuechanged[12] & this[12])
      SimulateKeyDown(VK_Return, S_Return);
   if (_Valuechanged[12] & !this[12])
      SimulateKeyUp(VK_Return, S_Return);
```

```
            this[13] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDownArrows(VK_Space, S_Space);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUpArrows(VK_Space, S_Space);
            this[7] = LeftButtonSTICK;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[23] = LeftButtonSHOULDER_1;
            if (_Valuechanged[23] & this[23])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[23] & !this[23])
               SimulateKeyUp(VK_T, S_T);
            this[8] = LeftButtonMINUS;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUpArrows(VK_UP, S_UP);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[24] = LeftButtonCAPTURE;
            if (_Valuechanged[24] & this[24])
               input.SendKey(Keys.RightAlt, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF(Keys.RightAlt, 1);
         }
```

JoyLeft4MissionAmmunition.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{ }

//keyboard control

public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];

public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[3] = LeftButtonSR;
          if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_X, S_X);
          if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_X, S_X);
          this[2] = LeftButtonSL;
          if (_Valuechanged[2] & this[2])
            SimulateKeyDown(VK_C, S_C);
          if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_C, S_C);
          this[6] = LeftButtonSHOULDER_2;
          if (_Valuechanged[6] & this[6])
            SimulateKeyDown(VK_G, S_G);
          if (_Valuechanged[6] & !this[6])
            SimulateKeyUp(VK_G, S_G);
          this[11] = LeftButtonDPAD_DOWN;
          if (_Valuechanged[11] & this[11])
            SimulateKeyDownArrows(VK_Return, S_Return);
          if (_Valuechanged[11] & !this[11])
            SimulateKeyUpArrows(VK_Return, S_Return);
          this[22] = LeftButtonDPAD_UP;
          if (_Valuechanged[22] & this[22])
            SimulateKeyDownArrows(VK_Escape, S_Escape);
          if (_Valuechanged[22] & !this[22])
            SimulateKeyUpArrows(VK_Escape, S_Escape);
          this[12] = LeftButtonDPAD_LEFT;
          if (_Valuechanged[12] & this[12])
            SimulateKeyDownArrows(VK_DOWN, S_DOWN);
          if (_Valuechanged[12] & !this[12])
            SimulateKeyUpArrows(VK_DOWN, S_DOWN);
          this[13] = LeftButtonDPAD_RIGHT;
          if (_Valuechanged[13] & this[13])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[13] & !this[13])
            SimulateKeyUpArrows(VK_UP, S_UP);
          this[7] = LeftButtonSTICK;
          if (_Valuechanged[7] & this[7])
            SimulateKeyDown(VK_Tab, S_Tab);
          if (_Valuechanged[7] & !this[7])
            SimulateKeyUp(VK_Tab, S_Tab);
```

```
            this[23] = LeftButtonSHOULDER_1;
            if (_Valuechanged[23] & this[23])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[23] & !this[23])
               SimulateKeyUp(VK_T, S_T);
            this[8] = LeftButtonMINUS;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUpArrows(VK_UP, S_UP);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[24] = LeftButtonCAPTURE;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_Z, S_Z);
         }
```

JoyRight4ZombieNightTerror.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f;
         public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
         public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
         public static double sumX = 0f, sumY = 0f;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = -(EulerAnglesRightX * 1500f) / 0.5f;
          iry = -(EulerAnglesRightZ * 1500f) / 0.5f;
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[1] = RightButtonSHOULDER_1 & RightButtonSHOULDER_2;
   if (_Valuechanged[1] & this[1])
      SimulateKeyDown(VK_NUMPAD0, S_NUMPAD0);
   if (_Valuechanged[1] & !this[1])
      SimulateKeyUp(VK_NUMPAD0, S_NUMPAD0);
   this[3] = RightButtonSR;
   if (_Valuechanged[3] & this[3])
      LeftClick();
   if (_Valuechanged[3] & !this[3])
      LeftClickF();
   this[2] = RightButtonSL;
   if (_Valuechanged[2] & this[2])
      RightClick();
   if (_Valuechanged[2] & !this[2])
      RightClickF();
   this[6] = RightButtonSHOULDER_2;
   if (_Valuechanged[6] & this[6])
      SimulateKeyDown(VK_G, S_G);
   if (_Valuechanged[6] & !this[6])
      SimulateKeyUp(VK_G, S_G);
   this[11] = RightButtonDPAD_DOWN;
   if (_Valuechanged[11] & this[11])
      SimulateKeyDownArrows(VK_LEFT, S_LEFT);
   if (_Valuechanged[11] & !this[11])
      SimulateKeyUpArrows(VK_LEFT, S_LEFT);
   this[22] = RightButtonDPAD_UP;
   if (_Valuechanged[22] & this[22])
      SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
   if (_Valuechanged[22] & !this[22])
      SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
```

```
this[12] = RightButtonDPAD_LEFT;
if (_Valuechanged[12] & this[12])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[12] & !this[12])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[13] = RightButtonDPAD_RIGHT;
if (_Valuechanged[13] & this[13])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[13] & !this[13])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[7] = RightButtonSTICK;
if (_Valuechanged[7] & this[7])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUp(VK_Tab, S_Tab);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_T, S_T);
this[8] = RightButtonPLUS;
if (_Valuechanged[8] & this[8])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUp(VK_Escape, S_Escape);
this[16] = GetStickRightX > 0.33f;
this[17] = GetStickRightX < -0.33f;
this[18] = GetStickRightY > 0.33f;
this[19] = GetStickRightY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_S, S_S);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_Q, S_Q);
this[24] = RightButtonHOME;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_P, S_P);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_P, S_P);
```

```
        }

JoyRightCam4ZombieNightTerror.txt
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
            public static System.Collections.Generic.List<double> valListX = new System.Collections.Generic.List<double>(), valListY = new System.Collections.Generic.List<double>();
            public static double sumX = 0f, sumY = 0f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            if (valListX.Count >= 200 & valListY.Count >= 200)
          {
            valListX.RemoveAt(0);
            valListX.Add(camx * 6f);
            irx = valListXAverage(valListX);
            valListY.RemoveAt(0);
            valListY.Add(camy * 6f);
            iry = valListYAverage(valListY);
```

```
                }
                else
                {
            valListX.Add(0);
            valListY.Add(0);
                }
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
      public double valListXAverage(System.Collections.Generic.List<double> valList)
      {
            sumX = 0f;
            foreach (double val in valList)
          sumX += val;
            return sumX / 200f;
      }
      public double valListYAverage(System.Collections.Generic.List<double> valList)
      {
            sumY = 0f;
            foreach (double val in valList)
          sumY += val;
            return sumY / 200f;
      }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[1] = RightButtonSHOULDER_1 & RightButtonSHOULDER_2;
          if (_Valuechanged[1] & this[1])
            SimulateKeyDown(VK_NUMPAD0, S_NUMPAD0);
          if (_Valuechanged[1] & !this[1])
            SimulateKeyUp(VK_NUMPAD0, S_NUMPAD0);
          this[3] = RightButtonSR;
          if (_Valuechanged[3] & this[3])
            LeftClick();
          if (_Valuechanged[3] & !this[3])
            LeftClickF();
          this[2] = RightButtonSL;
          if (_Valuechanged[2] & this[2])
            RightClick();
          if (_Valuechanged[2] & !this[2])
            RightClickF();
          this[6] = RightButtonSHOULDER_2;
          if (_Valuechanged[6] & this[6])
            SimulateKeyDown(VK_G, S_G);
          if (_Valuechanged[6] & !this[6])
            SimulateKeyUp(VK_G, S_G);
          this[11] = RightButtonDPAD_DOWN;
          if (_Valuechanged[11] & this[11])
            SimulateKeyDownArrows(VK_LEFT, S_LEFT);
          if (_Valuechanged[11] & !this[11])
            SimulateKeyUpArrows(VK_LEFT, S_LEFT);
          this[22] = RightButtonDPAD_UP;
          if (_Valuechanged[22] & this[22])
            SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
          if (_Valuechanged[22] & !this[22])
            SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
          this[12] = RightButtonDPAD_LEFT;
          if (_Valuechanged[12] & this[12])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[12] & !this[12])
            SimulateKeyUpArrows(VK_UP, S_UP);
          this[13] = RightButtonDPAD_RIGHT;
          if (_Valuechanged[13] & this[13])
            SimulateKeyDownArrows(VK_DOWN, S_DOWN);
          if (_Valuechanged[13] & !this[13])
            SimulateKeyUpArrows(VK_DOWN, S_DOWN);
```

```
            this[7] = RightButtonSTICK;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[23] = RightButtonSHOULDER_1;
            if (_Valuechanged[23] & this[23])
               SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[23] & !this[23])
               SimulateKeyUp(VK_T, S_T);
            this[8] = RightButtonPLUS;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUp(VK_Escape, S_Escape);
            this[16] = GetStickRightX > 0.33f;
            this[17] = GetStickRightX < -0.33f;
            this[18] = GetStickRightY > 0.33f;
            this[19] = GetStickRightY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[16] & !this[16])
               SimulateKeyUp(VK_S, S_S);
            if (_Valuechanged[17] & this[17])
               SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[17] & !this[17])
               SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[18] & this[18])
               SimulateKeyDown(VK_D, S_D);
            if (_Valuechanged[18] & !this[18])
               SimulateKeyUp(VK_D, S_D);
            if (_Valuechanged[19] & this[19])
               SimulateKeyDown(VK_Q, S_Q);
            if (_Valuechanged[19] & !this[19])
               SimulateKeyUp(VK_Q, S_Q);
            this[24] = RightButtonHOME;
            if (_Valuechanged[24] & this[24])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[24] & !this[24])
               SimulateKeyUp(VK_P, S_P);
         }
```

Joys4NightFall.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f,
hardness = 0f, irx, iry;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = -GetStickRightX * 1500f;
          iry = -GetStickRightY * 1500f;
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 40f;
          mouseyp += mousey * watchM / 40f;
          input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[2] = LeftButtonSHOULDER_1;
   if (_Valuechanged[2] & this[2])
      SimulateKeyDown(VK_Z, S_Z);
   if (_Valuechanged[2] & !this[2])
      SimulateKeyUp(VK_Z, S_Z);
   if (LeftButtonSHOULDER_1)
      SimulateKeyDown(VK_Z, S_Z);
   this[3] = LeftButtonMINUS;
   if (_Valuechanged[3] & this[3])
      SimulateKeyDown(VK_Return, S_Return);
   if (_Valuechanged[3] & !this[3])
      SimulateKeyUp(VK_Return, S_Return);
   this[27] = LeftButtonCAPTURE;
   if (_Valuechanged[27] & this[27])
      SimulateKeyDown(VK_P, S_P);
   if (_Valuechanged[27] & !this[27])
      SimulateKeyUp(VK_P, S_P);
   this[5] = LeftButtonDPAD_UP;
   if (_Valuechanged[5] & this[5])
      SimulateKeyDownArrows(VK_UP, S_UP);
   if (_Valuechanged[5] & !this[5])
      SimulateKeyUpArrows(VK_UP, S_UP);
   this[6] = LeftButtonDPAD_LEFT;
   if (_Valuechanged[6] & this[6])
      SimulateKeyDownArrows(VK_LEFT, S_LEFT);
   if (_Valuechanged[6] & !this[6])
      SimulateKeyUpArrows(VK_LEFT, S_LEFT);
   this[7] = LeftButtonDPAD_DOWN;
   if (_Valuechanged[7] & this[7])
      SimulateKeyDownArrows(VK_DOWN, S_DOWN);
```

```
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
/*if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);*/
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_LeftShift, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
if (_Valuechanged[15] & this[15])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
this[11] = RightButtonSHOULDER_2;
if (_Valuechanged[11] & this[11])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[11] & !this[11])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[34] = LeftButtonSHOULDER_2;
if (_Valuechanged[34] & this[34])
   SimulateKeyDown(VK_Z, S_Z);
```

```
            if (_Valuechanged[34] & !this[34])
               SimulateKeyUp(VK_Z, S_Z);
            if (LeftButtonSHOULDER_2)
               SimulateKeyDown(VK_Z, S_Z);
            this[35] = GetStickRightX > 0.33f;
            this[36] = GetStickRightX < -0.33f;
            this[37] = GetStickRightY > 0.33f;
            this[38] = GetStickRightY < -0.33f;
            if (_Valuechanged[35] & this[35])
               SimulateKeyDown(VK_1, S_1);
            if (_Valuechanged[35] & !this[35])
               SimulateKeyUp(VK_1, S_1);
            if (_Valuechanged[36] & this[36])
               SimulateKeyDown(VK_2, S_2);
            if (_Valuechanged[36] & !this[36])
               SimulateKeyUp(VK_2, S_2);
            if (_Valuechanged[37] & this[37])
               SimulateKeyDown(VK_3, S_3);
            if (_Valuechanged[37] & !this[37])
               SimulateKeyUp(VK_3, S_3);
            if (_Valuechanged[38] & this[38])
               SimulateKeyDown(VK_4, S_4);
            if (_Valuechanged[38] & !this[38])
               SimulateKeyUp(VK_4, S_4);
         }
```

Joys4SuperHotMindControlDelete.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 300f, slowing;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
         double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
         float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
         float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 29)
            {
           irx = -(EulerAnglesX * 1500f) / 0.5f - GetStickRightX * 1500f;
           iry = -(EulerAnglesZ * 1500f) / 0.5f - GetStickRightY * 1500f;
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          input.MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f), 11);
           input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
              slowing = 0;
            }
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          this[1] = RightButtonSHOULDER_1 & RightButtonSHOULDER_2;
          if (_Valuechanged[1] & this[1])
            SimulateKeyDown(VK_NUMPAD0, S_NUMPAD0);
          if (_Valuechanged[1] & !this[1])
            SimulateKeyUp(VK_NUMPAD0, S_NUMPAD0);
          this[2] = LeftButtonSHOULDER_1;
          if (_Valuechanged[2] & this[2])
            SimulateKeyDown(VK_LeftControl, S_LeftControl);
          if (_Valuechanged[2] & !this[2])
            SimulateKeyUp(VK_LeftControl, S_LeftControl);
          this[3] = LeftButtonMINUS;
          if (_Valuechanged[3] & this[3])
            SimulateKeyDown(VK_Return, S_Return);
          if (_Valuechanged[3] & !this[3])
            SimulateKeyUp(VK_Return, S_Return);
          this[27] = LeftButtonCAPTURE;
          if (_Valuechanged[27] & this[27])
            SimulateKeyDown(VK_P, S_P);
          if (_Valuechanged[27] & !this[27])
            SimulateKeyUp(VK_P, S_P);
          this[5] = LeftButtonDPAD_UP;
          if (_Valuechanged[5] & this[5])
            SimulateKeyDownArrows(VK_UP, S_UP);
          if (_Valuechanged[5] & !this[5])
            SimulateKeyUpArrows(VK_UP, S_UP);
          this[6] = LeftButtonDPAD_LEFT;
          if (_Valuechanged[6] & this[6])
            SimulateKeyDownArrows(VK_LEFT, S_LEFT);
          if (_Valuechanged[6] & !this[6])
            SimulateKeyUpArrows(VK_LEFT, S_LEFT);
          this[7] = LeftButtonDPAD_DOWN;
          if (_Valuechanged[7] & this[7])
            SimulateKeyDownArrows(VK_DOWN, S_DOWN);
          if (_Valuechanged[7] & !this[7])
            SimulateKeyUpArrows(VK_DOWN, S_DOWN);
          this[8] = LeftButtonDPAD_RIGHT;
          if (_Valuechanged[8] & this[8])
            SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
          if (_Valuechanged[8] & !this[8])
            SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
          this[9] = LeftButtonSTICK;
          if (_Valuechanged[9] & this[9])
            SimulateKeyDown(VK_LeftShift, S_LeftShift);
          if (_Valuechanged[9] & !this[9])
```

```
    SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
    SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
    SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
    SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
    SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
    SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
    SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
    SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
    SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
if (_Valuechanged[20] & this[20])
    SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
    SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
    SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
    SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
    SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
```

```
                SimulateKeyUp(VK_F, S_F);
            this[23] = RightButtonSHOULDER_1;
            if (_Valuechanged[23] & this[23])
                SimulateKeyDown(VK_U, S_U);
            if (_Valuechanged[23] & !this[23])
                SimulateKeyUp(VK_U, S_U);
            this[24] = RightButtonDPAD_LEFT;
            if (_Valuechanged[24] & this[24])
                SimulateKeyDown(VK_Y, S_Y);
            if (_Valuechanged[24] & !this[24])
                SimulateKeyUp(VK_Y, S_Y);
            this[25] = RightButtonDPAD_UP;
            if (_Valuechanged[25] & this[25])
                SimulateKeyDown(VK_X, S_X);
            if (_Valuechanged[25] & !this[25])
                SimulateKeyUp(VK_X, S_X);
            this[26] = RightButtonSTICK;
            if (_Valuechanged[26] & this[26])
                SimulateKeyDown(VK_Escape, S_Escape);
            if (_Valuechanged[26] & !this[26])
                SimulateKeyUp(VK_Escape, S_Escape);
            this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
            if (_Valuechanged[14] & this[14])
                SimulateKeyDown(VK_G, S_G);
            if (_Valuechanged[14] & !this[14])
                SimulateKeyUp(VK_G, S_G);
            this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
            if (_Valuechanged[15] & this[15])
                SimulateKeyDown(VK_T, S_T);
            if (_Valuechanged[15] & !this[15])
                SimulateKeyUp(VK_T, S_T);
            this[11] = RightButtonSHOULDER_2;
            if (_Valuechanged[11] & this[11])
                LeftClick();
            if (_Valuechanged[11] & !this[11])
                LeftClickF();
            if (ApressIO)
            {
                foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
                this[32] = LeftButtonSHOULDER_2;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        LeftButtonSHOULDER_2io = true;
                        randA = true;
                    }
                    else
                        if (randA)
```

```
                    {
                        LeftButtonSHOULDER_2io = false;
                        randA = false;
                    }
                if (LeftButtonSHOULDER_2io & foraorcison)
                {
                    LeftButtonSHOULDER_2io = false;
                    randA = false;
                }
                this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = LeftButtonSHOULDER_2;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
            this[35] = GetStickRightX > 0.33f;
            this[36] = GetStickRightX < -0.33f;
            this[37] = GetStickRightY > 0.33f;
            this[38] = GetStickRightY < -0.33f;
            if (_Valuechanged[35] & this[35])
                SimulateKeyDown(VK_1, S_1);
            if (_Valuechanged[35] & !this[35])
                SimulateKeyUp(VK_1, S_1);
            if (_Valuechanged[36] & this[36])
                SimulateKeyDown(VK_2, S_2);
            if (_Valuechanged[36] & !this[36])
                SimulateKeyUp(VK_2, S_2);
            if (_Valuechanged[37] & this[37])
                SimulateKeyDown(VK_3, S_3);
            if (_Valuechanged[37] & !this[37])
                SimulateKeyUp(VK_3, S_3);
            if (_Valuechanged[38] & this[38])
                SimulateKeyDown(VK_4, S_4);
            if (_Valuechanged[38] & !this[38])
                SimulateKeyUp(VK_4, S_4);
        }
```

Joys4TheExpendabros.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
        public double mousex, mousey, mousexp, mouseyp, deadzone = 30f, zoning = 100f,
hardness = 0f, irx, iry;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = -GetStickRightX * 1500f;
          iry = -GetStickRightY * 1500f;
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * ((1500f) /
Math.Pow(1500f, zoning / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 40f;
          mouseyp += mousey * watchM / 40f;
          input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
        }

//keyboard control
public static bool[] _Valuechanged = new bool[42], _valuechanged = new bool[42];
        public bool foraorcison, LeftButtonSHOULDER_2io, randA, ApressIO = false,
HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
{
   this[2] = LeftButtonSHOULDER_1;
   if (_Valuechanged[2] & this[2])
      SimulateKeyDown(VK_LeftControl, S_LeftControl);
   if (_Valuechanged[2] & !this[2])
      SimulateKeyUp(VK_LeftControl, S_LeftControl);
   this[3] = LeftButtonMINUS;
   if (_Valuechanged[3] & this[3])
      SimulateKeyDown(VK_Return, S_Return);
   if (_Valuechanged[3] & !this[3])
      SimulateKeyUp(VK_Return, S_Return);
   this[27] = LeftButtonCAPTURE;
   if (_Valuechanged[27] & this[27])
      SimulateKeyDown(VK_P, S_P);
   if (_Valuechanged[27] & !this[27])
      SimulateKeyUp(VK_P, S_P);
   this[5] = LeftButtonDPAD_UP;
   if (_Valuechanged[5] & this[5])
      SimulateKeyDownArrows(VK_UP, S_UP);
   if (_Valuechanged[5] & !this[5])
      SimulateKeyUpArrows(VK_UP, S_UP);
   this[6] = LeftButtonDPAD_LEFT;
   if (_Valuechanged[6] & this[6])
      SimulateKeyDownArrows(VK_LEFT, S_LEFT);
   if (_Valuechanged[6] & !this[6])
      SimulateKeyUpArrows(VK_LEFT, S_LEFT);
   this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = RightButtonDPAD_DOWN;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_A, S_A);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_E, S_E);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_E, S_E);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[20] = RightButtonPLUS;
```

```
if (_Valuechanged[20] & this[20])
   SimulateKeyDown(VK_Tab, S_Tab);
if (_Valuechanged[20] & !this[20])
   SimulateKeyUp(VK_Tab, S_Tab);
this[21] = RightButtonDPAD_RIGHT;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = RightButtonHOME;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = RightButtonSHOULDER_1;
if (_Valuechanged[23] & this[23])
   SimulateKeyDown(VK_U, S_U);
if (_Valuechanged[23] & !this[23])
   SimulateKeyUp(VK_U, S_U);
this[24] = RightButtonDPAD_LEFT;
if (_Valuechanged[24] & this[24])
   SimulateKeyDown(VK_Y, S_Y);
if (_Valuechanged[24] & !this[24])
   SimulateKeyUp(VK_Y, S_Y);
this[25] = RightButtonDPAD_UP;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = RightButtonSTICK;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = RightButtonSL | (HomeFTG & RightButtonHOME);
if (_Valuechanged[14] & this[14])
   SimulateKeyDown(VK_G, S_G);
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = RightButtonSR | (HomeFTG & RightButtonHOME);
if (_Valuechanged[15] & this[15])
   SimulateKeyDown(VK_T, S_T);
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
this[11] = RightButtonSHOULDER_2;
if (_Valuechanged[11] & this[11])
   LeftClick();
if (_Valuechanged[11] & !this[11])
   LeftClickF();
if (ApressIO)
```

```
            {
                foraorcison = (LeftButtonMINUS | RightButtonPLUS | RightButtonHOME |
(GetStickLeftY > 0.33f & LeftButtonSTICK) | LeftButtonDPAD_UP |
LeftButtonDPAD_DOWN | LeftButtonDPAD_LEFT | LeftButtonDPAD_RIGHT);
                this[32] = LeftButtonSHOULDER_2;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        LeftButtonSHOULDER_2io = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            LeftButtonSHOULDER_2io = false;
                            randA = false;
                        }
                if (LeftButtonSHOULDER_2io & foraorcison)
                {
                    LeftButtonSHOULDER_2io = false;
                    randA = false;
                }
                this[33] = LeftButtonSHOULDER_2io | (LeftButtonSHOULDER_2 &
foraorcison);
                if (_Valuechanged[33] & this[33])
                    RightClick();
                if (_Valuechanged[33] & !this[33])
                    RightClickF();
            }
            else
            {
                this[34] = LeftButtonSHOULDER_2;
                if (_Valuechanged[34] & this[34])
                    RightClick();
                if (_Valuechanged[34] & !this[34])
                    RightClickF();
            }
            this[35] = GetStickRightX > 0.33f;
            this[36] = GetStickRightX < -0.33f;
            this[37] = GetStickRightY > 0.33f;
            this[38] = GetStickRightY < -0.33f;
            if (_Valuechanged[35] & this[35])
                SimulateKeyDown(VK_1, S_1);
            if (_Valuechanged[35] & !this[35])
                SimulateKeyUp(VK_1, S_1);
            if (_Valuechanged[36] & this[36])
                SimulateKeyDown(VK_2, S_2);
            if (_Valuechanged[36] & !this[36])
                SimulateKeyUp(VK_2, S_2);
            if (_Valuechanged[37] & this[37])
```

```
            SimulateKeyDown(VK_3, S_3);
        if (_Valuechanged[37] & !this[37])
            SimulateKeyUp(VK_3, S_3);
        if (_Valuechanged[38] & this[38])
            SimulateKeyDown(VK_4, S_4);
        if (_Valuechanged[38] & !this[38])
            SimulateKeyUp(VK_4, S_4);
    }
```

WiiJoy412IsBetterThan6.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
```

```
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
```

```
if (_Valuechanged[13] & this[13])
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

8Days.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
                if (_Valuechanged[10] & this[10])
                    input.SendKey((Keys)S_Space, 1);
                if (_Valuechanged[10] & !this[10])
                    input.SendKeyF((Keys)S_Space, 1);
                this[29] = LeftButtonSL;
                if (_Valuechanged[29] & this[29])
                    input.SendKey((Keys)S_B, 1);
                if (_Valuechanged[29] & !this[29])
                    input.SendKeyF((Keys)S_B, 1);
                this[28] = LeftButtonSR;
                if (_Valuechanged[28] & this[28])
                    input.SendKey((Keys)S_N, 1);
                if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4AdvancedWarfare.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
```

```
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
```

```
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
```

```
            input.SendKey(Keys.Two, 1);
          if (keys123 == 2)
            input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
          if (keys123 == 0)
          {
            input.SendKeyF(Keys.One, 1);
            keys123 = 1;
          }
          else
            if (keys123 == 1)
            {
                  input.SendKeyF(Keys.Two, 1);
                  keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
          if (keys456 == 0)
            input.SendKey(Keys.Four, 1);
          if (keys456 == 1)
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
          }
          else
            if (keys456 == 1)
            {
                  input.SendKeyF(Keys.Five, 1);
                  keys456 = 2;
            }
```

```
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

AFistfulOfGun.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
```

```
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
```

```
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
```

```
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4AlanWake.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
          MoveMouseTo((int)(-mousex), (int)(mousey));
              System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
              SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
              SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
              SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

AlienSwarm.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4AmericanFugitive.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
               SendKeyF(VK_Q, S_Q);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SendKey(VK_D, S_D);
            if (_Valuechanged[8] & !this[8])
               SendKeyF(VK_D, S_D);
            this[9] = LeftButtonSTICK;
```

```
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
      if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
      if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
      else
      {
            wheelminus = 0;
            wheelminusbool = false;
      }
      if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
```

```
            this[17] = wheelminusbool;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               SendKey(VK_D, S_D);
            if (_Valuechanged[16] & !this[16])
               SendKeyF(VK_D, S_D);
            if (_Valuechanged[17] & this[17])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[17] & !this[17])
               SendKeyF(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
```

```
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
```

```
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ArtOfGravity.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
```

```
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Attractio.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
              SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
              SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
              SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
```

```
                    input.SendKeyF(Keys.Two, 1);
                    keys123 = 2;
                }
                else
                  if (keys123 == 2)
                    {
                      input.SendKeyF(Keys.Three, 1);
                      keys123 = 0;
                    }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
```

```
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Battlefield3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
           MoveMouseTo((int)(-mousex), (int)(mousey));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
```

```
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
```

```
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Bioshock2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
              System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                }
            if (_Valuechanged[23] & !this[23])
                {
                input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4BlackOps1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4BlackOps2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4BlackOps3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
           MoveMouseTo((int)(-mousex), (int)(mousey));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4BlackOps4.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 400f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
                irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f,
1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
             MouseMW3((int)(-mousex * 15f), (int)(mousey * 15f));
             System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
             public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
```

```
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftShift, S_LeftShift);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_E, S_E);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
                SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
                SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
                SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
```

```
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SimulateKeyUp(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   SimulateKeyDown(VK_Z, S_Z);
if (_Valuechanged[18] & !this[18])
   SimulateKeyUp(VK_Z, S_Z);
if (_Valuechanged[19] & this[19])
   SimulateKeyDown(VK_S, S_S);
if (_Valuechanged[19] & !this[19])
   SimulateKeyUp(VK_S, S_S);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   SimulateKeyDown(VK_A, S_A);
if (_Valuechanged[30] & !this[30])
   SimulateKeyUp(VK_A, S_A);
this[31] = DirectAnglesLeftY >= 0.888f;
```

```
            if (_Valuechanged[31] & this[31])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
               SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
               SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
               SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               SimulateKeyDown(VK_C, S_C);
            if (_Valuechanged[21] & !this[21])
               SimulateKeyUp(VK_C, S_C);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               SimulateKeyDown(VK_F, S_F);
            if (_Valuechanged[22] & !this[22])
               SimulateKeyUp(VK_F, S_F);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                    SimulateKeyDown(VK_U, S_U);
                    if (keys123 == 0)
                      SimulateKeyDown(VK_1, S_1);
                    if (keys123 == 1)
                      SimulateKeyDown(VK_2, S_2);
                    if (keys123 == 2)
                      SimulateKeyDown(VK_3, S_3);
                }
            if (_Valuechanged[23] & !this[23])
                {
                    SimulateKeyUp(VK_U, S_U);
                    if (keys123 == 0)
                    {
                       SimulateKeyUp(VK_1, S_1);
                       keys123 = 1;
                    }
                    else
                       if (keys123 == 1)
                       {
                            SimulateKeyUp(VK_2, S_2);
                            keys123 = 2;
```

```
                }
                else
                   if (keys123 == 2)
                      {
                         SimulateKeyUp(VK_3, S_3);
                         keys123 = 0;
                      }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
            SimulateKeyDown(VK_Y, S_Y);
            if (keys456 == 0)
              SimulateKeyDown(VK_4, S_4);
            if (keys456 == 1)
              SimulateKeyDown(VK_5, S_5);
            if (keys456 == 2)
              SimulateKeyDown(VK_6, S_6);
      }
if (_Valuechanged[24] & !this[24])
      {
            SimulateKeyUp(VK_Y, S_Y);
            if (keys456 == 0)
            {
               SimulateKeyUp(VK_4, S_4);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     SimulateKeyUp(VK_5, S_5);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        SimulateKeyUp(VK_6, S_6);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
```

```
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
                    SimulateKeyDown(VK_G, S_G);
               }
            if (_Valuechanged[14] & !this[14])
               SimulateKeyUp(VK_G, S_G);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                    SimulateKeyDown(VK_T, S_T);
               }
            if (_Valuechanged[15] & !this[15])
               SimulateKeyUp(VK_T, S_T);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  RightClick();
               if (_Valuechanged[34] & !this[34])
                  RightClickF();
        }
```

WiiJoy4BlairWitch.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 300f, center = 200f, slowing;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
                slowing += watchM;
                if (slowing >= 30)
                {
                  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f,
0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
                  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
                  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
                  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
                  input.MoveMouseBy((int)(-mousex / 4f), (int)(mousey / 4f), 11);
                  input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
                        slowing = 0;
                 }
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
           return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_Return, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_A, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_W, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_W, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_Q, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_Q, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Blameless.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
          MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
```

```
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
```

```
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
```

```
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                           input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                           input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
```

```
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
```

```
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
```

```
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4BloodbathKavkaz.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
```

```
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

BloodFeed.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
             MoveMouseTo((int)(-mousex), (int)(mousey));
                slowing = 0;
              }
           System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            {
            input.SendKey((Keys)S_U, 1);
                if (keys123 == 0)
                  input.SendKey(Keys.One, 1);
                if (keys123 == 1)
                  input.SendKey(Keys.Two, 1);
                if (keys123 == 2)
                  input.SendKey(Keys.Three, 1);
            }
        if (_Valuechanged[23] & !this[23])
            {
            input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                   input.SendKeyF(Keys.One, 1);
                   keys123 = 1;
                }
                else
                   if (keys123 == 1)
                   {
                        input.SendKeyF(Keys.Two, 1);
                        keys123 = 2;
                   }
                   else
                      if (keys123 == 2)
```

```
                    {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                    }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    input.SendKey((Keys)S_G, 1);
```

```
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
```

```
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4BloodyWalls.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
         public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
      double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
      float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
      float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
      bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
      {
         irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
         iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
```

```
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
```

```
if (_Valuechanged[13] & this[13])
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4BlueEstate.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
```

```
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
```

```
            input.SendKey(Keys.Enter, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
```

```
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
```

```
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Borderlands1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Borderlands2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[3] = LeftButtonMINUS;
        if (_Valuechanged[3] & this[3])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
```

```
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
```

```
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
```

```
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
```

```
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4BorderlandsThePresequel.txt

```
//mouse control
```

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4Brink.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
    public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
    public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
    public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
    public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
    double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
    float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
    float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
    bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        slowing += watchM;
        if (slowing >= 30)
        {
    irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
    iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
    mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
                mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
                MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                    slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
```

```
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey(Keys.Control, 1);
```

```
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
```

```
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4CallOfJuarezGunslinger.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
      public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
        mousexp += mousex * watchM / 10f;
        mouseyp += mousey * watchM / 10f;
        MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
```

```
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
```

```
                input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
        {
            WheelUpF();
    input.SendKey((Keys)S_G, 1);
        }
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
        {
            WheelDownF();
    input.SendKey((Keys)S_T, 1);
        }
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Capsized.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 520f, center = 200f, max = 1720f, slowing;
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5), 11);
}
private double Scale(double value, double min, double max, double minScale, double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
                if (_Valuechanged[10] & this[10])
                    input.SendKey((Keys)S_Space, 1);
                if (_Valuechanged[10] & !this[10])
                    input.SendKeyF((Keys)S_Space, 1);
                this[29] = LeftButtonSL;
                if (_Valuechanged[29] & this[29])
                    input.SendKey((Keys)S_B, 1);
                if (_Valuechanged[29] & !this[29])
                    input.SendKeyF((Keys)S_B, 1);
                this[28] = LeftButtonSR;
                if (_Valuechanged[28] & this[28])
                    input.SendKey((Keys)S_N, 1);
                if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4CrossCode.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
```

```
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
```

```
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4CSGO.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
```

```
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                            input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                            input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
```

```
                input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                }
                else
                    if (keys123 == 1)
                    {
                        input.SendKeyF(Keys.Two, 1);
                        keys123 = 2;
                    }
                    else
                        if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
```

```
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                    }
                this[25] = mWSButtonStateUp;
                if (_Valuechanged[25] & this[25])
                    input.SendKey((Keys)S_X, 1);
                if (_Valuechanged[25] & !this[25])
                    input.SendKeyF((Keys)S_X, 1);
                this[26] = mWSButtonStateTwo;
                if (_Valuechanged[26] & this[26])
                    input.SendKey((Keys)S_Escape, 1);
                if (_Valuechanged[26] & !this[26])
                    input.SendKeyF((Keys)S_Escape, 1);
                this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[14] & this[14])
                    {
                        WheelUpF();
                    input.SendKey((Keys)S_G, 1);
                    }
                if (_Valuechanged[14] & !this[14])
                    input.SendKeyF((Keys)S_G, 1);
                this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[15] & this[15])
                    {
                        WheelDownF();
                    input.SendKey((Keys)S_T, 1);
                    }
                if (_Valuechanged[15] & !this[15])
                    input.SendKeyF((Keys)S_T, 1);
                this[11] = mWSButtonStateB;
                if (_Valuechanged[11] & this[11])
                    input.SendLeftClick(11);
                if (_Valuechanged[11] & !this[11])
                    input.SendLeftClickF(11);
                if (ApressIO)
                {
                    foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                    this[32] = mWSButtonStateA;
                    if (_Valuechanged[32] & this[32])
                        if (!randA)
                        {
                            mWSButtonStateAio = true;
                            randA = true;
```

```
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Dark.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
```

```
            input.SendKeyF((Keys)S_Q, 1);
          if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
```

```
                input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                }
                else
                    if (keys123 == 1)
                    {
                        input.SendKeyF(Keys.Two, 1);
                        keys123 = 2;
                    }
                    else
                        if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
```

```
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                    }
                this[25] = mWSButtonStateUp;
                if (_Valuechanged[25] & this[25])
                    input.SendKey((Keys)S_X, 1);
                if (_Valuechanged[25] & !this[25])
                    input.SendKeyF((Keys)S_X, 1);
                this[26] = mWSButtonStateTwo;
                if (_Valuechanged[26] & this[26])
                    input.SendKey((Keys)S_Escape, 1);
                if (_Valuechanged[26] & !this[26])
                    input.SendKeyF((Keys)S_Escape, 1);
                this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[14] & this[14])
                    {
                        WheelUpF();
                    input.SendKey((Keys)S_G, 1);
                    }
                if (_Valuechanged[14] & !this[14])
                    input.SendKeyF((Keys)S_G, 1);
                this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[15] & this[15])
                    {
                        WheelDownF();
                    input.SendKey((Keys)S_T, 1);
                    }
                if (_Valuechanged[15] & !this[15])
                    input.SendKeyF((Keys)S_T, 1);
                this[11] = mWSButtonStateB;
                if (_Valuechanged[11] & this[11])
                    input.SendLeftClick(11);
                if (_Valuechanged[11] & !this[11])
                    input.SendLeftClickF(11);
                if (ApressIO)
                {
                    foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                    this[32] = mWSButtonStateA;
                    if (_Valuechanged[32] & this[32])
                        if (!randA)
                        {
                            mWSButtonStateAio = true;
                            randA = true;
```

```
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4DeadIsland.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
```

```
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
```

```
if (_Valuechanged[27] & this[27])
   input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
```

```
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
```

```
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
```

```
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
```

```
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4DeadIslandRiptide.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            {
                WheelUpF();
            input.SendKey((Keys)S_G, 1);
            }
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            {
                WheelDownF();
            input.SendKey((Keys)S_T, 1);
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
```

```
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4DeadRising2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4DeadSpace.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[3] = LeftButtonMINUS;
        if (_Valuechanged[3] & this[3])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
```

```
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
```

```
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
```

```
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
```

```
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4DeadSpace2.txt

```
//mouse control
```

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4DeepBlackReloaded.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
```

```
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey(Keys.H, 1);
if (_Valuechanged[31] & !this[31])
    input.SendKeyF(Keys.H, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
                        else
                           if (keys123 == 1)
                           {
                                 input.SendKeyF(Keys.Two, 1);
                                 keys123 = 2;
                           }
                           else
                              if (keys123 == 2)
                                 {
```

```
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
              input.SendKeyF(Keys.Five, 1);
              keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
  input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    {
        WheelUpF();
```

```
            input.SendKey((Keys)S_G, 1);
              }
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
              {
                  WheelDownF();
            input.SendKey((Keys)S_T, 1);
              }
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
              mWSButtonStateAio = false;
              randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
              input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
              input.SendRightClickF(11);
          }
          else
```

```
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4DeusExMankindDivided.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
              }
           System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            {
            input.SendKey((Keys)S_U, 1);
                if (keys123 == 0)
                  input.SendKey(Keys.One, 1);
                if (keys123 == 1)
                  input.SendKey(Keys.Two, 1);
                if (keys123 == 2)
                  input.SendKey(Keys.Three, 1);
            }
        if (_Valuechanged[23] & !this[23])
            {
            input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                   input.SendKeyF(Keys.One, 1);
                   keys123 = 1;
                }
                else
                   if (keys123 == 1)
                   {
                         input.SendKeyF(Keys.Two, 1);
                         keys123 = 2;
                   }
                   else
                      if (keys123 == 2)
```

```
                    {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                    }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    input.SendKey((Keys)S_G, 1);
```

```
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
```

```
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4DinosaurForest.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
      public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
      double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
      float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
      float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
      bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
      {
            slowing += watchM;
            if (slowing >= 30)
            {
         irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
              MoveMouseTo((int)(-mousex), (int)(mousey));
                  slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
```

```
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
    input.SendKeyF((Keys)S_E, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
                        else
                           if (keys123 == 1)
                           {
                                 input.SendKeyF(Keys.Two, 1);
                                 keys123 = 2;
                           }
                           else
                              if (keys123 == 2)
                                 {
```

```
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
              input.SendKeyF(Keys.Five, 1);
              keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
  input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
  input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
```

```
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            {
            input.SendLeftClick(11);
            mWSButtonStateAio = true;
            randA = true;
            }
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
```

```
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4DinosaurHunt.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
              MoveMouseTo((int)(-mousex), (int)(mousey));
                  slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
```

```
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
```

```
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
    input.SendKeyF((Keys)S_E, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
                        else
                           if (keys123 == 1)
                           {
                                 input.SendKeyF(Keys.Two, 1);
                                 keys123 = 2;
                           }
                           else
                              if (keys123 == 2)
                                 {
```

```
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
              input.SendKeyF(Keys.Five, 1);
              keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
  input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
  input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
```

```
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            {
            input.SendLeftClick(11);
            mWSButtonStateAio = true;
            randA = true;
            }
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
```

```
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Dishonored.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
```

```
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
                     if (keys123 == 1)
                     {
                           input.SendKeyF(Keys.Two, 1);
                           keys123 = 2;
                     }
```

```
                else
                    if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
                input.SendKeyF(Keys.Four, 1);
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                      input.SendKeyF(Keys.Five, 1);
                      keys456 = 2;
                }
                else
                    if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Disoriented.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
```

```
                if (slowing >= 30)
                {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
                }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
                     if (keys123 == 1)
                     {
                           input.SendKeyF(Keys.Two, 1);
                           keys123 = 2;
                     }
```

```
                else
                    if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
                input.SendKeyF(Keys.Four, 1);
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                      input.SendKeyF(Keys.Five, 1);
                      keys456 = 2;
                }
                else
                    if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Doom2016.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
```

```
                    input.SendKeyF(Keys.Two, 1);
                    keys123 = 2;
                }
                else
                  if (keys123 == 2)
                    {
                      input.SendKeyF(Keys.Three, 1);
                      keys123 = 0;
                    }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
```

```
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4DoubleOik.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
```

```
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
```

```
        }

WiiJoy4DrLangeskov.txt
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
        mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
                mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
                MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                    slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
```

```
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4DukeNukemForever.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
```

```
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
```

```
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
```

```
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
```

```
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4DungeonCreepster.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
    public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
    public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
    double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
    float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
    float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
    bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
        mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
        mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
```

```
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
```

```
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
```

```
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4DyingLight.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[3] = LeftButtonMINUS;
        if (_Valuechanged[3] & this[3])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
```

```
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
```

```
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
```

```
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
```

```
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4EnemyFront.txt

```
//mouse control
```

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
               if (_Valuechanged[3] & this[3])
                  input.SendKey((Keys)S_E, 1);
               if (_Valuechanged[3] & !this[3])
                  input.SendKeyF((Keys)S_E, 1);
               this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
               if (_Valuechanged[4] & this[4])
                  input.SendKey(Keys.V, 1);
               if (_Valuechanged[4] & !this[4])
                  input.SendKeyF(Keys.V, 1);
               this[27] = LeftButtonCAPTURE;
               if (_Valuechanged[27] & this[27])
                  input.SendKey(Keys.P, 1);
               if (_Valuechanged[27] & !this[27])
                  input.SendKeyF(Keys.P, 1);
               this[5] = LeftButtonDPAD_UP;
               if (_Valuechanged[5] & this[5])
                  input.SendKey((Keys)S_UP, 1);
               if (_Valuechanged[5] & !this[5])
                  input.SendKeyF((Keys)S_UP, 1);
               this[6] = LeftButtonDPAD_LEFT;
               if (_Valuechanged[6] & this[6])
                  input.SendKey((Keys)S_LEFT, 1);
               if (_Valuechanged[6] & !this[6])
                  input.SendKeyF((Keys)S_LEFT, 1);
               this[7] = LeftButtonDPAD_DOWN;
               if (_Valuechanged[7] & this[7])
                  input.SendKey((Keys)S_DOWN, 1);
               if (_Valuechanged[7] & !this[7])
                  input.SendKeyF((Keys)S_DOWN, 1);
               this[8] = LeftButtonDPAD_RIGHT;
               if (_Valuechanged[8] & this[8])
                  input.SendKey((Keys)S_RIGHT, 1);
               if (_Valuechanged[8] & !this[8])
                  input.SendKeyF((Keys)S_RIGHT, 1);
               this[9] = LeftButtonSTICK;
               if (_Valuechanged[9] & this[9])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[9] & !this[9])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4EnterTheGungeon.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4EscapeDeadIsland.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
           MoveMouseTo((int)(-mousex), (int)(mousey));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Evolve.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4FalloutShelter.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
               public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
               public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Farcry3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
```

```
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
```

```
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
```

```
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                       if (keys456 == 2)
                        {
                           input.SendKeyF(Keys.Six, 1);
                           keys456 = 0;
                        }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Farcry4.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4FarcryPrimal.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[3] = LeftButtonMINUS;
        if (_Valuechanged[3] & this[3])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
```

```
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
```

```
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
```

```
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
```

```
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4Flockers.txt

```
//mouse control
```

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseTo((int)(-mousex), (int)(mousey));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
```

```
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
```

```
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
```

```
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            input.SendKey(Keys.Four, 1);
          if (keys456 == 1)
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
```

```
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Fortnite.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 300f, center = 200f, slowing;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 14)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseBy((int)(-mousex), (int)(mousey), 11);
            input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
                slowing = 0;
            }
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateLeft;
if (_Valuechanged[23] & this[23])
      {
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateRight;
if (_Valuechanged[24] & this[24])
      {
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
```

```
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                    input.SendKeyF(Keys.Five, 1);
                    keys456 = 2;
                }
                else
                    if (keys456 == 2)
                    {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                    }
        }
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus;
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus;
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
            this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
```

WiiJoy4GenerationZero.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4GTAV.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 400f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
                  public double wheelplus, wheelminus;
            public bool wheelplusbool, wheelminusbool;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
               SendKeyF(VK_Q, S_Q);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SendKey(VK_D, S_D);
            if (_Valuechanged[8] & !this[8])
               SendKeyF(VK_D, S_D);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
      if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
      if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
      else
      {
            wheelminus = 0;
            wheelminusbool = false;
      }
      if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               SendKey(VK_D, S_D);
            if (_Valuechanged[16] & !this[16])
               SendKeyF(VK_D, S_D);
            if (_Valuechanged[17] & this[17])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[17] & !this[17])
               SendKeyF(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
```

```
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
```

```
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Gunpoint.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
```

```
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4HardReset.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
            MoveMouseTo((int)(-mousex), (int)(mousey));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
```

```
                    input.SendKeyF(Keys.Two, 1);
                    keys123 = 2;
                }
                else
                  if (keys123 == 2)
                    {
                      input.SendKeyF(Keys.Three, 1);
                      keys123 = 0;
                    }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
```

```
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4HardResetRedux.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          MoveMouseTo((int)(-mousex), (int)(mousey));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
```

```
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
```

```
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
```

```
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Hatred.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
                irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f,
1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
                iry = (mWSButtonStateIRY + center >= 0 ?
Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) :
Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
                mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
                mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
                MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS
+ mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey
* HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
           return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey(Keys.Control, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF(Keys.Control, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey(Keys.RightAlt, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF(Keys.RightAlt, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_A, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_W, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_W, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_Q, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_Q, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
```

```
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4HitmanAbsolution.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4HomefrontTheRevolution.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
```

```
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Hook.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

```
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
```

```
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
```

```
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
```

```
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4HotlineMiami1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
```

```
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
```

```
        }
```

WiiJoy4HotlineMiami2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseTo((int)(-mousex), (int)(mousey));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4HyperScape.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
    public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, hardness = 520f, center = 200f;
    public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
    public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
    public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
    double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
    float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
    float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
    bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
      irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
      iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
      mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
      mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
      mousexp += mousex * watchM / 10f;
```

```
            mouseyp += mousey * watchM / 10f;
            input.MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f), 11);
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_Return, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
```

```
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
```

```
if (_Valuechanged[13] & this[13])
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Return, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Return, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
```

```
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
```

```
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            {
                WheelDownF();
            input.SendKey((Keys)S_T, 1);
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
```

```
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4InsideMe.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
    public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
    public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
    double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
    float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
    float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
    bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
        mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
```

```
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
    input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
    input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
    input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
    input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
    input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
    input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
    input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
    input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
    input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
```

```
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4InsurgencySandstorm.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 270f, hardness = 33f, center = 200f, max = 1500f, slowing;
```

```
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 14)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            input.MoveMouseBy((int)(-mousex * 4f), (int)(mousey * 4f), 11);
                slowing = 0;
            }
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
```

```
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4JustCause3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
```

```
if (_Valuechanged[4] & this[4])
   input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
   input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
   input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[6] & !this[6])
   SendKeyF(VK_Q, S_Q);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
   SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
```

```
            wheelplusbool = true;
        else
            wheelplusbool = false;
        if (wheelplus > 20f)
            wheelplus = 0;
        if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
        else
            wheelminusbool = false;
        if (wheelminus < -20f)
            wheelminus = 0;
        if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
        else
        {
            wheelplus = 0;
            wheelplusbool = false;
        }
        if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
        if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
        else
        {
            wheelminus = 0;
            wheelminusbool = false;
        }
        if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
```

```
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
```

```
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_Z, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_S, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
              mWSButtonStateAio = false;
              randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
              input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
              input.SendRightClickF(11);
          }
          else
          {
```

```
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4Keebles.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
         public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
      double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
      float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
      float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
      bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
      {
         irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
         iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
```

```
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
```

```
if (_Valuechanged[13] & this[13])
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4KickAssCommandos.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
```

```
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
```

```
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4KillingFloor2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            {
                WheelUpF();
            input.SendKey((Keys)S_G, 1);
            }
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            {
                WheelDownF();
            input.SendKey((Keys)S_T, 1);
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
```

```
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4KillingRoom.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Klocki.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

```
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
```

```
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
```

```
if (_Valuechanged[27] & this[27])
   input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
```

```
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
```

```
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
```

```
                randA = false;
            }
        if (mWSButtonStateAio & foraorcison)
        {
            mWSButtonStateAio = false;
            randA = false;
        }
        this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
        if (_Valuechanged[33] & this[33])
            input.SendRightClick(11);
        if (_Valuechanged[33] & !this[33])
            input.SendRightClickF(11);
    }
    else
    {
        this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
}
```

WiiJoy4LaraCroftAndTheGuardianOfLight.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 100f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
          input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
```

```
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
```

```
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
```

```
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Left4Dead1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          MoveMouseTo((int)(-mousex), (int)(mousey));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
```

```
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
```

```
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                           input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                           input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
```

```
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
```

```
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Left4Dead2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          MoveMouseTo((int)(-mousex), (int)(mousey));
              System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
              SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4LichdomBattlemage.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
```

```
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4MachineHunt.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
             MoveMouseTo((int)(-mousex), (int)(mousey));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                keys456 = 0;
            }
        }
    this[25] = mWSButtonStateUp;
    if (_Valuechanged[25] & this[25])
        input.SendKey((Keys)S_X, 1);
    if (_Valuechanged[25] & !this[25])
        input.SendKeyF((Keys)S_X, 1);
    this[26] = mWSButtonStateTwo;
    if (_Valuechanged[26] & this[26])
        input.SendKey((Keys)S_Escape, 1);
    if (_Valuechanged[26] & !this[26])
        input.SendKeyF((Keys)S_Escape, 1);
    this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
    if (_Valuechanged[14] & this[14])
        input.SendKey((Keys)S_G, 1);
    if (_Valuechanged[14] & !this[14])
        input.SendKeyF((Keys)S_G, 1);
    this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
    if (_Valuechanged[15] & this[15])
        input.SendKey((Keys)S_T, 1);
    if (_Valuechanged[15] & !this[15])
        input.SendKeyF((Keys)S_T, 1);
    this[11] = mWSButtonStateB;
    if (_Valuechanged[11] & this[11])
        {
        input.SendLeftClick(11);
        mWSButtonStateAio = true;
        randA = true;
        }
    if (_Valuechanged[11] & !this[11])
        input.SendLeftClickF(11);
    if (ApressIO)
    {
        foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
        this[32] = mWSButtonStateA;
        if (_Valuechanged[32] & this[32])
            if (!randA)
            {
                mWSButtonStateAio = true;
                randA = true;
            }
            else
                if (randA)
                {
```

```
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4MadBullets.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4MadMax.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
                public double wheelplus, wheelminus;
            public bool wheelplusbool, wheelminusbool;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
```

```
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
                SendKeyF(VK_Q, S_Q);
            this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
   SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
      if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
```

```
            if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
                wheelminus += GetStickLeftX * watchM;
            else
            {
                wheelminus = 0;
                wheelminusbool = false;
            }
            if (GetStickLeftX <= -0.5f)
                wheelminusbool = true;
        this[16] = wheelplusbool;
        this[17] = wheelminusbool;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
          SendKey(VK_D, S_D);
        if (_Valuechanged[16] & !this[16])
          SendKeyF(VK_D, S_D);
        if (_Valuechanged[17] & this[17])
          SendKey(VK_Q, S_Q);
        if (_Valuechanged[17] & !this[17])
          SendKeyF(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
          input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
          input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
          input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
          input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
          input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
          input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
          input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
          input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
          input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
          input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
          input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
```

```
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_Z, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_S, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
```

```
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Mafia2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
}

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
                public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
```

```
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
               SendKeyF(VK_Q, S_Q);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SendKey(VK_D, S_D);
            if (_Valuechanged[8] & !this[8])
               SendKeyF(VK_D, S_D);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
```

```
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
      if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
      if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
      else
      {
            wheelminus = 0;
            wheelminusbool = false;
      }
      if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
```

```
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
        if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
        if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
```

```
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Mafia3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
                public double wheelplus, wheelminus;
            public bool wheelplusbool, wheelminusbool;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
```

```
    SendKeyF(VK_Q, S_Q);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
    SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
```

```
            if (GetStickLeftX >= 0.5f)
                wheelplusbool = true;
            if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
                wheelminus += GetStickLeftX * watchM;
            else
            {
                wheelminus = 0;
                wheelminusbool = false;
            }
            if (GetStickLeftX <= -0.5f)
                wheelminusbool = true;
        this[16] = wheelplusbool;
        this[17] = wheelminusbool;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
          SendKey(VK_D, S_D);
        if (_Valuechanged[16] & !this[16])
          SendKeyF(VK_D, S_D);
        if (_Valuechanged[17] & this[17])
          SendKey(VK_Q, S_Q);
        if (_Valuechanged[17] & !this[17])
          SendKeyF(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
          input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
          input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
          input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
          input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
          input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
          input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
          input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
          input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
          input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
          input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
```

```
    input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
    input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
    input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
    input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
    input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
    input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_Z, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_S, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
    input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
    input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4MagneticCageClosed.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
      public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
```

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
   input.SendLeftClick(11);
   mWSButtonStateAio = true;
   randA = true;
      }
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4Mandagon.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
```

```
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
```

```
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4MarkOfTheNinja.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
```

```
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
```

```
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
```

```
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Metro2033.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
```

```
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
```

```
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
```

```
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                           input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                           input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
```

```
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
```

```
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4MetroLastLight.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4MissingTranslation.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
```

```
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ModernWarfare.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 400f, center = 200f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
```

```
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            MouseMW3((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftShift, S_LeftShift);
```

```
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[3] & !this[3])
               SimulateKeyUp(VK_E, S_E);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
               SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
               SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               SimulateKeyDownArrows(VK_UP, S_UP);
            if (_Valuechanged[5] & !this[5])
               SimulateKeyUpArrows(VK_UP, S_UP);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SimulateKeyDownArrows(VK_LEFT, S_LEFT);
            if (_Valuechanged[6] & !this[6])
               SimulateKeyUpArrows(VK_LEFT, S_LEFT);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               SimulateKeyDownArrows(VK_DOWN, S_DOWN);
            if (_Valuechanged[7] & !this[7])
               SimulateKeyUpArrows(VK_DOWN, S_DOWN);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
            if (_Valuechanged[8] & !this[8])
               SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               SimulateKeyDown(VK_LeftShift, S_LeftShift);
            if (_Valuechanged[9] & !this[9])
               SimulateKeyUp(VK_LeftShift, S_LeftShift);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               SimulateKeyDown(VK_Space, S_Space);
            if (_Valuechanged[10] & !this[10])
               SimulateKeyUp(VK_Space, S_Space);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
```

```
            SimulateKeyDown(VK_B, S_B);
        if (_Valuechanged[29] & !this[29])
            SimulateKeyUp(VK_B, S_B);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            SimulateKeyDown(VK_N, S_N);
        if (_Valuechanged[28] & !this[28])
            SimulateKeyUp(VK_N, S_N);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            SimulateKeyDown(VK_D, S_D);
        if (_Valuechanged[16] & !this[16])
            SimulateKeyUp(VK_D, S_D);
        if (_Valuechanged[17] & this[17])
            SimulateKeyDown(VK_Q, S_Q);
        if (_Valuechanged[17] & !this[17])
            SimulateKeyUp(VK_Q, S_Q);
        if (_Valuechanged[18] & this[18])
            SimulateKeyDown(VK_Z, S_Z);
        if (_Valuechanged[18] & !this[18])
            SimulateKeyUp(VK_Z, S_Z);
        if (_Valuechanged[19] & this[19])
            SimulateKeyDown(VK_S, S_S);
        if (_Valuechanged[19] & !this[19])
            SimulateKeyUp(VK_S, S_S);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            SimulateKeyDown(VK_A, S_A);
        if (_Valuechanged[30] & !this[30])
            SimulateKeyUp(VK_A, S_A);
        this[31] = DirectAnglesLeftY >= 0.888f;
        if (_Valuechanged[31] & this[31])
            SimulateKeyDown(VK_E, S_E);
        if (_Valuechanged[31] & !this[31])
            SimulateKeyUp(VK_E, S_E);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            SimulateKeyDown(VK_R, S_R);
        if (_Valuechanged[13] & !this[13])
            SimulateKeyUp(VK_R, S_R);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            SimulateKeyDown(VK_Tab, S_Tab);
        if (_Valuechanged[20] & !this[20])
            SimulateKeyUp(VK_Tab, S_Tab);
```

```
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   SimulateKeyDown(VK_C, S_C);
if (_Valuechanged[21] & !this[21])
   SimulateKeyUp(VK_C, S_C);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   SimulateKeyDown(VK_F, S_F);
if (_Valuechanged[22] & !this[22])
   SimulateKeyUp(VK_F, S_F);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
            SimulateKeyDown(VK_U, S_U);
            if (keys123 == 0)
              SimulateKeyDown(VK_1, S_1);
            if (keys123 == 1)
              SimulateKeyDown(VK_2, S_2);
            if (keys123 == 2)
              SimulateKeyDown(VK_3, S_3);
      }
if (_Valuechanged[23] & !this[23])
      {
            SimulateKeyUp(VK_U, S_U);
            if (keys123 == 0)
            {
               SimulateKeyUp(VK_1, S_1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     SimulateKeyUp(VK_2, S_2);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        SimulateKeyUp(VK_3, S_3);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
            SimulateKeyDown(VK_Y, S_Y);
            if (keys456 == 0)
              SimulateKeyDown(VK_4, S_4);
            if (keys456 == 1)
              SimulateKeyDown(VK_5, S_5);
```

```
            if (keys456 == 2)
              SimulateKeyDown(VK_6, S_6);
      }
if (_Valuechanged[24] & !this[24])
      {
            SimulateKeyUp(VK_Y, S_Y);
            if (keys456 == 0)
            {
               SimulateKeyUp(VK_4, S_4);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     SimulateKeyUp(VK_5, S_5);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        SimulateKeyUp(VK_6, S_6);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   SimulateKeyDown(VK_X, S_X);
if (_Valuechanged[25] & !this[25])
   SimulateKeyUp(VK_X, S_X);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   SimulateKeyDown(VK_Escape, S_Escape);
if (_Valuechanged[26] & !this[26])
   SimulateKeyUp(VK_Escape, S_Escape);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
            SimulateKeyDown(VK_G, S_G);
   }
if (_Valuechanged[14] & !this[14])
   SimulateKeyUp(VK_G, S_G);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
            SimulateKeyDown(VK_T, S_T);
   }
if (_Valuechanged[15] & !this[15])
   SimulateKeyUp(VK_T, S_T);
```

```
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               LeftClick();
            if (_Valuechanged[11] & !this[11])
               LeftClickF();
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  RightClick();
               if (_Valuechanged[34] & !this[34])
                  RightClickF();
         }
```

WiiJoy4ModernWarfare2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
               public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
               public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
         double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
         float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
         float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
         bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
         {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
                          input.SendKeyF(Keys.Two, 1);
                          keys123 = 2;
                    }
```

```
                else
                    if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ModernWarfare2019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 400f, center = 200f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
```

```
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
         }
         private double Scale(double value, double min, double max, double minScale,
double maxScale)
         {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
         }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
         public bool HomeFTG = true;
         public int keys123, keys456;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
         double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
         float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
         float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
         bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
         {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
```

```
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_Return, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.Tab, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.Tab, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF(Keys.Control, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
```

```
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_A, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_W, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_W, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey(Keys.Q, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF(Keys.Q, 1);
            this[31] = DirectAnglesLeftY >= 0.888f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
{
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
}
if (_Valuechanged[23] & !this[23])
{
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
}
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
{
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
```

```
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
}
if (_Valuechanged[24] & !this[24])
{
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
}
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
  input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
```

```
            input.SendLeftClickF(11);
        this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
```

WiiJoy4ModernWarfare3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
```

```
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
            MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5));
         }
         private double Scale(double value, double min, double max, double minScale,
double maxScale)
         {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
         }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
         public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
         double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
         float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
         float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
         bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
         {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
```

```
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
```

```
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
```

```
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            input.SendKey(Keys.Four, 1);
          if (keys456 == 1)
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
          WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
          WheelDownF();
   input.SendKey((Keys)S_T, 1);
```

```
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4MouseCraft.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
```

```
                }
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
```

```
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
```

```
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4NinjaReflexSteamworksEdition.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
```

```
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
```

```
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
```

```
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Oik3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
               public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
               public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4OnlyIf.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
```

```
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
```

```
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
```

```
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
```

```
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
```

```
                    }
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
```

```
                randA = true;
            }
            else
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
        if (mWSButtonStateAio & foraorcison)
        {
            mWSButtonStateAio = false;
            randA = false;
        }
        this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
        if (_Valuechanged[33] & this[33])
            input.SendRightClick(11);
        if (_Valuechanged[33] & !this[33])
            input.SendRightClickF(11);
    }
    else
    {
        this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
}
```

WiiJoy4OrbitalGear.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
```

```
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4OriAndTheBlindForest.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
         public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
      public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
      double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
      float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
      float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
      bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
```

```
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
              mWSButtonStateAio = false;
              randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
              input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
              input.SendRightClickF(11);
          }
          else
          {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
              input.SendRightClick(11);
```

```
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Outland.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
```

```
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
```

```
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
```

```
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4OverdosedATripToHell.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4PainTrain.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4PainTrain2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
   input.SendLeftClick(11);
   mWSButtonStateAio = true;
   randA = true;
      }
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
```

```
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4PartyHard.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Payday1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
```

```
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
```

```
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
```

```
            input.SendKey(Keys.Two, 1);
          if (keys123 == 2)
            input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
          if (keys123 == 0)
          {
            input.SendKeyF(Keys.One, 1);
            keys123 = 1;
          }
          else
            if (keys123 == 1)
            {
                  input.SendKeyF(Keys.Two, 1);
                  keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
          if (keys456 == 0)
            input.SendKey(Keys.Four, 1);
          if (keys456 == 1)
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
          }
          else
            if (keys456 == 1)
            {
                  input.SendKeyF(Keys.Five, 1);
                  keys456 = 2;
            }
```

```
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Payday2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
```

```
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
```

```
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
```

```
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                       if (keys456 == 2)
                        {
                           input.SendKeyF(Keys.Six, 1);
                           keys456 = 0;
                        }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4PhantomSoldier.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
              SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
              SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
              SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
   input.SendLeftClick(11);
   mWSButtonStateAio = true;
   randA = true;
      }
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
```

```
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Portal.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4Portal2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
                if (_Valuechanged[10] & this[10])
                    input.SendKey((Keys)S_Space, 1);
                if (_Valuechanged[10] & !this[10])
                    input.SendKeyF((Keys)S_Space, 1);
                this[29] = LeftButtonSL;
                if (_Valuechanged[29] & this[29])
                    input.SendKey((Keys)S_B, 1);
                if (_Valuechanged[29] & !this[29])
                    input.SendKeyF((Keys)S_B, 1);
                this[28] = LeftButtonSR;
                if (_Valuechanged[28] & this[28])
                    input.SendKey((Keys)S_N, 1);
                if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4PortalStoriesMel.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, zoning = 460f, center =
200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
```

```
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Prey.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public                double                WidthS                =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public                double                HeightS                =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
        mousexp += mousex * watchM / 10f;
        mouseyp += mousey * watchM / 10f;
        MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
```

```
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
```

```
                input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
        {
            WheelUpF();
    input.SendKey((Keys)S_G, 1);
        }
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
        {
            WheelDownF();
    input.SendKey((Keys)S_T, 1);
        }
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Push.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
```

```
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
```

```
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RadicalRoach.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
```

```
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
```

```
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
```

```
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
```

```
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Rage.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
```

```
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                            input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                            input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
```

```
                input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                }
                else
                    if (keys123 == 1)
                    {
                        input.SendKeyF(Keys.Two, 1);
                        keys123 = 2;
                    }
                    else
                        if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
```

```
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                    }
                this[25] = mWSButtonStateUp;
                if (_Valuechanged[25] & this[25])
                    input.SendKey((Keys)S_X, 1);
                if (_Valuechanged[25] & !this[25])
                    input.SendKeyF((Keys)S_X, 1);
                this[26] = mWSButtonStateTwo;
                if (_Valuechanged[26] & this[26])
                    input.SendKey((Keys)S_Escape, 1);
                if (_Valuechanged[26] & !this[26])
                    input.SendKeyF((Keys)S_Escape, 1);
                this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[14] & this[14])
                    {
                        WheelUpF();
                    input.SendKey((Keys)S_G, 1);
                    }
                if (_Valuechanged[14] & !this[14])
                    input.SendKeyF((Keys)S_G, 1);
                this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[15] & this[15])
                    {
                        WheelDownF();
                    input.SendKey((Keys)S_T, 1);
                    }
                if (_Valuechanged[15] & !this[15])
                    input.SendKeyF((Keys)S_T, 1);
                this[11] = mWSButtonStateB;
                if (_Valuechanged[11] & this[11])
                    input.SendLeftClick(11);
                if (_Valuechanged[11] & !this[11])
                    input.SendLeftClickF(11);
                if (ApressIO)
                {
                    foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                    this[32] = mWSButtonStateA;
                    if (_Valuechanged[32] & this[32])
                        if (!randA)
                        {
                            mWSButtonStateAio = true;
                            randA = true;
```

```
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Rage2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 400f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 6)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseBy((int)(-mousex * 1f), (int)(mousey * 1f), 11);
            input.MoveMouseTo((int)(32767.5 - irx * 32f), (int)(iry * 32f + 32767.5), 11);
                slowing = 0;
            }
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
           double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
           return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false, forstickoff, randS, LeftButtonSHOULDER_2io;
        public int keys123, keys456;
        public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            forstickoff = GetStickLeftY <= 0.25f;
            this[1] = LeftButtonSHOULDER_2;
            if (_Valuechanged[1] & this[1])
               if (!randS)
               {
                  LeftButtonSHOULDER_2io = true;
                  randS = true;
               }
               else
                  if (randS)
                  {
                     LeftButtonSHOULDER_2io = false;
                     randS = false;
                  }
               if (LeftButtonSHOULDER_2io & forstickoff)
               {
                  LeftButtonSHOULDER_2io = false;
                  randS = false;
               }
            this[2] = LeftButtonSHOULDER_2io;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
```

```
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_Return, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
```

```
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
       if (wheelplus >= 5f & wheelplus <= 7f)
            wheelplusbool = true;
       else
            wheelplusbool = false;
       if (wheelplus > 7f)
            wheelplus = 0;
       if (wheelminus <= -5f & wheelminus >= -7f)
            wheelminusbool = true;
       else
            wheelminusbool = false;
       if (wheelminus < -7f)
            wheelminus = 0;
       if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
       else
       {
            wheelplus = 0;
            wheelplusbool = false;
       }
       if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
       if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
       else
       {
            wheelminus = 0;
            wheelminusbool = false;
       }
       if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool | GetStickLeftX > 0.25f;
this[17] = wheelminusbool | GetStickLeftX < -0.25f;
this[18] = GetStickLeftY > 0.25f;
this[19] = GetStickLeftY < -0.25f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
```

```
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               WheelUpF();
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                           input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
```

```
                input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
                if (keys123 == 0)
                {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                }
                else
                    if (keys123 == 1)
                    {
                            input.SendKeyF(Keys.Two, 1);
                            keys123 = 2;
                    }
                    else
                        if (keys123 == 2)
                            {
                                input.SendKeyF(Keys.Three, 1);
                                keys123 = 0;
                            }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    WheelDownF();
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                            input.SendKeyF(Keys.Five, 1);
                            keys456 = 2;
                    }
                    else
```

```
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                    }
                this[25] = mWSButtonStateUp;
                if (_Valuechanged[25] & this[25])
                    input.SendKey((Keys)S_X, 1);
                if (_Valuechanged[25] & !this[25])
                    input.SendKeyF((Keys)S_X, 1);
                this[26] = mWSButtonStateTwo;
                if (_Valuechanged[26] & this[26])
                    input.SendKey((Keys)S_Escape, 1);
                if (_Valuechanged[26] & !this[26])
                    input.SendKeyF((Keys)S_Escape, 1);
                this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[14] & this[14])
                    input.SendKey((Keys)S_G, 1);
                if (_Valuechanged[14] & !this[14])
                    input.SendKeyF((Keys)S_G, 1);
                this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
                if (_Valuechanged[15] & this[15])
                    input.SendKey((Keys)S_T, 1);
                if (_Valuechanged[15] & !this[15])
                    input.SendKeyF((Keys)S_T, 1);
                this[11] = mWSButtonStateB;
                if (_Valuechanged[11] & this[11])
                    input.SendLeftClick(11);
                if (_Valuechanged[11] & !this[11])
                    input.SendLeftClickF(11);
                if (ApressIO)
                {
                    foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                    this[32] = mWSButtonStateA;
                    if (_Valuechanged[32] & this[32])
                        if (!randA)
                        {
                            mWSButtonStateAio = true;
                            randA = true;
                        }
                        else
                            if (randA)
                            {
                                mWSButtonStateAio = false;
```

```
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4RamboTheVideoGame.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
          MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
```

```
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
```

```
                    input.SendKeyF(Keys.Two, 1);
                    keys123 = 2;
                }
                else
                  if (keys123 == 2)
                    {
                      input.SendKeyF(Keys.Three, 1);
                      keys123 = 0;
                    }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
```

```
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Redie.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
        if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
        if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
```

```
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4RedTrigger.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
                MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                    slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
```

```
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
```

```
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
```

```
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
            input.SendKey(Keys.Four, 1);
          if (keys456 == 1)
            input.SendKey(Keys.Five, 1);
          if (keys456 == 2)
            input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
          if (keys456 == 0)
          {
             input.SendKeyF(Keys.Four, 1);
             keys456 = 1;
          }
          else
             if (keys456 == 1)
             {
                   input.SendKeyF(Keys.Five, 1);
                   keys456 = 2;
             }
             else
                if (keys456 == 2)
                   {
                      input.SendKeyF(Keys.Six, 1);
                      keys456 = 0;
                   }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
```

```
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                    }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RelicHuntersZero.txt
//mouse control

```
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
```

```
            input.SendKey(Keys.Enter, 1);
        if (_Valuechanged[3] & !this[3])
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
```

```
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
```

```
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RememberMe.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RemnantFromTheAshes.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
   input.SendLeftClick(11);
   mWSButtonStateAio = true;
   randA = true;
      }
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
```

```
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ResidentEvil6.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousexp += mousex * watchM / 10f;
        mouseyp += mousey * watchM / 10f;
        MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
```

```
                input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
                input.SendKeyF(Keys.Four, 1);
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                    input.SendKeyF(Keys.Five, 1);
                    keys456 = 2;
                }
                else
                    if (keys456 == 2)
                    {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                    }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
        {
            WheelUpF();
    input.SendKey((Keys)S_G, 1);
        }
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
        {
            WheelDownF();
    input.SendKey((Keys)S_T, 1);
        }
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4RetroCityRampageDX.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
```

```
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
```

```
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Reus.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
```

```
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
```

```
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
```

```
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
```

```
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
            }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4RiseAndShine.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
```

```
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
```

```
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4RiseOfTheTriad.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
```

```
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
                     if (keys123 == 1)
                     {
                           input.SendKeyF(Keys.Two, 1);
                           keys123 = 2;
                     }
```

```
                else
                    if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
                input.SendKeyF(Keys.Four, 1);
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                      input.SendKeyF(Keys.Five, 1);
                      keys456 = 2;
                }
                else
                    if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RiseOfTombRaider.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
```

```
                if (slowing >= 30)
                {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
                }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
```

```
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
```

```
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
                          input.SendKeyF(Keys.Two, 1);
                          keys123 = 2;
                    }
```

```
                else
                    if (keys123 == 2)
                        {
                            input.SendKeyF(Keys.Three, 1);
                            keys123 = 0;
                        }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
                if (keys456 == 0)
                  input.SendKey(Keys.Four, 1);
                if (keys456 == 1)
                  input.SendKey(Keys.Five, 1);
                if (keys456 == 2)
                  input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                  {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                  }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Ronin.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
        if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
        if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
```

```
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4RunningThroughRussia.txt
```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey(Keys.Control, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF(Keys.Control, 1);
```

```
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
```

```
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
```

```
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
```

```
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4SaintsRowIV.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
```

```
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4SaintsRowTheThird.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
         public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseTo((int)(-mousex), (int)(mousey));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
      {
   input.SendLeftClick(11);
   mWSButtonStateAio = true;
   randA = true;
      }
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
```

```
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ShadowWarrior.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        MoveMouseTo((int)(-mousex), (int)(mousey));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
    }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
```

```
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ShadowWarrior2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public                double                WidthS                =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
MoveMouseTo((int)(-mousex), (int)(mousey));
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
```

```
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
```

```
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
```

```
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
```

```
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
```

```
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Shank2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
}
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
```

```
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Shank2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
```

```
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
```

```
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
```

```
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
```

```
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Skye.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 400f, hardness = 400f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 26)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
          input.MoveMouseBy((int)(-mousex), (int)(-mousey), 11);
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS - iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
this[2] = LeftButtonSTICK;
if (_Valuechanged[2] & this[2])
input.SendKey(Keys.Control, 1);
if (_Valuechanged[2] & !this[2])
input.SendKeyF(Keys.Control, 1);
this[3] = LeftButtonMINUS;
if (_Valuechanged[3] & this[3])
input.SendKey(Keys.Enter, 1);
if (_Valuechanged[3] & !this[3])
input.SendKeyF(Keys.Enter, 1);
this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
if (_Valuechanged[4] & this[4])
input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSHOULDER_2;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_A, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_W, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_W, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_Q, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_Q, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
```

```
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Skyrim.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                }
            if (_Valuechanged[23] & !this[23])
                {
                input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SkyrimSpecialEdition.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
```

```
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SleepingDogs.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
             MoveMouseTo((int)(-mousex), (int)(mousey));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                            input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                            input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                            input.SendKey(Keys.Three, 1);
                }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
```

```
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SniperElite3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning
= 250f, hardness = 330f, center = 200f, max = 1500f, slowing;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 26)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            input.MoveMouseBy((int)(-mousex), (int)(mousey), 11);
                slowing = 0;
            }
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
```

```
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
```

```
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SniperElite4.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 30f, zoning = 250f, hardness = 330f, center = 200f, max = 1500f, slowing;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 26)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
                mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
                input.MoveMouseBy((int)(-mousex), (int)(mousey), 11);
                    slowing = 0;
                }
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
                this[2] = LeftButtonSHOULDER_2;
                if (_Valuechanged[2] & this[2])
                    input.SendKey(Keys.Control, 1);
```

```
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
```

```
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4SniperEliteNaziZomby1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
      public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousexp += mousex * watchM / 10f;
        mouseyp += mousey * watchM / 10f;
        MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
```

```
                input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
        {
            WheelUpF();
    input.SendKey((Keys)S_G, 1);
        }
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
        {
            WheelDownF();
    input.SendKey((Keys)S_T, 1);
        }
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4SniperEliteNaziZomby2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousexp += mousex * watchM / 10f;
mouseyp += mousey * watchM / 10f;
MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4SniperEliteNaziZombyArmyTrilogy.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
```

```
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey(Keys.H, 1);
if (_Valuechanged[31] & !this[31])
    input.SendKeyF(Keys.H, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
               input.SendKey((Keys)S_U, 1);
                     if (keys123 == 0)
                       input.SendKey(Keys.One, 1);
                     if (keys123 == 1)
                       input.SendKey(Keys.Two, 1);
                     if (keys123 == 2)
                       input.SendKey(Keys.Three, 1);
                }
            if (_Valuechanged[23] & !this[23])
                {
               input.SendKeyF((Keys)S_U, 1);
                     if (keys123 == 0)
                     {
                        input.SendKeyF(Keys.One, 1);
                        keys123 = 1;
                     }
                     else
                        if (keys123 == 1)
                        {
                              input.SendKeyF(Keys.Two, 1);
                              keys123 = 2;
                        }
                        else
                           if (keys123 == 2)
                              {
```

```
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
              input.SendKeyF(Keys.Five, 1);
              keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
  input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    {
        WheelUpF();
```

```
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
            }
            else
```

```
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4SniperEliteV2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                    input.SendKeyF(Keys.One, 1);
                    keys123 = 1;
                  }
                  else
                    if (keys123 == 1)
                    {
```

```
                    input.SendKeyF(Keys.Two, 1);
                    keys123 = 2;
                }
                else
                    if (keys123 == 2)
                    {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                    }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
                input.SendKeyF(Keys.Four, 1);
                keys456 = 1;
            }
            else
                if (keys456 == 1)
                {
                    input.SendKeyF(Keys.Five, 1);
                    keys456 = 2;
                }
                else
                    if (keys456 == 2)
                    {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                    }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
```

```
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
```

```
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoySniperGhostWarrior1.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                }
            if (_Valuechanged[23] & !this[23])
                {
                input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
```

```
            keys123 = 1;
        }
        else
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
                if (keys123 == 2)
                {
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
            input.SendKeyF(Keys.Four, 1);
            keys456 = 1;
        }
        else
            if (keys456 == 1)
            {
                input.SendKeyF(Keys.Five, 1);
                keys456 = 2;
            }
            else
                if (keys456 == 2)
                {
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
```

```
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SniperGhostWarrior2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
```

```
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
   input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
   input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
```

```
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                {
                input.SendKey((Keys)S_U, 1);
                    if (keys123 == 0)
                        input.SendKey(Keys.One, 1);
                    if (keys123 == 1)
                        input.SendKey(Keys.Two, 1);
                    if (keys123 == 2)
                        input.SendKey(Keys.Three, 1);
                }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
               input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
```

```
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SniperGhostWarrior3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public double wheelplus, wheelminus;
```

```
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
```

```
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[6] & !this[6])
   SendKeyF(VK_Q, S_Q);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
   SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
```

```
        else
            wheelminusbool = false;
        if (wheelminus < -20f)
            wheelminus = 0;
        if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
        else
        {
            wheelplus = 0;
            wheelplusbool = false;
        }
        if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
        if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
        else
        {
            wheelminus = 0;
            wheelminusbool = false;
        }
        if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
   input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   input.SendKey((Keys)S_E, 1);
```

```
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Sparkle.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
                this[2] = LeftButtonSHOULDER_2;
                if (_Valuechanged[2] & this[2])
                    input.SendKey(Keys.Control, 1);
```

```
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
```

```
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
            if (mWSButtonStateAio & foraorcison)
            {
               mWSButtonStateAio = false;
               randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
               input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
               input.SendRightClickF(11);
         }
         else
         {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
               input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
               input.SendRightClickF(11);
         }
      }
```

WiiJoy4SpecOpsTheLine.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
      public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
      public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
         public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousexp += mousex * watchM / 10f;
        mouseyp += mousey * watchM / 10f;
        MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
```

```
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = LeftButtonMINUS;
```

```
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
```

```
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
```

```
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
```

```
                input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
                if (keys456 == 0)
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
        }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
        {
            WheelUpF();
    input.SendKey((Keys)S_G, 1);
        }
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
        {
            WheelDownF();
    input.SendKey((Keys)S_T, 1);
        }
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
```

```
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4SplatterZombiecalypseNow.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
```

```
public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale, double maxScale)
```

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4StateOfDecay.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
```

```
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
```

```
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
   input.SendKeyF((Keys)S_D, 1);
```

```
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
```

```
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
```

```
                    }
                    else
                        if (keys456 == 2)
                            {
                                input.SendKeyF(Keys.Six, 1);
                                keys456 = 0;
                            }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
```

```
                randA = true;
            }
            else
                if (randA)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
        if (mWSButtonStateAio & foraorcison)
        {
            mWSButtonStateAio = false;
            randA = false;
        }
        this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
        if (_Valuechanged[33] & this[33])
            input.SendRightClick(11);
        if (_Valuechanged[33] & !this[33])
            input.SendRightClickF(11);
    }
    else
    {
        this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
}
```

WiiJoy4StrandedDeep.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 330f, hardness = 330f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
```

```
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseBy((int)(-mousex / 2f), (int)(mousey / 2f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

```
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                SimulateKeyDown(VK_LeftControl, S_LeftControl);
            if (_Valuechanged[2] & !this[2])
                SimulateKeyUp(VK_LeftControl, S_LeftControl);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                SimulateKeyDown(VK_Return, S_Return);
            if (_Valuechanged[3] & !this[3])
                SimulateKeyUp(VK_Return, S_Return);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                SimulateKeyDown(VK_V, S_V);
            if (_Valuechanged[4] & !this[4])
                SimulateKeyUp(VK_V, S_V);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                SimulateKeyDown(VK_P, S_P);
            if (_Valuechanged[27] & !this[27])
                SimulateKeyUp(VK_P, S_P);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
   SimulateKeyDownArrows(VK_UP, S_UP);
if (_Valuechanged[5] & !this[5])
   SimulateKeyUpArrows(VK_UP, S_UP);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SimulateKeyDownArrows(VK_LEFT, S_LEFT);
if (_Valuechanged[6] & !this[6])
   SimulateKeyUpArrows(VK_LEFT, S_LEFT);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   SimulateKeyDownArrows(VK_DOWN, S_DOWN);
if (_Valuechanged[7] & !this[7])
   SimulateKeyUpArrows(VK_DOWN, S_DOWN);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SimulateKeyDownArrows(VK_RIGHT, S_RIGHT);
if (_Valuechanged[8] & !this[8])
   SimulateKeyUpArrows(VK_RIGHT, S_RIGHT);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   SimulateKeyDown(VK_LeftShift, S_LeftShift);
if (_Valuechanged[9] & !this[9])
   SimulateKeyUp(VK_LeftShift, S_LeftShift);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   SimulateKeyDown(VK_Space, S_Space);
if (_Valuechanged[10] & !this[10])
   SimulateKeyUp(VK_Space, S_Space);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   SimulateKeyDown(VK_B, S_B);
if (_Valuechanged[29] & !this[29])
   SimulateKeyUp(VK_B, S_B);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   SimulateKeyDown(VK_N, S_N);
if (_Valuechanged[28] & !this[28])
   SimulateKeyUp(VK_N, S_N);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SimulateKeyDown(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SimulateKeyUp(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SimulateKeyDown(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
```

```
                SimulateKeyUp(VK_Q, S_Q);
            if (_Valuechanged[18] & this[18])
                SimulateKeyDown(VK_Z, S_Z);
            if (_Valuechanged[18] & !this[18])
                SimulateKeyUp(VK_Z, S_Z);
            if (_Valuechanged[19] & this[19])
                SimulateKeyDown(VK_S, S_S);
            if (_Valuechanged[19] & !this[19])
                SimulateKeyUp(VK_S, S_S);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                SimulateKeyDown(VK_A, S_A);
            if (_Valuechanged[30] & !this[30])
                SimulateKeyUp(VK_A, S_A);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                SimulateKeyDown(VK_E, S_E);
            if (_Valuechanged[31] & !this[31])
                SimulateKeyUp(VK_E, S_E);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                SimulateKeyDown(VK_R, S_R);
            if (_Valuechanged[13] & !this[13])
                SimulateKeyUp(VK_R, S_R);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                SimulateKeyDown(VK_Tab, S_Tab);
            if (_Valuechanged[20] & !this[20])
                SimulateKeyUp(VK_Tab, S_Tab);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                SimulateKeyDown(VK_C, S_C);
            if (_Valuechanged[21] & !this[21])
                SimulateKeyUp(VK_C, S_C);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                SimulateKeyDown(VK_F, S_F);
            if (_Valuechanged[22] & !this[22])
                SimulateKeyUp(VK_F, S_F);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                SimulateKeyDown(VK_U, S_U);
            if (_Valuechanged[23] & !this[23])
                SimulateKeyUp(VK_U, S_U);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                SimulateKeyDown(VK_Y, S_Y);
            if (_Valuechanged[24] & !this[24])
```

```
            SimulateKeyUp(VK_Y, S_Y);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            SimulateKeyDown(VK_X, S_X);
          if (_Valuechanged[25] & !this[25])
            SimulateKeyUp(VK_X, S_X);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            SimulateKeyDown(VK_Escape, S_Escape);
          if (_Valuechanged[26] & !this[26])
            SimulateKeyUp(VK_Escape, S_Escape);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            SimulateKeyDown(VK_G, S_G);
          if (_Valuechanged[14] & !this[14])
            SimulateKeyUp(VK_G, S_G);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            SimulateKeyDown(VK_T, S_T);
          if (_Valuechanged[15] & !this[15])
            SimulateKeyUp(VK_T, S_T);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            LeftClick();
          if (_Valuechanged[11] & !this[11])
            LeftClickF();
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
              mWSButtonStateAio = false;
```

```
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                RightClick();
            if (_Valuechanged[33] & !this[33])
                RightClickF();
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                RightClick();
            if (_Valuechanged[34] & !this[34])
                RightClickF();
        }
    }
```

WiiJoy4StreetsOfRogue.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
```

```
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
```

```
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
```

```
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
                  randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
              mWSButtonStateAio = false;
              randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
              input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
              input.SendRightClickF(11);
          }
          else
          {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
              input.SendRightClick(11);
```

```
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4SuperDuperFlyingGenocideSimulator2017.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
```

```
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
```

```
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
```

```
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
```

```
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4SuperHotMindControlDelete.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, irx, iry, zoning = 300f, center = 200f, slowing;
         public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f), 11);
             input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
                slowing = 0;
            }
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_Return, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
```

```
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Return, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Return, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
```

```
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
```

```
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                       if (keys456 == 2)
                        {
                           input.SendKeyF(Keys.Six, 1);
                           keys456 = 0;
                        }
            }
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
             input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
             input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
             input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
             input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
              {
                  WheelUpF();
             input.SendKey((Keys)S_G, 1);
              }
          if (_Valuechanged[14] & !this[14])
             input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
              {
                  WheelDownF();
             input.SendKey((Keys)S_T, 1);
              }
          if (_Valuechanged[15] & !this[15])
             input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
             input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
             input.SendLeftClickF(11);
          if (ApressIO)
          {
             foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4SurvivalZombiesTIE.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
           MoveMouseTo((int)(-mousex), (int)(mousey));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Terraria.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

```
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
```

```
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
```

```
if (_Valuechanged[27] & this[27])
   input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
```

```
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
```

```
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
                mWSButtonStateAio = true;
                randA = true;
              }
              else
                if (randA)
                {
                  mWSButtonStateAio = false;
```

```
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4TheCave.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4TheCycle.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 300f, center = 200f, slowing;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
                slowing += watchM;
                if (slowing >= 14)
                {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseBy((int)(-mousex), (int)(mousey), 11);
             input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f +
32767.5), 11);
                slowing = 0;
              }
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateLeft;
if (_Valuechanged[23] & this[23])
      {
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateRight;
if (_Valuechanged[24] & this[24])
```

```
      {
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus;
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus;
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
```

```
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
            this[34] = mWSButtonStateA;
        if (_Valuechanged[34] & this[34])
            input.SendRightClick(11);
        if (_Valuechanged[34] & !this[34])
            input.SendRightClickF(11);
    }
```

WiiJoy4TheEscapists.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
        if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
        if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
```

```
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4TheEvilWithin2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 104f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          mousexp += mousex * watchM / 50f;
          mouseyp += mousey * watchM / 50f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false, forstickoff, randS, LeftButtonSHOULDER_2io;
                  public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
        {
            forstickoff = GetStickLeftY <= 0.33f;
            this[1] = LeftButtonSHOULDER_2;
            if (_Valuechanged[1] & this[1])
               if (!randS)
               {
                  LeftButtonSHOULDER_2io = true;
                  randS = true;
               }
               else
                  if (randS)
                  {
                     LeftButtonSHOULDER_2io = false;
                     randS = false;
                  }
            if (LeftButtonSHOULDER_2io & forstickoff)
            {
               LeftButtonSHOULDER_2io = false;
               randS = false;
            }
          this[2] = LeftButtonSHOULDER_2io;
          if (_Valuechanged[2] & this[2])
            input.SendKey((Keys)S_LeftShift, 1);
          if (_Valuechanged[2] & !this[2])
            input.SendKeyF((Keys)S_LeftShift, 1);
          this[3] = LeftButtonMINUS;
          if (_Valuechanged[3] & this[3])
            input.SendKey((Keys)S_Return, 1);
          if (_Valuechanged[3] & !this[3])
            input.SendKeyF((Keys)S_Return, 1);
          this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
          if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
          if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
          this[27] = LeftButtonCAPTURE;
          if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
          if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
          this[5] = LeftButtonDPAD_UP;
          if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
          if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
          this[6] = LeftButtonDPAD_LEFT;
          if (_Valuechanged[6] & this[6])
```

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
            }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4TheForest.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 270f, hardness = 33f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
```

```
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 14)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
            input.MoveMouseBy((int)(-mousex * 4f), (int)(mousey * 4f), 11);
                slowing = 0;
            }
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
```

```
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4TheMasterplan.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
```

```
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
        if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
        this[23] = mWSButtonStateRight;
        if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
        if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
        this[24] = mWSButtonStateLeft;
        if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
        if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
```

```
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey(Keys.Control, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF(Keys.Control, 1);
```

```
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
```

```
            input.SendKey((Keys)S_B, 1);
          if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
          this[28] = LeftButtonSR;
          if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
          if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
          this[16] = GetStickLeftX > 0.33f;
          this[17] = GetStickLeftX < -0.33f;
          this[18] = GetStickLeftY > 0.33f;
          this[19] = GetStickLeftY < -0.33f;
          if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
          if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
          if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
          if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
          if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
          if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
          if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
```

```
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
```

```
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4TheTalosPrinciple.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -
max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f +
1f);
           MoveMouseTo((int)(-mousex), (int)(mousey));
              slowing = 0;
            }
         System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
                input.SendLeftClick(11);
                mWSButtonStateAio = true;
                randA = true;
                }
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
```

```
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }

WiiJoy4TheTypingOfTheDeadOverkill.txt
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
```

```
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY +
center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f,
-1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                slowing = 0;
            }
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
                if (_Valuechanged[10] & this[10])
                    input.SendKey((Keys)S_Space, 1);
                if (_Valuechanged[10] & !this[10])
                    input.SendKeyF((Keys)S_Space, 1);
                this[29] = LeftButtonSL;
                if (_Valuechanged[29] & this[29])
                    input.SendKey((Keys)S_B, 1);
                if (_Valuechanged[29] & !this[29])
                    input.SendKeyF((Keys)S_B, 1);
                this[28] = LeftButtonSR;
                if (_Valuechanged[28] & this[28])
                    input.SendKey((Keys)S_N, 1);
                if (_Valuechanged[28] & !this[28])
```

```
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
        if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
        this[20] = mWSButtonStateOne;
        if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
        if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
        this[21] = mWSButtonStateDown;
        if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
        if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
        this[22] = mWSButtonStateHome;
        if (_Valuechanged[22] & this[22])
```

```
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
```

```
                {
                    input.SendKeyF(Keys.Four, 1);
                    keys456 = 1;
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            {
            input.SendLeftClick(11);
            mWSButtonStateAio = true;
            randA = true;
            }
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
```

```
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4TheWay.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale, double maxScale)
{
  double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
  return scaled;
}
```

```
//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
```

```
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
```

```
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
```

```
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
```

```
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4TheWitcher3.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
```

```
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public double wheelplus, wheelminus;
```

```
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
```

```
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[6] & !this[6])
   SendKeyF(VK_Q, S_Q);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
   SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
```

```
        else
            wheelminusbool = false;
        if (wheelminus < -20f)
            wheelminus = 0;
        if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
        else
        {
            wheelplus = 0;
            wheelplusbool = false;
        }
        if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
        if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
        else
        {
            wheelminus = 0;
            wheelminusbool = false;
        }
        if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
   input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   input.SendKey((Keys)S_E, 1);
```

```
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
```

```
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Thief.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
```

```
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
   input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
```

```
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
```

```
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
               input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
```

```
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4ThinkToDie.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
```

```
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
```

```
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
               input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
```

```
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
                input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
                input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
```

```
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
```

```
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ThinkToDie2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
  irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
  iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
  mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
  mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
  System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
    SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
    SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
    SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
}
private double Scale(double value, double min, double max, double minScale,
double maxScale)
{
```

```
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
```

```
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
```

```
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
```

```
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
              if (!randA)
              {
```

```
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Titanfall.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          MoveMouseTo((int)(-mousex), (int)(mousey));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
```

```
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
```

```
if (_Valuechanged[27] & this[27])
   input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
   input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
   input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
   input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   input.SendKey((Keys)S_D, 1);
```

```
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
```

```
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
        }
if (_Valuechanged[23] & !this[23])
        {
    input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
        }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
        {
    input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
        }
if (_Valuechanged[24] & !this[24])
        {
    input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
```

```
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                {
                    WheelUpF();
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
```

```
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Tokyo42.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
```

```
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
```

```
if (_Valuechanged[5] & this[5])
   input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
   input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[6] & !this[6])
   SendKeyF(VK_Q, S_Q);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
   input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
   input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
   SendKey(VK_D, S_D);
if (_Valuechanged[8] & !this[8])
   SendKeyF(VK_D, S_D);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
   input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
   input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
   input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
   input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
   input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
   input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
   input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
   input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
```

```
            wheelminus = 0;
        if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
        else
        {
            wheelplus = 0;
            wheelplusbool = false;
        }
        if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
        if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
        else
        {
            wheelminus = 0;
            wheelminusbool = false;
        }
        if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
   SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
   SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
   SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
   SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
   input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
   input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
   input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
   input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
   input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
   input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
   input.SendKey((Keys)S_E, 1);
if (_Valuechanged[31] & !this[31])
   input.SendKeyF((Keys)S_E, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_S, 1);
```

```
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
```

```
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
                double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
                return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
            {
                this[2] = LeftButtonSHOULDER_2;
                if (_Valuechanged[2] & this[2])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[2] & !this[2])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[3] = LeftButtonMINUS;
                if (_Valuechanged[3] & this[3])
                    input.SendKey((Keys)S_E, 1);
                if (_Valuechanged[3] & !this[3])
                    input.SendKeyF((Keys)S_E, 1);
                this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
        }
    }
```

WiiJoy4TomClancyTheDivision.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 300f, hardness = 600f, center = 200f, max = 1500f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            slowing += watchM;
            if (slowing >= 30)
            {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
                mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
                MoveMouseBy((int)(-mousex / 1.5f), (int)(mousey / 1.5f));
                    slowing = 0;
                }
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
            {
                this[2] = LeftButtonSHOULDER_2;
                if (_Valuechanged[2] & this[2])
                    input.SendKey(Keys.Control, 1);
                if (_Valuechanged[2] & !this[2])
                    input.SendKeyF(Keys.Control, 1);
                this[3] = LeftButtonMINUS;
                if (_Valuechanged[3] & this[3])
                    input.SendKey(Keys.Enter, 1);
                if (_Valuechanged[3] & !this[3])
                    input.SendKeyF(Keys.Enter, 1);
                this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
```

```
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                {
               input.SendLeftClick(11);
               mWSButtonStateAio = true;
               randA = true;
                }
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft | mWSButtonStateRight
| (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) | LeftButtonSHOULDER_2);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4Tower57.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
                SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
                SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
                SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[2] = LeftButtonSHOULDER_2;
               if (_Valuechanged[2] & this[2])
                  input.SendKey(Keys.Control, 1);
```

```
            if (_Valuechanged[2] & !this[2])
               input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
               input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
               input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
```

```
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
```

```
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
   input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
   input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
   input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
   input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
   input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
   input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
   input.SendLeftClickF(11);
if (ApressIO)
{
```

```
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
               if (!randA)
               {
                  mWSButtonStateAio = true;
                  randA = true;
               }
               else
                  if (randA)
                  {
                     mWSButtonStateAio = false;
                     randA = false;
                  }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
                  input.SendRightClickF(11);
            }
            else
            {
               this[34] = mWSButtonStateA;
               if (_Valuechanged[34] & this[34])
                  input.SendRightClick(11);
               if (_Valuechanged[34] & !this[34])
                  input.SendRightClickF(11);
            }
         }
```

WiiJoy4TransmissionsElement120.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
         public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 100f, hardness = 520f, center = 200f, max = 1720f, slowing;
         public                double                WidthS                =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public                double                HeightS                =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
```

```
public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
    {
        irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
        iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max,
0f));
        mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f /
Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
        mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max /
Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone /
100f + 1f);
        MoveMouseTo((int)(-mousex), (int)(mousey));
            System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
    }
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF((Keys)S_E, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
```

```
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
      {
            WheelDownF();
   input.SendKey((Keys)S_T, 1);
      }
if (_Valuechanged[15] & !this[15])
   input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
   input.SendLeftClick(11);
if (_Valuechanged[11] & !this[11])
```

```
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4VoidRaiders.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
```

```
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
}
```

```
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
               input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
               input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
                input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
                input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
                input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
                input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
                input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
                input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
                input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
```

```
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
               input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
               input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
               input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
               input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
```

```
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4Warframe.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
```

```
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
```

```
    input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
    input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
    input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
    input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
    input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
    input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
```

```
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
```

```
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
```

```
                }
                else
                    if (keys456 == 1)
                    {
                        input.SendKeyF(Keys.Five, 1);
                        keys456 = 2;
                    }
                    else
                        if (keys456 == 2)
                        {
                            input.SendKeyF(Keys.Six, 1);
                            keys456 = 0;
                        }
            }
        this[25] = mWSButtonStateUp;
        if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
        if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
        this[26] = mWSButtonStateTwo;
        if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            {
                WheelUpF();
            input.SendKey((Keys)S_G, 1);
            }
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            {
                WheelDownF();
            input.SendKey((Keys)S_T, 1);
            }
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
```

```
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4WatchDogs.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,

float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,

float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,

bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)

{

irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));

iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));

mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);

mousexp += mousex * watchM / 10f;

mouseyp += mousey * watchM / 10f;

MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));

System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));

}

private double Scale(double value, double min, double max, double minScale, double maxScale)

```
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
            public double wheelplus, wheelminus;
        public bool wheelplusbool, wheelminusbool;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
```

```
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
               input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
               input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
               input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
               input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
               input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
               input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
               SendKey(VK_Q, S_Q);
            if (_Valuechanged[6] & !this[6])
               SendKeyF(VK_Q, S_Q);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
               input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
               input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
               SendKey(VK_D, S_D);
            if (_Valuechanged[8] & !this[8])
               SendKeyF(VK_D, S_D);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
               input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
               input.SendKeyF((Keys)S_LeftShift, 1);
            this[10] = LeftButtonSHOULDER_1;
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
```

```
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
      if (wheelplus >= 10f & wheelplus <= 20f)
            wheelplusbool = true;
      else
            wheelplusbool = false;
      if (wheelplus > 20f)
            wheelplus = 0;
      if (wheelminus <= -10f & wheelminus >= -20f)
            wheelminusbool = true;
      else
            wheelminusbool = false;
      if (wheelminus < -20f)
            wheelminus = 0;
      if (GetStickLeftX > 0.1f & GetStickLeftX < 0.5f)
            wheelplus += GetStickLeftX * watchM;
      else
      {
            wheelplus = 0;
            wheelplusbool = false;
      }
      if (GetStickLeftX >= 0.5f)
            wheelplusbool = true;
      if (GetStickLeftX < -0.1f & GetStickLeftX > -0.5f)
            wheelminus += GetStickLeftX * watchM;
      else
      {
            wheelminus = 0;
            wheelminusbool = false;
      }
      if (GetStickLeftX <= -0.5f)
            wheelminusbool = true;
this[16] = wheelplusbool;
this[17] = wheelminusbool;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    SendKey(VK_D, S_D);
if (_Valuechanged[16] & !this[16])
    SendKeyF(VK_D, S_D);
if (_Valuechanged[17] & this[17])
    SendKey(VK_Q, S_Q);
if (_Valuechanged[17] & !this[17])
    SendKeyF(VK_Q, S_Q);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
                input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
```

```
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_Z, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_S, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
               }
               this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
               if (_Valuechanged[33] & this[33])
                  input.SendRightClick(11);
               if (_Valuechanged[33] & !this[33])
```

```
                input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4WolfensteinTheNewOrder.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
```

```
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale, double maxScale)
            {
               double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
               return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
```

```
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
```

```
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
```

```
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
                  {
                     input.SendKeyF(Keys.One, 1);
                     keys123 = 1;
                  }
                  else
```

```
            if (keys123 == 1)
            {
                input.SendKeyF(Keys.Two, 1);
                keys123 = 2;
            }
            else
               if (keys123 == 2)
                  {
                     input.SendKeyF(Keys.Three, 1);
                     keys123 = 0;
                  }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
```

```
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
                     }
               if (mWSButtonStateAio & foraorcison)
               {
                  mWSButtonStateAio = false;
                  randA = false;
```

```
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4WolfensteinTheOldBlood.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning
= 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS =
System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
```

```
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
          mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
          mousexp += mousex * watchM / 10f;
          mouseyp += mousey * watchM / 10f;
          MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp), (int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
            SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
this[2] = LeftButtonSHOULDER_2;
if (_Valuechanged[2] & this[2])
input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[2] & !this[2])
input.SendKeyF((Keys)S_LeftShift, 1);
this[3] = LeftButtonMINUS;
if (_Valuechanged[3] & this[3])
input.SendKey((Keys)S_E, 1);
if (_Valuechanged[3] & !this[3])
input.SendKeyF((Keys)S_E, 1);
this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
if (_Valuechanged[4] & this[4])
input.SendKey(Keys.V, 1);
if (_Valuechanged[4] & !this[4])
input.SendKeyF(Keys.V, 1);
this[27] = LeftButtonCAPTURE;
if (_Valuechanged[27] & this[27])
input.SendKey(Keys.P, 1);
if (_Valuechanged[27] & !this[27])
input.SendKeyF(Keys.P, 1);
this[5] = LeftButtonDPAD_UP;
if (_Valuechanged[5] & this[5])
input.SendKey((Keys)S_UP, 1);
if (_Valuechanged[5] & !this[5])
input.SendKeyF((Keys)S_UP, 1);
this[6] = LeftButtonDPAD_LEFT;
if (_Valuechanged[6] & this[6])

```
    input.SendKey((Keys)S_LEFT, 1);
if (_Valuechanged[6] & !this[6])
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
```

```
            input.SendKey((Keys)S_S, 1);
          if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
          this[30] = DirectAnglesLeftY <= -0.666f;
          if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
          if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
          this[31] = DirectAnglesLeftY >= 0.666f;
          if (_Valuechanged[31] & this[31])
            input.SendKey(Keys.H, 1);
          if (_Valuechanged[31] & !this[31])
            input.SendKeyF(Keys.H, 1);
          this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
          if (_Valuechanged[13] & this[13])
            input.SendKey((Keys)S_R, 1);
          if (_Valuechanged[13] & !this[13])
            input.SendKeyF((Keys)S_R, 1);
          this[20] = mWSButtonStateOne;
          if (_Valuechanged[20] & this[20])
            input.SendKey((Keys)S_Tab, 1);
          if (_Valuechanged[20] & !this[20])
            input.SendKeyF((Keys)S_Tab, 1);
          this[21] = mWSButtonStateDown;
          if (_Valuechanged[21] & this[21])
            input.SendKey((Keys)S_C, 1);
          if (_Valuechanged[21] & !this[21])
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
              {
            input.SendKey((Keys)S_U, 1);
                  if (keys123 == 0)
                    input.SendKey(Keys.One, 1);
                  if (keys123 == 1)
                    input.SendKey(Keys.Two, 1);
                  if (keys123 == 2)
                    input.SendKey(Keys.Three, 1);
              }
          if (_Valuechanged[23] & !this[23])
              {
            input.SendKeyF((Keys)S_U, 1);
                  if (keys123 == 0)
```

```
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
```

```
            if (_Valuechanged[25] & this[25])
               input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
               input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
               input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
               input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                  {
                        WheelUpF();
               input.SendKey((Keys)S_G, 1);
                  }
            if (_Valuechanged[14] & !this[14])
               input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                  {
                        WheelDownF();
               input.SendKey((Keys)S_T, 1);
                  }
            if (_Valuechanged[15] & !this[15])
               input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
               input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
               input.SendLeftClickF(11);
            if (ApressIO)
            {
               foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
               this[32] = mWSButtonStateA;
               if (_Valuechanged[32] & this[32])
                  if (!randA)
                  {
                     mWSButtonStateAio = true;
                     randA = true;
                  }
                  else
                     if (randA)
                     {
                        mWSButtonStateAio = false;
                        randA = false;
```

```
                }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiiJoy4Zenge.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
```

```
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f,
1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center,
0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f,
0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f /
Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f /
Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey *
HeightS / 1024f));
            SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f),
(int)(HeightS + mousey * HeightS / 1024f));
            SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
            SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS +
mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale,
double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG
= false;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
```

```
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
```

```
    input.SendKeyF((Keys)S_LEFT, 1);
this[7] = LeftButtonDPAD_DOWN;
if (_Valuechanged[7] & this[7])
    input.SendKey((Keys)S_DOWN, 1);
if (_Valuechanged[7] & !this[7])
    input.SendKeyF((Keys)S_DOWN, 1);
this[8] = LeftButtonDPAD_RIGHT;
if (_Valuechanged[8] & this[8])
    input.SendKey((Keys)S_RIGHT, 1);
if (_Valuechanged[8] & !this[8])
    input.SendKeyF((Keys)S_RIGHT, 1);
this[9] = LeftButtonSTICK;
if (_Valuechanged[9] & this[9])
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
```

```
                input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
                input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
                input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
                input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
                input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
                input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
                input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
                input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
                input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
                input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
                input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
                input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                input.SendKey((Keys)S_U, 1);
            if (_Valuechanged[23] & !this[23])
                input.SendKeyF((Keys)S_U, 1);
            this[24] = mWSButtonStateLeft;
            if (_Valuechanged[24] & this[24])
                input.SendKey((Keys)S_Y, 1);
            if (_Valuechanged[24] & !this[24])
                input.SendKeyF((Keys)S_Y, 1);
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
```

```
            input.SendKey((Keys)S_Escape, 1);
        if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
        this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
        if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
        this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
        if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
        if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
        this[11] = mWSButtonStateB;
        if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
        if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
        if (ApressIO)
        {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
            if (mWSButtonStateAio & foraorcison)
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
```

```
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ZenoClash.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG = false;
            public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
```

```
            {
                this[2] = LeftButtonSHOULDER_2;
                if (_Valuechanged[2] & this[2])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[2] & !this[2])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[3] = LeftButtonMINUS;
                if (_Valuechanged[3] & this[3])
                    input.SendKey((Keys)S_E, 1);
                if (_Valuechanged[3] & !this[3])
                    input.SendKeyF((Keys)S_E, 1);
                this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
                if (_Valuechanged[4] & this[4])
                    input.SendKey(Keys.V, 1);
                if (_Valuechanged[4] & !this[4])
                    input.SendKeyF(Keys.V, 1);
                this[27] = LeftButtonCAPTURE;
                if (_Valuechanged[27] & this[27])
                    input.SendKey(Keys.P, 1);
                if (_Valuechanged[27] & !this[27])
                    input.SendKeyF(Keys.P, 1);
                this[5] = LeftButtonDPAD_UP;
                if (_Valuechanged[5] & this[5])
                    input.SendKey((Keys)S_UP, 1);
                if (_Valuechanged[5] & !this[5])
                    input.SendKeyF((Keys)S_UP, 1);
                this[6] = LeftButtonDPAD_LEFT;
                if (_Valuechanged[6] & this[6])
                    input.SendKey((Keys)S_LEFT, 1);
                if (_Valuechanged[6] & !this[6])
                    input.SendKeyF((Keys)S_LEFT, 1);
                this[7] = LeftButtonDPAD_DOWN;
                if (_Valuechanged[7] & this[7])
                    input.SendKey((Keys)S_DOWN, 1);
                if (_Valuechanged[7] & !this[7])
                    input.SendKeyF((Keys)S_DOWN, 1);
                this[8] = LeftButtonDPAD_RIGHT;
                if (_Valuechanged[8] & this[8])
                    input.SendKey((Keys)S_RIGHT, 1);
                if (_Valuechanged[8] & !this[8])
                    input.SendKeyF((Keys)S_RIGHT, 1);
                this[9] = LeftButtonSTICK;
                if (_Valuechanged[9] & this[9])
                    input.SendKey((Keys)S_LeftShift, 1);
                if (_Valuechanged[9] & !this[9])
                    input.SendKeyF((Keys)S_LeftShift, 1);
                this[10] = LeftButtonSHOULDER_1;
```

```
            if (_Valuechanged[10] & this[10])
               input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
               input.SendKeyF((Keys)S_Space, 1);
            this[29] = LeftButtonSL;
            if (_Valuechanged[29] & this[29])
               input.SendKey((Keys)S_B, 1);
            if (_Valuechanged[29] & !this[29])
               input.SendKeyF((Keys)S_B, 1);
            this[28] = LeftButtonSR;
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey(Keys.H, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF(Keys.H, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
```

```
   input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
   input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
   input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
   input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
   input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
   input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
   input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
      {
   input.SendKey((Keys)S_U, 1);
            if (keys123 == 0)
              input.SendKey(Keys.One, 1);
            if (keys123 == 1)
              input.SendKey(Keys.Two, 1);
            if (keys123 == 2)
              input.SendKey(Keys.Three, 1);
      }
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
```

```
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
                        keys456 = 0;
                     }
      }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
   input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
   input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
   input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
   input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
      {
            WheelUpF();
   input.SendKey((Keys)S_G, 1);
      }
if (_Valuechanged[14] & !this[14])
   input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
```

```
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
```

```
            }
        }
```

WiiJoy4ZenoClash2.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 460f, hardness = 520f, center = 200f, max = 1720f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = (Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) + 0.9) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = (Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) + 0.9) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
```

```
            mousexp += mousex * watchM / 10f;
            mouseyp += mousey * watchM / 10f;
            MoveMouseTo((int)(32767.5f - (mousex * hardness) / 100f - mousexp),
(int)((mousey * hardness) / 100f + mouseyp + 32767.5f));
                System.Windows.Forms.Cursor.Position = new
System.Drawing.Point((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry * HeightS /
1024f));
                SetPhysicalCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS +
iry * HeightS / 1024f));
                SetCaretPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
                SetCursorPos((int)(WidthS - irx * WidthS / 1024f), (int)(HeightS + iry *
HeightS / 1024f));
            }
            private double Scale(double value, double min, double max, double minScale,
double maxScale)
            {
              double scaled = minScale + (double)(value - min) / (max - min) * (maxScale -
minScale);
              return scaled;
            }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
            public bool foraorcison, mWSButtonStateAio, randA, ApressIO = true, HomeFTG
= false;
                public int keys123, keys456;
            public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
            double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
            float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
            float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
            bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF((Keys)S_E, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
```

```
    input.SendKey((Keys)S_LeftShift, 1);
if (_Valuechanged[9] & !this[9])
    input.SendKeyF((Keys)S_LeftShift, 1);
this[10] = LeftButtonSHOULDER_1;
if (_Valuechanged[10] & this[10])
    input.SendKey((Keys)S_Space, 1);
if (_Valuechanged[10] & !this[10])
    input.SendKeyF((Keys)S_Space, 1);
this[29] = LeftButtonSL;
if (_Valuechanged[29] & this[29])
    input.SendKey((Keys)S_B, 1);
if (_Valuechanged[29] & !this[29])
    input.SendKeyF((Keys)S_B, 1);
this[28] = LeftButtonSR;
if (_Valuechanged[28] & this[28])
    input.SendKey((Keys)S_N, 1);
if (_Valuechanged[28] & !this[28])
    input.SendKeyF((Keys)S_N, 1);
this[16] = GetStickLeftX > 0.33f;
this[17] = GetStickLeftX < -0.33f;
this[18] = GetStickLeftY > 0.33f;
this[19] = GetStickLeftY < -0.33f;
if (_Valuechanged[16] & this[16])
    input.SendKey((Keys)S_D, 1);
if (_Valuechanged[16] & !this[16])
    input.SendKeyF((Keys)S_D, 1);
if (_Valuechanged[17] & this[17])
    input.SendKey((Keys)S_Q, 1);
if (_Valuechanged[17] & !this[17])
    input.SendKeyF((Keys)S_Q, 1);
if (_Valuechanged[18] & this[18])
    input.SendKey((Keys)S_Z, 1);
if (_Valuechanged[18] & !this[18])
    input.SendKeyF((Keys)S_Z, 1);
if (_Valuechanged[19] & this[19])
    input.SendKey((Keys)S_S, 1);
if (_Valuechanged[19] & !this[19])
    input.SendKeyF((Keys)S_S, 1);
this[30] = DirectAnglesLeftY <= -0.666f;
if (_Valuechanged[30] & this[30])
    input.SendKey((Keys)S_A, 1);
if (_Valuechanged[30] & !this[30])
    input.SendKeyF((Keys)S_A, 1);
this[31] = DirectAnglesLeftY >= 0.666f;
if (_Valuechanged[31] & this[31])
    input.SendKey(Keys.H, 1);
if (_Valuechanged[31] & !this[31])
    input.SendKeyF(Keys.H, 1);
```

```
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                          input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                          input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                          input.SendKey(Keys.Three, 1);
                  }
            if (_Valuechanged[23] & !this[23])
                  {
               input.SendKeyF((Keys)S_U, 1);
                        if (keys123 == 0)
                        {
                           input.SendKeyF(Keys.One, 1);
                           keys123 = 1;
                        }
                        else
                           if (keys123 == 1)
                           {
                                 input.SendKeyF(Keys.Two, 1);
                                 keys123 = 2;
                           }
                           else
                              if (keys123 == 2)
                                 {
```

```
                    input.SendKeyF(Keys.Three, 1);
                    keys123 = 0;
                }
    }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    {
  input.SendKey((Keys)S_Y, 1);
        if (keys456 == 0)
          input.SendKey(Keys.Four, 1);
        if (keys456 == 1)
          input.SendKey(Keys.Five, 1);
        if (keys456 == 2)
          input.SendKey(Keys.Six, 1);
    }
if (_Valuechanged[24] & !this[24])
    {
  input.SendKeyF((Keys)S_Y, 1);
        if (keys456 == 0)
        {
          input.SendKeyF(Keys.Four, 1);
          keys456 = 1;
        }
        else
          if (keys456 == 1)
          {
              input.SendKeyF(Keys.Five, 1);
              keys456 = 2;
          }
          else
            if (keys456 == 2)
              {
                input.SendKeyF(Keys.Six, 1);
                keys456 = 0;
              }
    }
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
  input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
  input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
  input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
  input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    {
        WheelUpF();
```

```
                input.SendKey((Keys)S_G, 1);
                }
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                {
                    WheelDownF();
                input.SendKey((Keys)S_T, 1);
                }
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
            }
            else
```

```
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiiJoy4ZombieNightTerror.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, mousexp, mouseyp, irx, iry, deadzone = 0f, zoning = 100f, hardness = 330f, center = 200f, max = 1500f, slowing;
            public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
            public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1500f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1500f, 0f));
```

```
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, max) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -max, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1500f / Math.Pow(1500f, zoning / 100f)) * (irx > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning * (1500f / max) / 100f) * (max / Math.Pow(max, zoning * (1500f / max) / 100f)) * (iry > 0 ? 1f : -1f) * (-deadzone / 100f + 1f);
            System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[27] = LeftButtonCAPTURE;
            if (_Valuechanged[27] & this[27])
                input.SendKey(Keys.P, 1);
            if (_Valuechanged[27] & !this[27])
                input.SendKeyF(Keys.P, 1);
            this[5] = LeftButtonDPAD_UP;
            if (_Valuechanged[5] & this[5])
                input.SendKey((Keys)S_UP, 1);
            if (_Valuechanged[5] & !this[5])
                input.SendKeyF((Keys)S_UP, 1);
            this[6] = LeftButtonDPAD_LEFT;
            if (_Valuechanged[6] & this[6])
                input.SendKey((Keys)S_LEFT, 1);
            if (_Valuechanged[6] & !this[6])
                input.SendKeyF((Keys)S_LEFT, 1);
            this[7] = LeftButtonDPAD_DOWN;
            if (_Valuechanged[7] & this[7])
                input.SendKey((Keys)S_DOWN, 1);
            if (_Valuechanged[7] & !this[7])
                input.SendKeyF((Keys)S_DOWN, 1);
            this[8] = LeftButtonDPAD_RIGHT;
            if (_Valuechanged[8] & this[8])
                input.SendKey((Keys)S_RIGHT, 1);
            if (_Valuechanged[8] & !this[8])
                input.SendKeyF((Keys)S_RIGHT, 1);
            this[9] = LeftButtonSTICK;
            if (_Valuechanged[9] & this[9])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[9] & !this[9])
```

```
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
        if (_Valuechanged[28] & this[28])
            input.SendKey((Keys)S_N, 1);
        if (_Valuechanged[28] & !this[28])
            input.SendKeyF((Keys)S_N, 1);
        this[16] = GetStickLeftX > 0.33f;
        this[17] = GetStickLeftX < -0.33f;
        this[18] = GetStickLeftY > 0.33f;
        this[19] = GetStickLeftY < -0.33f;
        if (_Valuechanged[16] & this[16])
            input.SendKey((Keys)S_D, 1);
        if (_Valuechanged[16] & !this[16])
            input.SendKeyF((Keys)S_D, 1);
        if (_Valuechanged[17] & this[17])
            input.SendKey((Keys)S_Q, 1);
        if (_Valuechanged[17] & !this[17])
            input.SendKeyF((Keys)S_Q, 1);
        if (_Valuechanged[18] & this[18])
            input.SendKey((Keys)S_Z, 1);
        if (_Valuechanged[18] & !this[18])
            input.SendKeyF((Keys)S_Z, 1);
        if (_Valuechanged[19] & this[19])
            input.SendKey((Keys)S_S, 1);
        if (_Valuechanged[19] & !this[19])
            input.SendKeyF((Keys)S_S, 1);
        this[30] = DirectAnglesLeftY <= -0.666f;
        if (_Valuechanged[30] & this[30])
            input.SendKey((Keys)S_A, 1);
        if (_Valuechanged[30] & !this[30])
            input.SendKeyF((Keys)S_A, 1);
        this[31] = DirectAnglesLeftY >= 0.666f;
        if (_Valuechanged[31] & this[31])
            input.SendKey((Keys)S_E, 1);
        if (_Valuechanged[31] & !this[31])
            input.SendKeyF((Keys)S_E, 1);
        this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
        if (_Valuechanged[13] & this[13])
```

```
    input.SendKey((Keys)S_R, 1);
if (_Valuechanged[13] & !this[13])
    input.SendKeyF((Keys)S_R, 1);
this[20] = mWSButtonStateOne;
if (_Valuechanged[20] & this[20])
    input.SendKey((Keys)S_Tab, 1);
if (_Valuechanged[20] & !this[20])
    input.SendKeyF((Keys)S_Tab, 1);
this[21] = mWSButtonStateDown;
if (_Valuechanged[21] & this[21])
    input.SendKey((Keys)S_C, 1);
if (_Valuechanged[21] & !this[21])
    input.SendKeyF((Keys)S_C, 1);
this[22] = mWSButtonStateHome;
if (_Valuechanged[22] & this[22])
    input.SendKey((Keys)S_F, 1);
if (_Valuechanged[22] & !this[22])
    input.SendKeyF((Keys)S_F, 1);
this[23] = mWSButtonStateRight;
if (_Valuechanged[23] & this[23])
    input.SendKey((Keys)S_U, 1);
if (_Valuechanged[23] & !this[23])
    input.SendKeyF((Keys)S_U, 1);
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
    input.SendKey((Keys)S_Y, 1);
if (_Valuechanged[24] & !this[24])
    input.SendKeyF((Keys)S_Y, 1);
this[25] = mWSButtonStateUp;
if (_Valuechanged[25] & this[25])
    input.SendKey((Keys)S_X, 1);
if (_Valuechanged[25] & !this[25])
    input.SendKeyF((Keys)S_X, 1);
this[26] = mWSButtonStateTwo;
if (_Valuechanged[26] & this[26])
    input.SendKey((Keys)S_Escape, 1);
if (_Valuechanged[26] & !this[26])
    input.SendKeyF((Keys)S_Escape, 1);
this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[14] & this[14])
    input.SendKey((Keys)S_G, 1);
if (_Valuechanged[14] & !this[14])
    input.SendKeyF((Keys)S_G, 1);
this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
if (_Valuechanged[15] & this[15])
    input.SendKey((Keys)S_T, 1);
if (_Valuechanged[15] & !this[15])
    input.SendKeyF((Keys)S_T, 1);
this[11] = mWSButtonStateB;
if (_Valuechanged[11] & this[11])
```

```
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                    if (mWSButtonStateAio & foraorcison)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                    this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                    if (_Valuechanged[33] & this[33])
                        input.SendRightClick(11);
                    if (_Valuechanged[33] & !this[33])
                        input.SendRightClickF(11);
                }
                else
                {
                    this[34] = mWSButtonStateA;
                    if (_Valuechanged[34] & this[34])
                        input.SendRightClick(11);
                    if (_Valuechanged[34] & !this[34])
                        input.SendRightClickF(11);
                }
            }
```

WiiJoy4ZupZup.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f;
```

```
public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
{
irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
System.Windows.Forms.Cursor.Position = new System.Drawing.Point((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetPhysicalCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCaretPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
SetCursorPos((int)(WidthS - mousex * WidthS / 1024f), (int)(HeightS + mousey * HeightS / 1024f));
```

```
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
            double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
            return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = LeftButtonSHOULDER_2;
            if (_Valuechanged[2] & this[2])
                input.SendKey(Keys.Control, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF(Keys.Control, 1);
            this[3] = LeftButtonMINUS;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
```

```
            input.SendKeyF(Keys.Enter, 1);
        this[4] = (GetAccelLeftX > 1.5f | GetAccelLeftX < -1.5f) & !((mWSRawValuesZ
> 0 ? mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
        if (_Valuechanged[4] & this[4])
            input.SendKey(Keys.V, 1);
        if (_Valuechanged[4] & !this[4])
            input.SendKeyF(Keys.V, 1);
        this[27] = LeftButtonCAPTURE;
        if (_Valuechanged[27] & this[27])
            input.SendKey(Keys.P, 1);
        if (_Valuechanged[27] & !this[27])
            input.SendKeyF(Keys.P, 1);
        this[5] = LeftButtonDPAD_UP;
        if (_Valuechanged[5] & this[5])
            input.SendKey((Keys)S_UP, 1);
        if (_Valuechanged[5] & !this[5])
            input.SendKeyF((Keys)S_UP, 1);
        this[6] = LeftButtonDPAD_LEFT;
        if (_Valuechanged[6] & this[6])
            input.SendKey((Keys)S_LEFT, 1);
        if (_Valuechanged[6] & !this[6])
            input.SendKeyF((Keys)S_LEFT, 1);
        this[7] = LeftButtonDPAD_DOWN;
        if (_Valuechanged[7] & this[7])
            input.SendKey((Keys)S_DOWN, 1);
        if (_Valuechanged[7] & !this[7])
            input.SendKeyF((Keys)S_DOWN, 1);
        this[8] = LeftButtonDPAD_RIGHT;
        if (_Valuechanged[8] & this[8])
            input.SendKey((Keys)S_RIGHT, 1);
        if (_Valuechanged[8] & !this[8])
            input.SendKeyF((Keys)S_RIGHT, 1);
        this[9] = LeftButtonSTICK;
        if (_Valuechanged[9] & this[9])
            input.SendKey((Keys)S_LeftShift, 1);
        if (_Valuechanged[9] & !this[9])
            input.SendKeyF((Keys)S_LeftShift, 1);
        this[10] = LeftButtonSHOULDER_1;
        if (_Valuechanged[10] & this[10])
            input.SendKey((Keys)S_Space, 1);
        if (_Valuechanged[10] & !this[10])
            input.SendKeyF((Keys)S_Space, 1);
        this[29] = LeftButtonSL;
        if (_Valuechanged[29] & this[29])
            input.SendKey((Keys)S_B, 1);
        if (_Valuechanged[29] & !this[29])
            input.SendKeyF((Keys)S_B, 1);
        this[28] = LeftButtonSR;
```

```
            if (_Valuechanged[28] & this[28])
               input.SendKey((Keys)S_N, 1);
            if (_Valuechanged[28] & !this[28])
               input.SendKeyF((Keys)S_N, 1);
            this[16] = GetStickLeftX > 0.33f;
            this[17] = GetStickLeftX < -0.33f;
            this[18] = GetStickLeftY > 0.33f;
            this[19] = GetStickLeftY < -0.33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_Q, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_Z, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_Z, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[30] = DirectAnglesLeftY <= -0.666f;
            if (_Valuechanged[30] & this[30])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[30] & !this[30])
               input.SendKeyF((Keys)S_A, 1);
            this[31] = DirectAnglesLeftY >= 0.666f;
            if (_Valuechanged[31] & this[31])
               input.SendKey((Keys)S_E, 1);
            if (_Valuechanged[31] & !this[31])
               input.SendKeyF((Keys)S_E, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
```

```
            input.SendKeyF((Keys)S_C, 1);
          this[22] = mWSButtonStateHome;
          if (_Valuechanged[22] & this[22])
            input.SendKey((Keys)S_F, 1);
          if (_Valuechanged[22] & !this[22])
            input.SendKeyF((Keys)S_F, 1);
          this[23] = mWSButtonStateRight;
          if (_Valuechanged[23] & this[23])
            input.SendKey((Keys)S_U, 1);
          if (_Valuechanged[23] & !this[23])
            input.SendKeyF((Keys)S_U, 1);
          this[24] = mWSButtonStateLeft;
          if (_Valuechanged[24] & this[24])
            input.SendKey((Keys)S_Y, 1);
          if (_Valuechanged[24] & !this[24])
            input.SendKeyF((Keys)S_Y, 1);
          this[25] = mWSButtonStateUp;
          if (_Valuechanged[25] & this[25])
            input.SendKey((Keys)S_X, 1);
          if (_Valuechanged[25] & !this[25])
            input.SendKeyF((Keys)S_X, 1);
          this[26] = mWSButtonStateTwo;
          if (_Valuechanged[26] & this[26])
            input.SendKey((Keys)S_Escape, 1);
          if (_Valuechanged[26] & !this[26])
            input.SendKeyF((Keys)S_Escape, 1);
          this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[14] & this[14])
            input.SendKey((Keys)S_G, 1);
          if (_Valuechanged[14] & !this[14])
            input.SendKeyF((Keys)S_G, 1);
          this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
          if (_Valuechanged[15] & this[15])
            input.SendKey((Keys)S_T, 1);
          if (_Valuechanged[15] & !this[15])
            input.SendKeyF((Keys)S_T, 1);
          this[11] = mWSButtonStateB;
          if (_Valuechanged[11] & this[11])
            input.SendLeftClick(11);
          if (_Valuechanged[11] & !this[11])
            input.SendLeftClickF(11);
          if (ApressIO)
          {
            foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
            this[32] = mWSButtonStateA;
```

```
            if (_Valuechanged[32] & this[32])
                if (!randA)
                {
                    mWSButtonStateAio = true;
                    randA = true;
                }
                else
                    if (randA)
                    {
                        mWSButtonStateAio = false;
                        randA = false;
                    }
                if (mWSButtonStateAio & foraorcison)
                {
                    mWSButtonStateAio = false;
                    randA = false;
                }
                this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
                if (_Valuechanged[33] & this[33])
                    input.SendRightClick(11);
                if (_Valuechanged[33] & !this[33])
                    input.SendRightClickF(11);
            }
            else
            {
                this[34] = mWSButtonStateA;
                if (_Valuechanged[34] & this[34])
                    input.SendRightClick(11);
                if (_Valuechanged[34] & !this[34])
                    input.SendRightClickF(11);
            }
        }
```

WiimoteNunchuck4HeatGuardian.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 100f, center = 200f, slowing;
        public double WidthS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / 2f;
        public double HeightS = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / 2f;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
```

```
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR, bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
            iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
            mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
            mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
            input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5), 11);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool foraorcison, mWSButtonStateAio, randA, ApressIO = false, HomeFTG = false;
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
```

```
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float
GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
            this[2] = mWSNunchuckStateZ;
            if (_Valuechanged[2] & this[2])
                input.SendKey((Keys)S_LeftShift, 1);
            if (_Valuechanged[2] & !this[2])
                input.SendKeyF((Keys)S_LeftShift, 1);
            this[3] = mWSNunchuckStateZ & mWSNunchuckStateC;
            if (_Valuechanged[3] & this[3])
                input.SendKey(Keys.Enter, 1);
            if (_Valuechanged[3] & !this[3])
                input.SendKeyF(Keys.Enter, 1);
            this[4] = (mWSNunchuckStateRawValuesY > 33f) & !((mWSRawValuesZ > 0 ?
mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
            if (_Valuechanged[4] & this[4])
                input.SendKey(Keys.V, 1);
            if (_Valuechanged[4] & !this[4])
                input.SendKeyF(Keys.V, 1);
            this[10] = mWSNunchuckStateC;
            if (_Valuechanged[10] & this[10])
                input.SendKey((Keys)S_Space, 1);
            if (_Valuechanged[10] & !this[10])
                input.SendKeyF((Keys)S_Space, 1);
            this[16] = mWSNunchuckStateRawJoystickX > 33f;
            this[17] = mWSNunchuckStateRawJoystickX < -33f;
            this[18] = mWSNunchuckStateRawJoystickY > 33f;
```

```
            this[19] = mWSNunchuckStateRawJoystickY < -33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_A, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_W, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_W, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
                  {
               input.SendKey((Keys)S_U, 1);
                        if (keys123 == 0)
                           input.SendKey(Keys.One, 1);
                        if (keys123 == 1)
                           input.SendKey(Keys.Two, 1);
                        if (keys123 == 2)
                           input.SendKey(Keys.Three, 1);
                  }
```

```
if (_Valuechanged[23] & !this[23])
      {
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
      }
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
      {
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
      }
if (_Valuechanged[24] & !this[24])
      {
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
                        input.SendKeyF(Keys.Six, 1);
```

```
                        keys456 = 0;
                    }
                }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_Q, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_Q, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            if (ApressIO)
            {
                foraorcison = (mWSButtonStateMinus | mWSButtonStatePlus |
mWSButtonStateHome | ((mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ)
>= 40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f) |
mWSButtonStateUp | mWSButtonStateDown | mWSButtonStateLeft |
mWSButtonStateRight);
                this[32] = mWSButtonStateA;
                if (_Valuechanged[32] & this[32])
                    if (!randA)
                    {
                        mWSButtonStateAio = true;
                        randA = true;
                    }
                    else
                        if (randA)
                        {
                            mWSButtonStateAio = false;
                            randA = false;
                        }
                if (mWSButtonStateAio & foraorcison)
```

```
            {
                mWSButtonStateAio = false;
                randA = false;
            }
            this[33] = mWSButtonStateAio | (mWSButtonStateA & foraorcison);
            if (_Valuechanged[33] & this[33])
                input.SendRightClick(11);
            if (_Valuechanged[33] & !this[33])
                input.SendRightClickF(11);
        }
        else
        {
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
    }
```

WiimoteNunchuck4ModernWarfare2019.txt

```
//mouse control
public static bool[] _Valuechanged = new bool[2], _valuechanged = new bool[2];
        public double mousex, mousey, irx, iry, zoning = 400f, center = 200f;
        public void Main(double mWSNunchuckStateRawJoystickX, double
mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double
mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool
mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool
mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool
mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool
mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool
mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double
mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX,
float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float
EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX,
float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float
EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ,
bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool
LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool
LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool
LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool
RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool
RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX,
double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float
```

```
GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
        {
          irx = (mWSButtonStateIRX >= 0 ? Scale(mWSButtonStateIRX, 0f, 1360f, 0f, 1360f) : Scale(mWSButtonStateIRX, -1360f, 0f, -1360f, 0f));
          iry = (mWSButtonStateIRY + center >= 0 ? Scale(mWSButtonStateIRY + center, 0f, 768f + center, 0f, 1150f) : Scale(mWSButtonStateIRY + center, -768f + center, 0f, -1150f, 0f));
          mousex = Math.Pow(irx > 0 ? irx : -irx, zoning / 100f) * (1360f / Math.Pow(1360f, zoning / 100f)) * (irx > 0 ? 1f : -1f);
          mousey = Math.Pow(iry > 0 ? iry : -iry, zoning / 100f) * (1150f / Math.Pow(1150f, zoning / 100f)) * (iry > 0 ? 1f : -1f);
          input.MoveMouseTo((int)(32767.5 - mousex * 32f), (int)(mousey * 32f + 32767.5), 11);
        }
        private double Scale(double value, double min, double max, double minScale, double maxScale)
        {
          double scaled = minScale + (double)(value - min) / (max - min) * (maxScale - minScale);
          return scaled;
        }

//keyboard control
public static bool[] _Valuechanged = new bool[36], _valuechanged = new bool[36];
        public bool HomeFTG = true;
        public int keys123, keys456;
        public void Main(double mWSNunchuckStateRawJoystickX, double mWSNunchuckStateRawJoystickY, double mWSNunchuckStateRawValuesX, double mWSNunchuckStateRawValuesY, double mWSNunchuckStateRawValuesZ, bool mWSNunchuckStateC, bool mWSNunchuckStateZ,
        double mWSButtonStateIRX, double mWSButtonStateIRY, bool mWSButtonStateA, bool mWSButtonStateB, bool mWSButtonStateMinus, bool mWSButtonStateHome, bool mWSButtonStatePlus, bool mWSButtonStateOne, bool mWSButtonStateTwo, bool mWSButtonStateUp, bool mWSButtonStateDown, bool mWSButtonStateLeft, bool mWSButtonStateRight, double mWSRawValuesX, double mWSRawValuesY, double mWSRawValuesZ,
        float EulerAnglesX, float EulerAnglesY, float EulerAnglesZ, float DirectAnglesX, float DirectAnglesY, float DirectAnglesZ, double camx, double camy, float EulerAnglesLeftX, float EulerAnglesLeftY, float EulerAnglesLeftZ, float DirectAnglesLeftX, float DirectAnglesLeftY, float DirectAnglesLeftZ, float EulerAnglesRightX, float EulerAnglesRightY, float EulerAnglesRightZ,
        float DirectAnglesRightX, float DirectAnglesRightY, float DirectAnglesRightZ, bool LeftButtonSHOULDER_1, bool LeftButtonMINUS, bool LeftButtonCAPTURE, bool LeftButtonDPAD_UP, bool LeftButtonDPAD_LEFT, bool LeftButtonDPAD_DOWN, bool LeftButtonDPAD_RIGHT, bool LeftButtonSTICK, bool RightButtonDPAD_DOWN, bool LeftButtonSL, bool LeftButtonSR, double GetStickLeftX, double GetStickLeftY,
        bool RightButtonPLUS, bool RightButtonDPAD_RIGHT, bool RightButtonHOME, bool RightButtonSHOULDER_1, bool RightButtonDPAD_LEFT, bool RightButtonDPAD_UP, bool RightButtonSTICK, bool RightButtonSL, bool RightButtonSR,
```

```
bool RightButtonSHOULDER_2, bool LeftButtonSHOULDER_2, double GetStickRightX, double GetStickRightY, float GetAccelX, float GetAccelY, float GetAccelZ, float GetAccelRightX, float GetAccelRightY, float GetAccelRightZ, float GetAccelLeftX, float GetAccelLeftY, float GetAccelLeftZ, double watchM)
            {
               this[5] = mWSNunchuckStateRawValuesX >= 41;
               if (_Valuechanged[5] & this[5])
                  input.SendKey((Keys)S_E, 1);
               if (_Valuechanged[5] & !this[5])
                  input.SendKeyF((Keys)S_E, 1);
               this[6] = mWSNunchuckStateRawValuesX <= -41;
               if (_Valuechanged[6] & this[6])
                  input.SendKey((Keys)S_Q, 1);
               if (_Valuechanged[6] & !this[6])
                  input.SendKeyF((Keys)S_Q, 1);
               this[7] = mWSRawValuesX <= -41;
               if (_Valuechanged[7] & this[7])
                  input.SendKey((Keys)S_B, 1);
               if (_Valuechanged[7] & !this[7])
                  input.SendKeyF((Keys)S_B, 1);
               this[8] = mWSRawValuesX >= 41;
               if (_Valuechanged[8] & this[8])
                  input.SendKey((Keys)S_N, 1);
               if (_Valuechanged[8] & !this[8])
                  input.SendKeyF((Keys)S_N, 1);
               this[2] = mWSNunchuckStateZ;
               if (_Valuechanged[2] & this[2])
                  input.SendKey((Keys)S_LeftShift, 1);
               if (_Valuechanged[2] & !this[2])
                  input.SendKeyF((Keys)S_LeftShift, 1);
               this[3] = mWSNunchuckStateZ & mWSNunchuckStateC;
               if (_Valuechanged[3] & this[3])
                  input.SendKey(Keys.Control, 1);
               if (_Valuechanged[3] & !this[3])
                  input.SendKeyF(Keys.Control, 1);
               this[4] = (mWSNunchuckStateRawValuesY > 33f) & !((mWSRawValuesZ > 0 ?
mWSRawValuesZ : -mWSRawValuesZ) >= 40f & (mWSRawValuesY > 0 ?
mWSRawValuesY : -mWSRawValuesY) >= 40f & (mWSRawValuesX > 0 ?
mWSRawValuesX : -mWSRawValuesX) >= 40f);
               if (_Valuechanged[4] & this[4])
                  input.SendKey(Keys.V, 1);
               if (_Valuechanged[4] & !this[4])
                  input.SendKeyF(Keys.V, 1);
               this[10] = mWSNunchuckStateC;
               if (_Valuechanged[10] & this[10])
                  input.SendKey((Keys)S_Space, 1);
               if (_Valuechanged[10] & !this[10])
                  input.SendKeyF((Keys)S_Space, 1);
               this[16] = mWSNunchuckStateRawJoystickX > 33f;
               this[17] = mWSNunchuckStateRawJoystickX < -33f;
```

```
            this[18] = mWSNunchuckStateRawJoystickY > 33f;
            this[19] = mWSNunchuckStateRawJoystickY < -33f;
            if (_Valuechanged[16] & this[16])
               input.SendKey((Keys)S_D, 1);
            if (_Valuechanged[16] & !this[16])
               input.SendKeyF((Keys)S_D, 1);
            if (_Valuechanged[17] & this[17])
               input.SendKey((Keys)S_A, 1);
            if (_Valuechanged[17] & !this[17])
               input.SendKeyF((Keys)S_A, 1);
            if (_Valuechanged[18] & this[18])
               input.SendKey((Keys)S_W, 1);
            if (_Valuechanged[18] & !this[18])
               input.SendKeyF((Keys)S_W, 1);
            if (_Valuechanged[19] & this[19])
               input.SendKey((Keys)S_S, 1);
            if (_Valuechanged[19] & !this[19])
               input.SendKeyF((Keys)S_S, 1);
            this[13] = (mWSRawValuesZ > 0 ? mWSRawValuesZ : -mWSRawValuesZ) >=
40f & (mWSRawValuesY > 0 ? mWSRawValuesY : -mWSRawValuesY) >= 40f &
(mWSRawValuesX > 0 ? mWSRawValuesX : -mWSRawValuesX) >= 40f;
            if (_Valuechanged[13] & this[13])
               input.SendKey((Keys)S_R, 1);
            if (_Valuechanged[13] & !this[13])
               input.SendKeyF((Keys)S_R, 1);
            this[20] = mWSButtonStateOne;
            if (_Valuechanged[20] & this[20])
               input.SendKey((Keys)S_Tab, 1);
            if (_Valuechanged[20] & !this[20])
               input.SendKeyF((Keys)S_Tab, 1);
            this[21] = mWSButtonStateDown;
            if (_Valuechanged[21] & this[21])
               input.SendKey((Keys)S_C, 1);
            if (_Valuechanged[21] & !this[21])
               input.SendKeyF((Keys)S_C, 1);
            this[22] = mWSButtonStateHome;
            if (_Valuechanged[22] & this[22])
               input.SendKey((Keys)S_F, 1);
            if (_Valuechanged[22] & !this[22])
               input.SendKeyF((Keys)S_F, 1);
            this[23] = mWSButtonStateRight;
            if (_Valuechanged[23] & this[23])
            {
               input.SendKey((Keys)S_U, 1);
                     if (keys123 == 0)
                        input.SendKey(Keys.One, 1);
                     if (keys123 == 1)
                        input.SendKey(Keys.Two, 1);
                     if (keys123 == 2)
                        input.SendKey(Keys.Three, 1);
```

```
}
if (_Valuechanged[23] & !this[23])
{
   input.SendKeyF((Keys)S_U, 1);
            if (keys123 == 0)
            {
               input.SendKeyF(Keys.One, 1);
               keys123 = 1;
            }
            else
               if (keys123 == 1)
               {
                     input.SendKeyF(Keys.Two, 1);
                     keys123 = 2;
               }
               else
                  if (keys123 == 2)
                     {
                        input.SendKeyF(Keys.Three, 1);
                        keys123 = 0;
                     }
}
this[24] = mWSButtonStateLeft;
if (_Valuechanged[24] & this[24])
{
   input.SendKey((Keys)S_Y, 1);
            if (keys456 == 0)
              input.SendKey(Keys.Four, 1);
            if (keys456 == 1)
              input.SendKey(Keys.Five, 1);
            if (keys456 == 2)
              input.SendKey(Keys.Six, 1);
}
if (_Valuechanged[24] & !this[24])
{
   input.SendKeyF((Keys)S_Y, 1);
            if (keys456 == 0)
            {
               input.SendKeyF(Keys.Four, 1);
               keys456 = 1;
            }
            else
               if (keys456 == 1)
               {
                     input.SendKeyF(Keys.Five, 1);
                     keys456 = 2;
               }
               else
                  if (keys456 == 2)
                     {
```

```
                    input.SendKeyF(Keys.Six, 1);
                    keys456 = 0;
                }
            }
            this[25] = mWSButtonStateUp;
            if (_Valuechanged[25] & this[25])
                input.SendKey((Keys)S_X, 1);
            if (_Valuechanged[25] & !this[25])
                input.SendKeyF((Keys)S_X, 1);
            this[26] = mWSButtonStateTwo;
            if (_Valuechanged[26] & this[26])
                input.SendKey((Keys)S_Escape, 1);
            if (_Valuechanged[26] & !this[26])
                input.SendKeyF((Keys)S_Escape, 1);
            this[14] = mWSButtonStatePlus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[14] & this[14])
                input.SendKey((Keys)S_G, 1);
            if (_Valuechanged[14] & !this[14])
                input.SendKeyF((Keys)S_G, 1);
            this[15] = mWSButtonStateMinus | (HomeFTG & mWSButtonStateHome);
            if (_Valuechanged[15] & this[15])
                input.SendKey((Keys)S_T, 1);
            if (_Valuechanged[15] & !this[15])
                input.SendKeyF((Keys)S_T, 1);
            this[11] = mWSButtonStateB;
            if (_Valuechanged[11] & this[11])
                input.SendLeftClick(11);
            if (_Valuechanged[11] & !this[11])
                input.SendLeftClickF(11);
            this[34] = mWSButtonStateA;
            if (_Valuechanged[34] & this[34])
                input.SendRightClick(11);
            if (_Valuechanged[34] & !this[34])
                input.SendRightClickF(11);
        }
```

4. Conclusion

• This book can be used as shortcut when you want play PC games with keyboard and mouse events. The configurations presented are the same on all computers. It corresponds to a lot of game genre because a lot of games are considerate with their own configuration. So it becomes easy for you to make the same in other games. Other explanations are available with the book “The Best of Fun with Chemistry and Gaming” by the same author.

5. Use and Agreement Contract

Owner: Michael Andre Franiatte.

Contact: michael.franiatte@gmail.com.

Owning: All works from scratch of the owner.

Proof of Owning: Works published, and writings/speakings all over.

Requirements of Use: Pay the owner, quote the owner, agreement of the owner.

Availability of Works: Only under the shapes of the owner built, only for personal use.

Subjects of Claims: Works published by the owner on Google Play and Google Books.

Concerning Author Rights: Equations and codes from scratch of the owner, softwares built from it, all things of people arising from it.

End User License Agreement: A commercial license is required to use in personal manner. Do not redistributing in any manner, including by computer media, a file server, an email attachment, etc. Do not embedding in or linking it to another programs, source codes and assistances including internal applications, scripts, batch files, etc. Do not use for any kind of technical support including on customer or retailer computer, hardware or software development, research, discovery, teachery, talk, speech, write, etc. Do not use for win money or for commercialisation of any products arising from my programs, source codes and assistances. Do not use and do not copy the way it run in other programs, source codes and assistances. Do not use without pay me, quote me and my agreement. Do not steal or copy or reproduce or modify or peer or share. Do not use in other manner than personal. It stand for my programs, source codes and assistances or programs, source codes and assistances stealing or copying or reproducing or modifying or peering or sharing my programs, source codes, and assistances. If you aren't agree you shall not use.

Terms of License and Price: The present contract acceptance is required to use works of the owner and built from it in all kind of manner. The price for each user shall be defined with the owner by contacting him and this for each subject of works the owner claims. Each user shall contact the owner for asking his agreement. It can be refused by the owner depending who asking and the price defined. People don’t respecting the present contract shall not use the works of the owner.